FIELD NOTES 2026

Ingeniería de IA, sin perderse nada.

La edición esencial, para leer y para imprimir.

Los conceptos que de verdad importan para construir aplicaciones con modelos base: fundamentos, evaluación, prompts, RAG, agentes, ajuste fino, datos, inferencia y arquitectura. Claro, directo y sin relleno.

50	10	4 h	38
LECCIONES	MÓDULOS	DE LECTURA	FIGURAS

AI Engineering
Manual esencial del curso

Basado en el libro de
Chip Huyen

ANTES DE EMPEZAR

Cómo usar este manual

Este PDF contiene exactamente el mismo contenido que el sitio web: los diez módulos, las 50 lecciones, las evaluaciones y los ejercicios de la edición esencial. Está pensado para leer sin conexión, estudiar en papel o imprimir solo los módulos que necesites.

SI LEES EN PANTALLA

Usa el índice del manual para saltar a cualquier lección. Los títulos del PDF también aparecen en el panel de marcadores de tu lector, así que puedes navegar por módulos sin perder la página.

SI IMPRIMES

El manual está maquetado en A4, con márgenes amplios y figuras que no se cortan. Puedes imprimir todo el libro o seleccionar un módulo completo. La lista de progreso del final sirve para marcar lo estudiado.

El recorrido

Los módulos siguen el ciclo de vida de una aplicación de IA: entender qué es un modelo base, elegir uno, evaluarlo, diseñar prompts, conectar datos con RAG, construir agentes, ajustar el modelo, preparar los datos, optimizar la inferencia y terminar con la arquitectura y el producto. Cada módulo condensa lo imprescindible en cuatro lecciones y una evaluación.

No es necesario memorizar cada cifra. Lo importante es quedarse con las decisiones de ingeniería: qué medir, qué comparar, cuándo ajustar y qué coste tiene cada alternativa. Si necesitas más detalle en algún tema, el curso completo del mismo autor sirve como referencia ampliada.

CÓMO ESTUDIAR CADA LECCIÓN

- Lee primero el resumen «En esta lección».
- Recorre el texto completo sin tomar notas exhaustivas.
- Vuelve a las figuras y a las ideas que cambien tu forma de decidir.
- Haz los ejercicios al cerrar el módulo.

LAS EVALUACIONES

Cada módulo termina con tres preguntas y una sección de ejercicios. En el sitio, la evaluación se considera superada con dos respuestas correctas de tres. Aquí encontrarás las preguntas y, en las páginas siguientes, las respuestas razonadas para comprobar tu criterio.

No hace falta una GPU ni entrenar un modelo desde cero. El curso se centra en construir sobre modelos existentes y en las decisiones que hacen que una aplicación funcione.

Ruta recomendada

No hay una única forma de recorrerlo, pero este orden funciona bien si quieres construir algo real al terminar. Las horas son una estimación de lectura, no un requisito.

FASE	MÓDULOS	QUÉ CONSIGUES	TIEMPO
Fundamentos	01–03	Entender los modelos base, saber evaluarlos y elegir con criterio.	1.1 h
Construcción	04–07	Diseñar prompts, montar RAG, construir agentes y ajustar modelos.	1.5 h
Producción	08–10	Preparar datos, optimizar inferencia y cerrar la arquitectura.	1.1 h

Qué vas a encontrar

CONTENIDO

- 50 lecciones en 10 módulos
- 30 ejercicios aplicados
- 10 evaluaciones con explicaciones
- Glosario de 31 términos

APÉNDICES

- Índice con números de página
- Glosario de referencia
- Lista de progreso imprimible
- Nota sobre la edición y las fuentes

Consejo: si estás empezando, no saltes del módulo 1 al 6. Los conceptos de evaluación y de prompt se apoyan en los fundamentos de los primeros módulos.

MAPA DEL CURSO

Índice

Diez módulos, 50 lecciones, diez evaluaciones y tres apéndices. Los números remiten a la página impresa.

01 **Introducción a la ingeniería de IA** 6

- 01.1 Del modelo de lenguaje al modelo base 7
- 01.2 El stack de ingeniería de IA 10
- 01.3 Elegir un caso de uso que funcione 13
- 01.4 Expectativas, mantenimiento y producción 16
- 01.5 Evaluación 19

02 **Cómo funcionan los modelos base** 21

- 02.1 Datos, tokens y autosupervisión 22
- 02.2 La arquitectura transformer 25
- 02.3 Escala, parámetros y postentrenamiento 28
- 02.4 Muestreo, probabilidad y salidas estructuradas 31
- 02.5 Evaluación 34

03 **Metodología de evaluación** 36

- 03.1 Por qué evaluar IA es distinto 37
- 03.2 Evaluación exacta y semántica 40
- 03.3 La IA como juez 43
- 03.4 Benchmarks y selección de modelos 46
- 03.5 Evaluación 49

04 **Evaluar sistemas de IA** 51

- 04.1 Qué hace buena a una aplicación 52
- 04.2 Instrucciones, coste y latencia 55
- 04.3 Construir o comprar 57
- 04.4 Diseñar un pipeline de evaluación 59
- 04.5 Evaluación 61

05 **Ingeniería de prompts** 63

- 05.1 Anatomía de un prompt 64
- 05.2 Instrucciones claras y contexto suficiente 66
- 05.3 Iterar, evaluar y versionar prompts 69
- 05.4 Ataques y defensas 72
- 05.5 Evaluación 75

06 RAG y agentes 77
06.1 Qué es RAG y cómo se monta 78
06.2 Recuperación: dispersa, densa e híbrida 81
06.3 Agentes: herramientas, planificación y memoria 84
06.4 Evaluar y optimizar RAG y agentes 87
06.5 Evaluación 90

07 Ajuste fino 92
07.1 Qué es el ajuste fino y cuándo usarlo 93
07.2 Memoria, PEFT y LoRA 95
07.3 Tácticas de entrenamiento 97
07.4 Evaluar y desplegar un modelo ajustado 100
07.5 Evaluación 103

08 Ingeniería de datasets 105
08.1 Calidad antes que cantidad 106
08.2 Adquisición, anotación y catálogos 109
08.3 Datos sintéticos y destilación 111
08.4 Procesar el dataset 113
08.5 Evaluación 116

09 Optimización de inferencia 118
09.1 Qué significa optimizar la inferencia 119
09.2 Métricas: latencia, throughput y memoria 122
09.3 Aceleradores, cuantización y serving 125
09.4 Optimizar coste y arquitectura 128
09.5 Evaluación 131

10 Arquitectura y feedback 133
10.1 Los primeros pasos de la arquitectura 134
10.2 Caché, monitorización y observabilidad 137
10.3 Orquestación y feedback conversacional 140
10.4 Diseñar feedback y sus límites 143
10.5 Evaluación 146

A Apéndices
Glosario 148
Lista de progreso 150
Colofón 153

01

Introducción a la ingeniería de IA

Qué cambió con los modelos base y cómo se construye con ellos

01.1 Del modelo de lenguaje al modelo base

01.2 El stack de ingeniería de IA

01.3 Elegir un caso de uso que funcione

01.4 Expectativas, mantenimiento y producción

01.5 Cierre del módulo 01

MÓDULO 01 · LECCIÓN 1 DE 4

Del modelo de lenguaje al modelo base

3 MIN DE LECTURA 593 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- De la estadística al aprendizaje
- Qué añade un modelo base
- Por qué importa para un ingeniero
- Del modelo al producto

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Los modelos base no aparecieron de la nada. Son el resultado de décadas de trabajo en modelos de lenguaje, datos y cómputo. Entender esa cadena evita tratar la IA como una caja negra.



Figura 1-1. Un ejemplo de cómo tokeniza GPT-4 una frase.

De la estadística al aprendizaje

Un modelo de lenguaje asigna probabilidad a secuencias de texto. Los primeros enfoques contaban palabras y secuencias; después llegaron los modelos neuronales, capaces de representar el contexto de forma más rica. El cambio decisivo fue entrenar con grandes volúmenes de texto sin etiquetas: el propio texto proporciona la señal de aprendizaje. A esto se le llama autosupervisión.

La autosupervisión elimina el cuello de botella del etiquetado manual. En lugar de que una persona marque cada ejemplo, el modelo aprende a predecir el siguiente token o a rellenar un hueco. Con suficientes datos y capacidad, ese objetivo aparentemente simple produce representaciones útiles para muchas tareas.

Qué añade un modelo base

Un modelo base es un modelo grande, preentrenado con datos amplios y diseñado para adaptarse a tareas muy distintas. No está entrenado para una única aplicación: sirve como punto de partida. Esa adaptabilidad es lo que separa un modelo base de un modelo de lenguaje especializado.

La adaptación puede hacerse con prompts, con recuperación de información, con ajuste fino o con una combinación. El ingeniero de IA no suele entrenar el modelo desde cero; decide cómo usarlo,

cómo evaluarlo y cómo integrarlo en un producto. El valor se desplaza del entrenamiento a la selección, la evaluación y la arquitectura.

Por qué importa para un ingeniero

La cadena de avances explica las capacidades y los límites actuales. Los modelos son buenos generalizando porque han visto mucha diversidad, pero también pueden inventar, repetir sesgos o fallar en tareas que requieren verificación. Saber de dónde vienen ayuda a elegir qué esperar y qué comprobar.

También explica por qué el campo cambia tan rápido. Mejoras en datos, arquitectura, postentrenamiento e inferencia alteran el coste y la calidad de las aplicaciones. Un ingeniero de IA necesita entender el sistema completo, no solo la API que llama.

Del modelo al producto

Un modelo base no es un producto. Entre ambos hay una cadena de decisiones: qué tarea resolver, qué contexto dar, cómo evaluar la salida y qué hacer cuando el modelo falla. La misma capacidad puede dar lugar a un asistente excelente o a una aplicación frustrante según esas decisiones.

Por eso la ingeniería de IA no consiste en esperar al siguiente modelo, sino en construir el sistema que lo rodea. El modelo aporta una capacidad general; el producto aporta el problema concreto, los datos, las restricciones y la confianza. Entender esa diferencia evita atribuir al modelo todos los méritos y todos los fallos.

IDEAS CLAVE

- La autosupervisión permite aprender de texto sin etiquetas y fue clave para escalar los modelos de lenguaje.
- Un modelo base es un modelo preentrenado y adaptable, no una aplicación terminada.
- La ingeniería de IA se centra en adaptar, evaluar e integrar modelos, más que en entrenarlos desde cero.
- Entender la evolución del modelo ayuda a interpretar sus capacidades y sus fallos.

EJEMPLO

Una empresa quiere clasificar correos de soporte. En lugar de entrenar un clasificador desde cero, puede usar un modelo base con instrucciones y ejemplos, medir su precisión en un conjunto etiquetado y ajustar el prompt o el modelo solo si la evaluación lo justifica.

CUIDADO

No confundas «modelo grande» con «modelo mejor para tu tarea». El tamaño ayuda, pero la calidad de los datos, la evaluación y el diseño del sistema deciden el resultado.

MÓDULO 01 · LECCIÓN 2 DE 4

El stack de ingeniería de IA

3 MIN DE LECTURA 540 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Las capas del sistema
- El papel del ingeniero de IA
- Construir, comprar o combinar
- El stack cambia con el producto

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Construir con modelos base es un trabajo de sistema. La aplicación, el modelo, los datos y la infraestructura se condicionan entre sí. El stack es el mapa de esas dependencias.

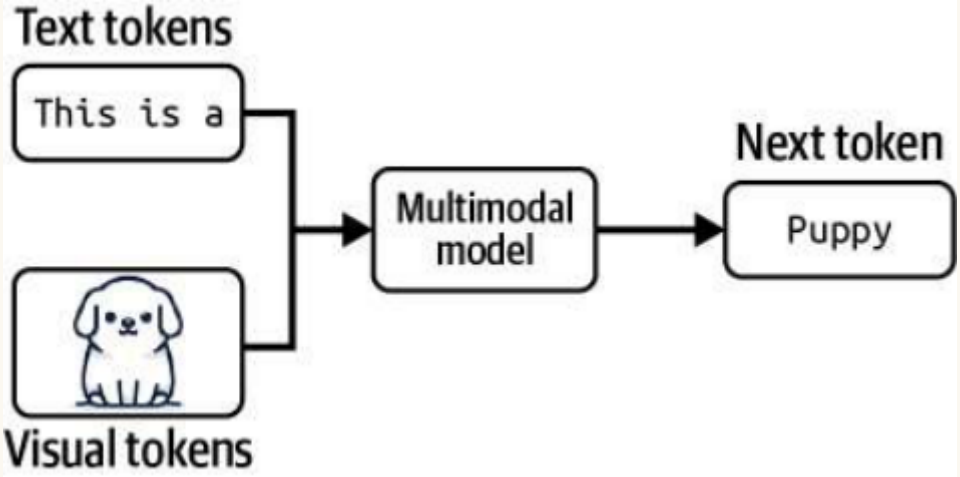


Figura 1-3. Un modelo multimodal puede generar el siguiente token usando información tanto de tokens de texto como visuales.

Las capas del sistema

En la parte superior está la aplicación: la interfaz, las instrucciones, la lógica de negocio y la experiencia de usuario. Debajo está el modelo o los modelos que generan, clasifican o recuperan información. Los datos incluyen el corpus de recuperación, los ejemplos de evaluación y los datos de ajuste. La infraestructura aporta cómputo, almacenamiento, latencia y observabilidad.

Cada capa tiene costes y límites. Un prompt mejor puede compensar un modelo mediocre; un buen modelo no arregla una recuperación deficiente; y una arquitectura sin monitorización no detecta regresiones. El ingeniero de IA trabaja en las fronteras entre capas.

El papel del ingeniero de IA

El ingeniero de IA diseña aplicaciones que usan modelos base. No es solo un experto en modelos ni solo un desarrollador de producto: combina criterio de evaluación, diseño de prompts, recuperación, datos, inferencia y arquitectura. Su trabajo es convertir una capacidad probabilística en un sistema útil y medible.

Eso incluye decidir cuándo usar una API y cuándo autohospedar, cuándo recuperar información y cuándo ajustar, y cómo detectar fallos en producción. La disciplina es más cercana a la ingeniería de producto que a la investigación de modelos.

Construir, comprar o combinar

La mayoría de los equipos no entrena su propio modelo fundacional. Usa modelos de proveedores, modelos abiertos o una mezcla. La decisión depende de coste, latencia, privacidad, control, calidad y dependencia del proveedor. No hay una respuesta única.

Una arquitectura razonable empieza simple: un modelo, un prompt y una evaluación. Después se añaden recuperación, caché, enrutamiento o ajuste fino solo cuando los datos muestran que hace falta. La complejidad debe ganarse con evidencia.

El stack cambia con el producto

No todas las aplicaciones necesitan la misma arquitectura. Un chat interactivo prioriza latencia y disponibilidad; un proceso nocturno de clasificación prioriza coste y throughput. Una herramienta interna puede aceptar más latencia y menos controles que un sistema que toma decisiones sobre clientes.

También cambia con la madurez del equipo. Un equipo pequeño gana usando una API y una arquitectura simple; uno con infraestructura propia puede aprovechar modelos abiertos. La decisión correcta no es la más avanzada, sino la que puedes operar, medir y mantener con los recursos que tienes.

IDEAS CLAVE

- El stack incluye aplicación, modelo, datos e infraestructura; las decisiones en una capa afectan a las demás.
- El ingeniero de IA combina evaluación, prompts, datos, inferencia y producto.
- La elección entre API, modelo abierto o ajuste fino depende de coste, control, privacidad y calidad.
- Empieza con la arquitectura más simple que puedas evaluar y añade complejidad solo con evidencia.

EJEMPLO

Un asistente interno de documentación puede empezar con un modelo de API, un índice de documentos y un prompt con citas. Si la latencia o el coste se vuelven críticos, se evalúa un modelo más pequeño, caché de respuestas o autohospedaje.

CUIDADO

Adoptar la arquitectura más sofisticada antes de medir el problema suele producir más coste y más puntos de fallo, no más calidad.

MÓDULO 01 · LECCIÓN 3 DE 4

Elegir un caso de uso que funcione

3 MIN DE LECTURA 541 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Valor y viabilidad
- Defendibilidad y ventaja
- Criterios antes de construir
- Del piloto al caso de negocio

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

No todas las ideas de IA merecen un producto. Un buen caso de uso combina un problema valioso, una capacidad realista del modelo y una forma clara de comprobar si funciona.

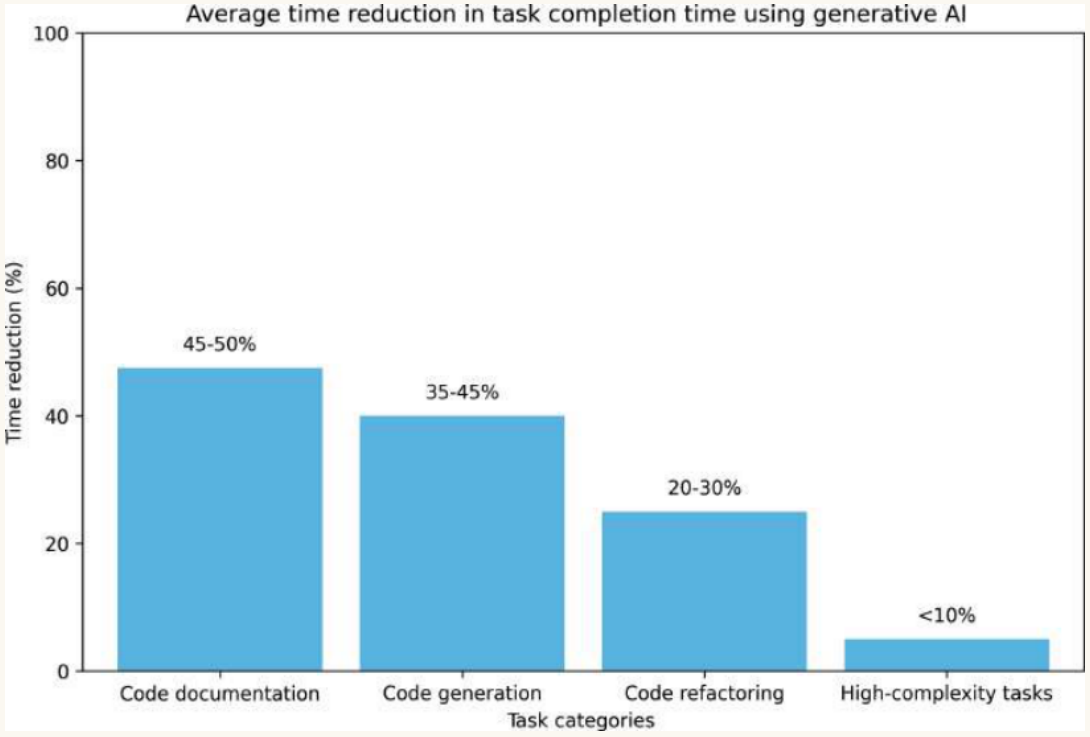


Figura 1-9. La IA puede ayudar a los desarrolladores a ser considerablemente más productivos, especialmente en tareas sencillas, pero esto se cumple menos en tareas muy complejas. Datos de McKinsey.

Valor y viabilidad

El primer filtro es el valor: ¿el problema cuesta dinero, tiempo o riesgo? El segundo es la viabilidad: ¿el modelo puede hacerlo con la calidad suficiente y a un coste razonable? Una demo impresionante no es un caso de uso; un flujo repetible con usuarios y métricas sí.

Conviene empezar por tareas con tolerancia al error y posibilidad de revisión humana. Clasificar, resumir, extraer, redactar borradores o responder sobre documentación suelen ser mejores primeros casos que decisiones autónomas de alto riesgo.

Defendibilidad y ventaja

Si cualquiera puede copiar tu prompt, tu ventaja no está en el prompt. Está en los datos propietarios, la integración con el flujo de trabajo, la confianza de la marca, la evaluación acumulada o el coste de cambio. La IA generativa reduce la barrera de entrada; la defendibilidad hay que construirla en otra parte.

También importa el contexto organizativo: quién revisa, quién asume el riesgo y cómo se mide el éxito. Un caso técnicamente posible puede fracasar si no encaja en el proceso o si nadie confía en la salida.

Criterios antes de construir

Antes de escribir código, define qué significa «funciona». Puede ser precisión en una tarea de extracción, tiempo de respuesta, reducción de tickets, satisfacción o coste por operación. Sin criterio, no hay forma de saber si el sistema mejora.

Elige una línea base honesta: el proceso actual, una regla simple o un modelo anterior. La IA debe compararse contra esa base, no contra cero. Y conviene fijar un umbral de lanzamiento y un plan para fallar de forma segura.

Del piloto al caso de negocio

Un piloto demuestra que algo es posible; un caso de negocio demuestra que merece la pena. Para cerrar esa distancia hay que estimar el coste actual del problema, el ahorro o ingreso esperado y el coste de construir y operar la solución. También hay que valorar el coste de un error y quién lo asume.

La mejor IA suele empezar en tareas repetitivas, con volumen y con revisión humana. Si el proceso es caótico, la IA no lo arregla: lo automatiza. Antes de elegir el modelo, documenta el flujo actual, la línea base y la métrica que decidirá si el proyecto sigue adelante.

IDEAS CLAVE

- Un buen caso de uso combina valor, viabilidad técnica y una métrica clara de éxito.
- Empieza por tareas con revisión humana y tolerancia al error.
- La ventaja competitiva rara vez está en el prompt; suele estar en datos, integración y confianza.
- Define una línea base y un umbral de lanzamiento antes de construir.

EJEMPLO

En soporte, un asistente que sugiere respuestas al agente humano tiene valor inmediato, es fácil de evaluar y permite medir cuánto reduce el tiempo de resolución sin automatizar la decisión final.

CUIDADO

«Usar IA» no es un objetivo. Si no puedes describir el problema, la métrica y el riesgo, todavía no tienes un caso de uso.

MÓDULO 01 · LECCIÓN 4 DE 4

Expectativas, mantenimiento y producción

3 MIN DE LECTURA 498 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El ciclo de expectativas
- Mantenimiento continuo
- Desplegar con red de seguridad
- El coste oculto de mantener IA

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

La IA generativa cambia rápido, pero un producto necesita estabilidad. Gestionar expectativas y mantener el sistema es tan importante como elegir el modelo.

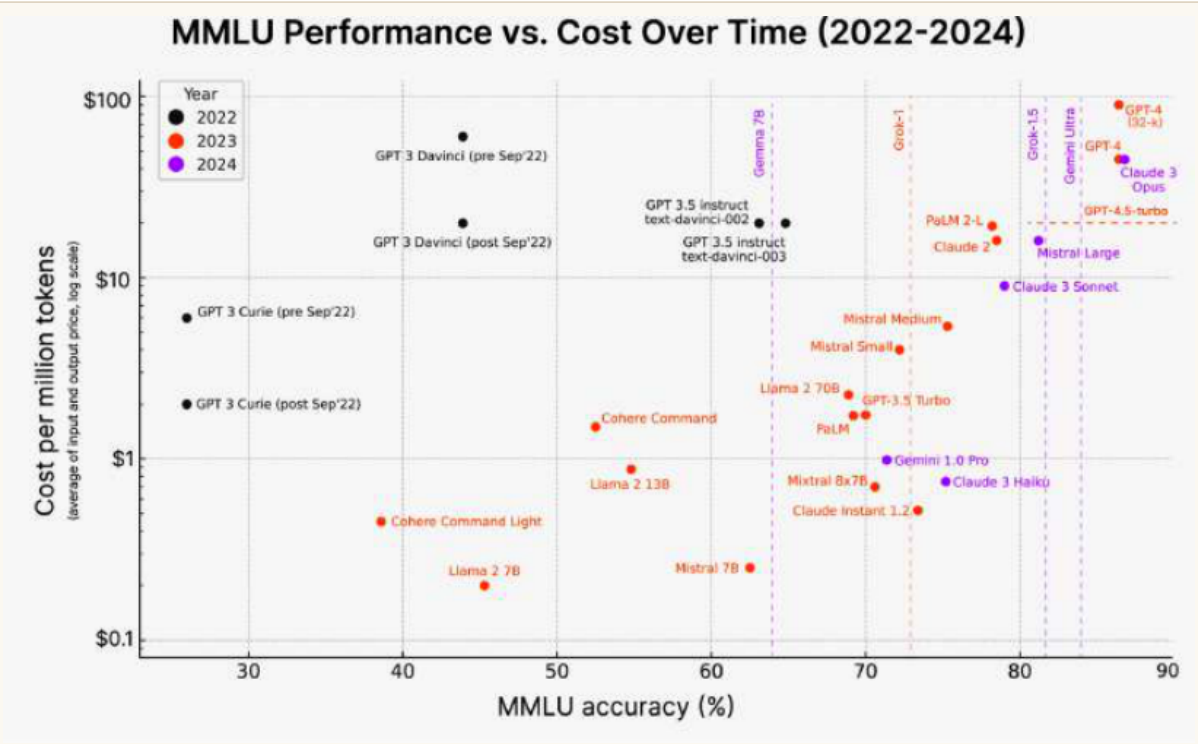


Figura 1-11. El coste del razonamiento de IA desciende rápidamente con el tiempo. Imagen de Katrina Nguyen (2024).

El ciclo de expectativas

Cada nueva capacidad genera entusiasmo, demos y promesas. Después aparece la realidad: los modelos fallan, los costes suben, los usuarios cambian de comportamiento y los proveedores ac-

tualizan versiones. El ciclo no es exclusivo de la IA, pero en IA es más rápido porque el modelo también cambia.

La respuesta no es ignorar el entusiasmo, sino convertirlo en hitos verificables. Un prototipo, una evaluación offline, una prueba con usuarios y un despliegue limitado son etapas distintas. Cada una debe responder una pregunta concreta.

Mantenimiento continuo

Un sistema de IA no termina cuando funciona en una demo. Hay que vigilar la calidad, el coste, la latencia y los sesgos; actualizar prompts y modelos; y revisar los datos de recuperación. Un cambio del proveedor puede alterar el comportamiento sin que cambie tu código.

Por eso la evaluación y la observabilidad son parte del producto. Sin ellas, no puedes distinguir una mejora de una regresión. La versión del modelo, del prompt y del conjunto de evaluación debería quedar registrada.

Desplegar con red de seguridad

Empieza con un alcance limitado y un mecanismo de reversión. Incluye revisión humana en los puntos de riesgo, limita el daño de una salida incorrecta y recoge feedback de forma estructurada. Un sistema que falla de forma visible y controlada es más útil que uno que falla en silencio.

La IA es probabilística: no se puede garantizar el mismo resultado siempre. El diseño debe asumir variación, detectar anomalías y permitir que una persona intervenga cuando haga falta.

El coste oculto de mantener IA

Mantener un sistema de IA no es solo pagar la API. Hay que revisar prompts, actualizar evaluaciones, vigilar la calidad y responder a incidentes. Un cambio del proveedor, una nueva versión del modelo o una variación en los datos pueden alterar el comportamiento sin tocar el código.

Por eso conviene tratar el sistema como un producto vivo: versiones, pruebas de regresión, registro de cambios y una persona responsable. El coste de no mantenerlo aparece como pérdida de confianza, respuestas obsoletas o fallos silenciosos. Prever ese coste desde el principio es parte de diseñar bien.

IDEAS CLAVE

- Convierte el entusiasmo en hitos verificables: prototipo, evaluación, piloto y despliegue.
- El mantenimiento incluye calidad, coste, latencia, sesgos y cambios del proveedor.
- Registra versiones de modelo, prompt y evaluación para poder explicar regresiones.
- Diseña para fallar de forma visible y con intervención humana en los puntos críticos.

EJEMPLO

Un asistente legal que resume contratos puede desplegarse primero para uso interno, con revisión obligatoria y un botón para marcar errores. Esas marcas alimentan la evaluación y permiten mejorar antes de abrirlo a clientes.

CUIDADO

Un cambio silencioso de modelo puede romper un flujo que funcionaba. Fija versiones cuando sea posible y prueba antes de actualizar.

EVALUACIÓN · MÓDULO 01

Cierre del módulo 01

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

La ingeniería de IA nace de modelos base preentrenados y adaptables. Su valor está en integrar, evaluar y operar esos modelos dentro de un producto.

El stack conecta aplicación, modelo, datos e infraestructura. La complejidad debe justificarse con métricas, no con moda.

Un caso de uso sólido tiene valor, viabilidad y una forma clara de medir el éxito. La producción exige mantenimiento, observabilidad y una red de seguridad.

Evaluación del módulo 01

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 01

¿Qué hace posible entrenar modelos de lenguaje a la escala actual?

- [ ] A. El etiquetado manual de cada ejemplo de internet
- [ ] B. La autosupervisión, que genera datos de entrenamiento sin etiquetar
- [ ] C. El uso exclusivo de datos privados de pago
- [ ] D. Los modelos de lenguaje enmascarados

Pregunta 02 · Módulo 01

¿Cuál es la diferencia central entre ML tradicional e ingeniería de IA?

- [ ] A. La ingeniería de IA no usa datos
- [ ] B. La ingeniería de IA parte de modelos base ya entrenados y construye aplicaciones encima
- [ ] C. El ML tradicional no necesita evaluación
- [ ] D. La ingeniería de IA solo sirve para chatbots

Pregunta 03 · Módulo 01

Al planificar una aplicación de IA, ¿qué conviene hacer primero?

- [ ] A. Elegir el modelo más grande disponible
- [ ] B. Evaluar el caso de uso y su defensibilidad antes de escribir código
- [ ] C. Comprar GPUs
- [ ] D. Ajustar un modelo con todos los datos de la empresa

RESPUESTAS Y EXPLICACIONES

01 · Opción B. La autosupervisión, que genera datos de entrenamiento sin etiquetar

La autosupervisión elimina el cuello de botella del etiquetado: el propio texto proporciona la señal de entrenamiento.

02 · Opción B. La ingeniería de IA parte de modelos base ya entrenados y construye aplicaciones encima

El ingeniero de IA adapta y compone modelos existentes; no suele entrenar el modelo desde cero.

03 · Opción B. Evaluar el caso de uso y su defensibilidad antes de escribir código

El capítulo insiste en evaluar el caso de uso, las expectativas y la defensibilidad del producto antes de invertir en implementación.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Elige una aplicación de IA que uses a diario. Escribe qué modelo base crees que hay detrás, qué tareas automatiza y qué parte sigue siendo humana.
2. Haz una lista de tres casos de uso de IA en tu sector. Para cada uno, anota el beneficio esperado, el riesgo principal y cómo medirías el éxito.
3. Dibuja el stack de tu primera aplicación de IA: modelo, herramienta de prompting, datos, interfaz y evaluación. Marca qué capa dominas y cuál tendrías que aprender.

02

Cómo funcionan los modelos base

Datos, transformer, escala, postentrenamiento y muestreo

02.1 Datos, tokens y autosupervisión

02.2 La arquitectura transformer

02.3 Escala, parámetros y postentrenamiento

02.4 Muestreo, probabilidad y salidas estructuradas

02.5 Cierre del módulo 02

MÓDULO 02 · LECCIÓN 1 DE 4

Datos, tokens y autosupervisión

3 MIN DE LECTURA 529 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Del texto al token
- Predecir lo siguiente
- Calidad y cobertura de los datos
- Qué mirar en un tokenizador

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un modelo base aprende de datos y de un objetivo. Antes de hablar de arquitectura conviene entender qué entra, cómo se representa y qué se optimiza.

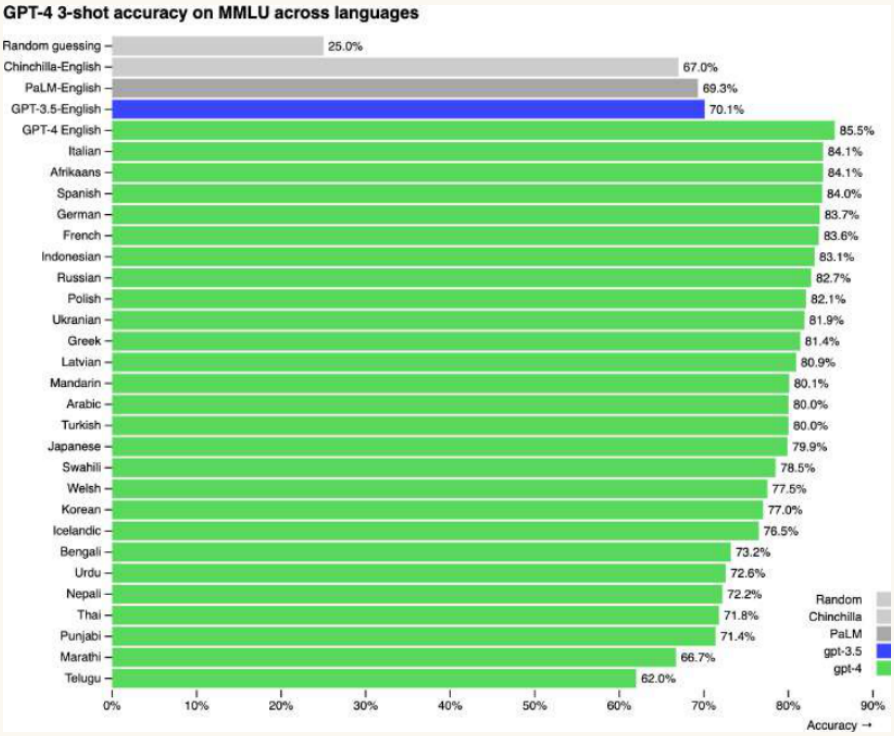


Figura 2-1. En el benchmark MMLU, GPT-4 rinde mejor en inglés que en cualquier otro idioma. Para obtener MMLU en otros idiomas, OpenAI tradujo las preguntas usando Azure AI Translator.

Del texto al token

El modelo no ve letras ni palabras completas: ve tokens. Un token puede ser un carácter, una palabra o un fragmento. Un tokenizador divide el texto en unidades y las mapea a identificadores numéricos. El conjunto de tokens posibles es el vocabulario.

La tokenización afecta al coste y a la calidad. Un vocabulario más pequeño necesita más tokens para representar el mismo texto; uno más grande puede cubrir más idiomas y dominios, pero tam-

bién introduce tokens poco frecuentes. Los modelos multilingües y los lenguajes de programación suelen necesitar vocabularios específicos.

Predecir lo siguiente

El objetivo clásico es predecir el siguiente token dada una secuencia. El modelo recibe un contexto, calcula una distribución sobre el vocabulario y se ajusta para asignar más probabilidad al token que realmente aparece. Al repetir esto sobre grandes corpus, aprende patrones de sintaxis, estilo, conocimiento y razonamiento superficial.

También existen objetivos enmascarados, que rellenan un token oculto usando el contexto a ambos lados. Los modelos autorregresivos generan hacia delante; los enmascarados suelen usarse para representaciones y clasificación. La elección depende de la tarea y del tipo de salida.

Calidad y cobertura de los datos

Más datos no garantizan mejor calidad. Importan la diversidad, la limpieza, la licencia y la representación de los idiomas y dominios que quieres cubrir. Un corpus enorme pero repetitivo o contaminado puede enseñar atajos indeseables.

La procedencia también importa. Datos personales, contenido con derechos o texto generado sin verificar plantean riesgos legales y de calidad. Un buen dataset documenta de dónde viene cada muestra y cómo se filtró.

Qué mirar en un tokenizador

El tamaño del vocabulario y la forma de segmentar afectan al coste y al rendimiento. Un tokenizador eficiente representa el texto con menos tokens, lo que reduce precio y permite incluir más contexto. Pero la eficiencia en un idioma no garantiza la misma en otro.

Antes de elegir un modelo, prueba con textos reales de tu dominio: nombres propios, jerga, código, cifras y acentos. Mide tokens por palabra y revisa si las palabras importantes se fragmentan demasiado. Un tokenizador que rompe los identificadores de tu negocio puede degradar la calidad de forma difícil de detectar.

IDEAS CLAVE

- El modelo procesa tokens, no palabras; la tokenización condiciona coste y cobertura.
- La autosupervisión usa la estructura del texto como señal, sin etiquetas manuales.
- La predicción del siguiente token es el objetivo típico de los modelos generativos.
- La diversidad, limpieza y procedencia de los datos importan tanto como la cantidad.

EJEMPLO

Si entrenas un modelo para español y portugués, un tokenizador entrenado solo con inglés fragmentará muchas palabras y gastará más tokens. Medir tokens por palabra y por idioma ayuda a detectar ese problema antes de entrenar.

CUIDADO

Un corpus grande no compensa una muestra sesgada. Si ciertos grupos o idiomas están poco representados, el modelo probablemente rendirá peor en ellos.

MÓDULO 02 · LECCIÓN 2 DE 4

La arquitectura transformer

3 MIN DE LECTURA 513 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Atención y contexto
- El bloque transformer
- Por qué escala
- Variantes de transformer

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

El transformer es la arquitectura dominante de los modelos base. Su idea central es la atención: cada token combina información de otros tokens según su relevancia.

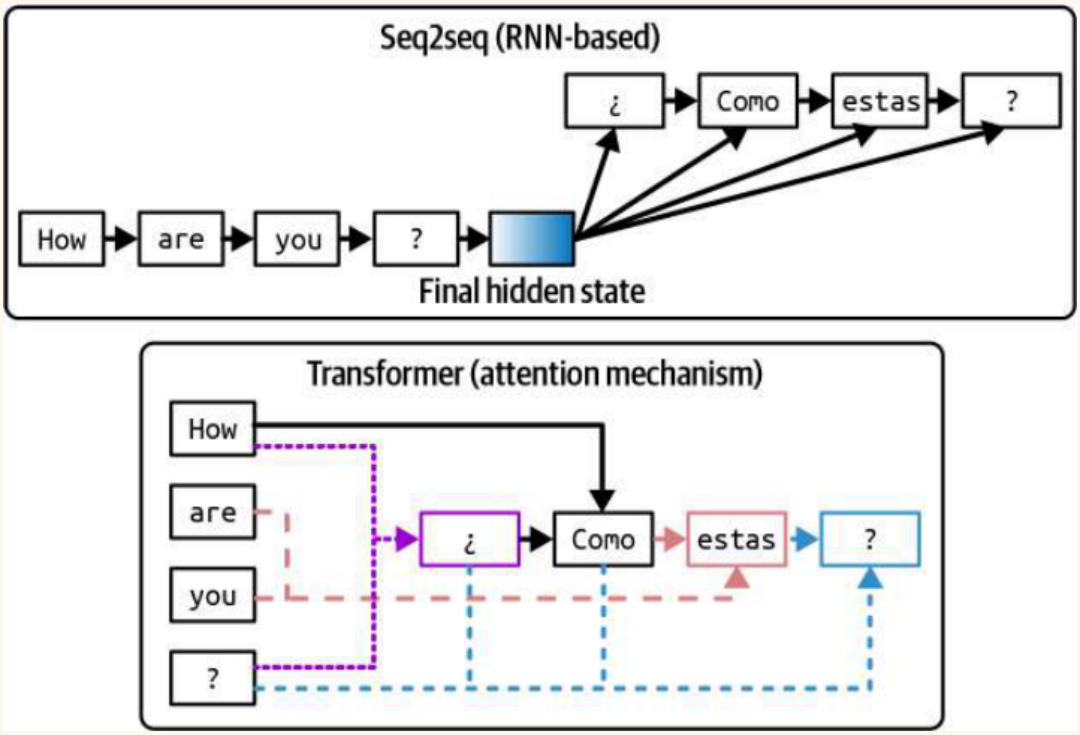


Figura 2-4. Arquitectura seq2seq frente a la arquitectura transformer. Para la arquitectura transformer, las flechas muestran los tokens a los que atiende el decodificador al generar cada token de salida.

Atención y contexto

Cada token se representa con un vector. Para actualizarlo, el modelo calcula consultas, claves y valores. La consulta de un token se compara con las claves de los demás para obtener pesos de atención; esos pesos se usan para combinar los valores. Así, una palabra puede apoyarse en otras que aparecen lejos en la secuencia.

La atención multi-cabeza repite este cálculo con varias proyecciones. Cada cabeza puede especializarse en relaciones distintas: sintaxis, referencias, patrones locales o dependencias largas. El resultado se concatena y se proyecta de nuevo.

El bloque transformer

Un bloque típico combina atención con una red feed-forward, conexiones residuales y normalización. La conexión residual facilita el flujo de gradientes y estabiliza el entrenamiento; la normalización controla la escala de las activaciones. Se apilan muchos bloques para construir representaciones cada vez más abstractas.

En los modelos generativos, la atención es causal: cada token solo puede mirar los anteriores, para no filtrar el futuro. En los modelos encoder, la atención puede ser bidireccional. Algunos modelos combinan ambas partes para tareas de secuencia a secuencia.

Por qué escala

La atención permite paralelizar el entrenamiento mucho mejor que las redes recurrentes, porque no depende de un estado secuencial. Esa eficiencia hizo posible entrenar con más datos y más parámetros. A cambio, el coste crece con la longitud del contexto: la atención compara cada token con los demás.

La arquitectura no lo explica todo. La calidad depende también de los datos, del postentrenamiento, de la optimización y del hardware. El transformer es una base eficaz, no una garantía de inteligencia.

Variantes de transformer

No todos los transformers son iguales. Los modelos encoder, como BERT, miran el contexto en ambas direcciones y son útiles para clasificar o representar texto. Los decoder, como GPT, generan de izquierda a derecha y dominan la conversación. Los encoder-decoder, como T5, están pensados para transformar una secuencia en otra.

También existen variantes eficientes: atención dispersa, mezcla de expertos o arquitecturas híbridas. Cada una cambia el coste y el tipo de tarea para la que brilla. Conocer la familia ayuda a interpretar la ficha del modelo y a no pedirle algo para lo que no está diseñado.

IDEAS CLAVE

- La atención combina información de otros tokens mediante consultas, claves y valores.
- Varias cabezas permiten capturar relaciones distintas en paralelo.
- Los bloques apilan atención, feed-forward, residuales y normalización.
- La atención causal impide que un token generativo vea el futuro.

EJEMPLO

En la frase «El banco aprobó el préstamo porque tenía liquidez», la atención ayuda a relacionar «tenía» con «banco» y no con «préstamo». Esa desambiguación depende del contexto.

CUIDADO

Un contexto más largo no siempre significa mejor comprensión. El modelo puede perder información relevante en el medio o gastar más cómputo sin mejorar la respuesta.

MÓDULO 02 · LECCIÓN 3 DE 4

Escala, parámetros y postentrenamiento

3 MIN DE LECTURA 537 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Parámetros y leyes de escala
- Postentrenamiento
- Qué conserva y qué pierde
- Cómo leer la ficha de un modelo

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un modelo preentrenado no es todavía un buen asistente. La escala mejora capacidades, pero el postentrenamiento enseña a seguir instrucciones y a comportarse de forma útil.

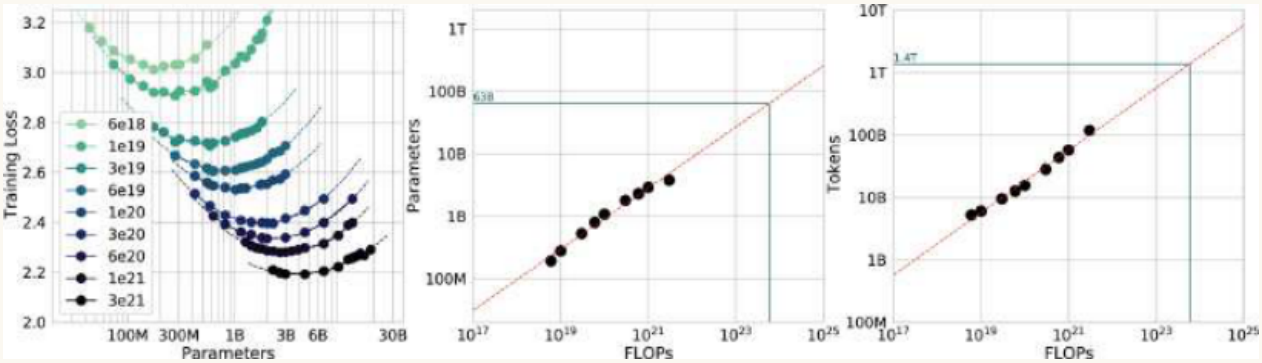


Figura 2-8. Gráficos que muestran las relaciones entre la pérdida de entrenamiento, el número de parámetros de un modelo, los FLOPs y el número de tokens de entrenamiento. Fuente: «Modelos de lenguaje grandes con cómputo óptimo para el entrenamiento» (DeepMind, 2022).

Parámetros y leyes de escala

Los parámetros son los pesos que el modelo aprende. En general, más parámetros y más datos de entrenamiento mejoran la capacidad, pero con rendimientos decrecientes y un coste creciente. Las leyes de escala describen esa relación de forma empírica: ayudan a estimar qué combinación de tamaño y datos es más eficiente para un presupuesto dado.

El tamaño puede ser engañoso. Un modelo disperso o de mezcla de expertos activa solo una parte de sus parámetros por token, así que su coste de inferencia no crece igual que su tamaño total. Un modelo pequeño bien entrenado puede superar a uno grande mal entrenado en una tarea concreta.

Postentrenamiento

El preentrenamiento produce un modelo que continúa texto, no necesariamente un asistente que responde. El postentrenamiento añade instrucciones, ejemplos y preferencias. El ajuste supervisado enseña a seguir instrucciones con pares de entrada y salida deseada.

Después, las técnicas de preferencia —como RLHF o variantes más simples— usan comparaciones humanas para alinear las respuestas con lo que los usuarios valoran: utilidad, seguridad, claridad y ausencia de daño. El resultado es un modelo más obediente, pero también más condicionado por las decisiones de sus creadores.

Qué conserva y qué pierde

El postentrenamiento puede mejorar el seguimiento de instrucciones y reducir respuestas tóxicas, pero también puede estrechar el estilo, aumentar la cautela o introducir sesgos de anotación. No elimina el conocimiento del preentrenamiento; lo reorganiza y lo condiciona.

Por eso conviene evaluar el modelo final, no solo el base. Dos modelos con el mismo preentrenamiento pueden comportarse de forma muy distinta tras el postentrenamiento.

Cómo leer la ficha de un modelo

La ficha de un modelo reúne más información que el número de parámetros: tamaño del contexto, fecha de corte del conocimiento, licencia, idiomas, modalidades y coste. También indica si es denso o disperso, si está cuantizado y qué benchmarks reporta.

Ninguno de esos datos basta por separado. Un contexto enorme no sirve si el modelo se pierde en el medio; una licencia abierta no sirve si no puedes servir el modelo. Lee la ficha como una hipótesis y confírmala con tu propia evaluación antes de comprometerte.

IDEAS CLAVE

- Más parámetros y datos suelen mejorar la capacidad, con rendimientos decrecientes y mayor coste.
- Los modelos dispersos o MoE activan solo una parte de los parámetros por token.
- El ajuste supervisado enseña a seguir instrucciones; las preferencias alinean el comportamiento.
- El postentrenamiento cambia el estilo y la seguridad, y puede introducir sesgos.

EJEMPLO

Un modelo base puede completar «La capital de Francia es» con «París». Un modelo postentrenado, además, puede responder a «Explícame por qué París es la capital» siguiendo el formato y el tono que pide el usuario.

CUIDADO

No compares modelos solo por su número de parámetros. Importan los datos, el postentrenamiento, la cuantización y el coste real de inferencia.

MÓDULO 02 · LECCIÓN 4 DE 4

Muestreo, probabilidad y salidas estructuradas

3 MIN DE LECTURA 470 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- De logits a probabilidades
- Qué significan los logprobs
- Salidas estructuradas
- Reproducibilidad y control

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

La generación es probabilística. El modelo calcula una distribución sobre el siguiente token y una estrategia de muestreo decide cuál elegir. Esa decisión afecta a creatividad, consistencia y formato.

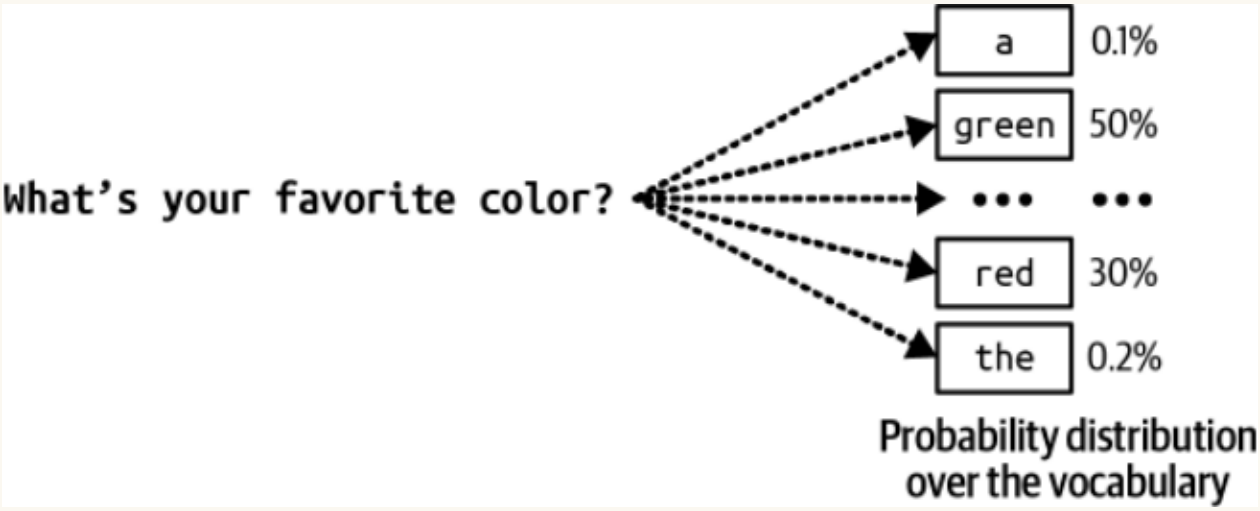


Figura 2-14. Para generar el siguiente token, el modelo de lenguaje calcula primero la distribución de probabilidad sobre todos los tokens del vocabulario.

De logits a probabilidades

La última capa produce un valor —logit— para cada token del vocabulario. La función softmax convierte esos valores en una distribución de probabilidad. El modelo puede elegir siempre el token más probable (greedy), muestrear según la distribución o recortar la cola con top-k o top-p.

La temperatura controla la forma de la distribución. Una temperatura baja la concentra en los tokens más probables y produce respuestas más deterministas; una temperatura alta aplana la distribución y aumenta la diversidad, también el riesgo de divagación.

Qué significan los logprobs

Los logprobs indican la probabilidad asignada a los tokens elegidos. Sirven para detectar incertidumbre, comparar alternativas o puntuar la confianza de una clasificación. No son una medida de verdad: un modelo puede estar muy seguro y equivocarse.

Aun así, son útiles para umbrales y para enrutar casos ambiguos a revisión humana. Si la probabilidad de la etiqueta elegida es baja, el sistema puede pedir más información en lugar de decidir.

Salidas estructuradas

Los modelos generan texto libre, pero muchas aplicaciones necesitan JSON, tablas o campos concretos. Se puede pedir un esquema en el prompt, usar modo JSON o validar la salida con un parser. La validación es obligatoria: el modelo puede omitir campos, inventar tipos o añadir texto extra.

Una estrategia robusta combina instrucciones claras, ejemplos, esquema y una capa de validación con reintento. Si el formato es crítico, no confíes solo en que el modelo «obedezca».

Reproducibilidad y control

La generación probabilística complica la depuración. Fija la temperatura, el top-p y la semilla cuando el proveedor lo permita; guarda la versión del modelo y el prompt exacto. Aun así, dos ejecuciones pueden diferir por cambios internos del servicio.

Para reducir el riesgo, valida la salida con reglas y esquemas, y diseña reintentos con instrucciones de corrección. Si necesitas resultados muy estables, considera un modelo más determinista o una etapa de verificación. La reproducibilidad total no siempre es posible, pero la trazabilidad sí.

IDEAS CLAVE

- La generación parte de una distribución de probabilidad sobre el siguiente token.
- Temperatura, top-k y top-p controlan diversidad y determinismo.
- Los logprobs miden confianza del modelo, no verdad.
- Las salidas estructuradas necesitan esquema y validación, no solo instrucciones.

EJEMPLO

Para extraer datos de una factura, pide un JSON con campos definidos, valida el esquema y reintenta con un mensaje de error si falta un campo. Si la confianza es baja, envía la factura a revisión.

CUIDADO

Temperatura cero reduce la variación, pero no garantiza respuestas correctas ni idénticas entre versiones del modelo.

EVALUACIÓN · MÓDULO 02

Cierre del módulo 02

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

Los modelos base aprenden de tokens y datos mediante autosupervisión. La cantidad importa, pero la calidad y la cobertura deciden el rendimiento.

El transformer organiza el cálculo con atención y bloques apilados. La escala mejora capacidades, y el postentrenamiento convierte el modelo en asistente.

La generación es probabilística. El muestreo, los logprobs y la validación de formato son herramientas de ingeniería, no detalles.

Evaluación del módulo 02

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 02

¿Qué es un token?

- ☐ A. Una palabra completa, siempre
- ☐ B. La unidad mínima de texto que procesa el modelo: carácter, palabra o fragmento
- ☐ C. Una capa de la red neuronal
- ☐ D. Un parámetro del modelo

Pregunta 02 · Módulo 02

¿Qué ocurre durante el postentrenamiento?

- ☐ A. El modelo memoriza internet
- ☐ B. El modelo aprende a seguir instrucciones y a comportarse de forma útil y segura
- ☐ C. Se reduce el vocabulario
- ☐ D. Se elimina la arquitectura transformer

Pregunta 03 · Módulo 02

¿Por qué la misma pregunta puede producir respuestas distintas?

- ☐ A. Porque el modelo cambia de arquitectura en cada llamada
- ☐ B. Porque el muestreo introduce aleatoriedad al elegir el siguiente token
- ☐ C. Porque internet cambia
- ☐ D. Porque el prompt se borra

RESPUESTAS Y EXPLICACIONES

01 · Opción B. La unidad mínima de texto que procesa el modelo: carácter, palabra o fragmento

Un token puede ser un carácter, una palabra o parte de una palabra, según el tokenizador.

02 · Opción B. El modelo aprende a seguir instrucciones y a comportarse de forma útil y segura

El postentrenamiento (supervisado y de preferencias) alinea el modelo para responder a instrucciones.

03 · Opción B. Porque el muestreo introduce aleatoriedad al elegir el siguiente token

La generación es probabilística: temperatura y otras estrategias de muestreo eligen entre tokens candidatos.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Tokeniza mentalmente la frase «La ingeniería de IA construye aplicaciones» y explica por qué un modelo no ve letras ni palabras, sino tokens.
2. Compara un modelo pequeño y uno grande: escribe dos tareas donde el pequeño bastaría y dos donde necesitarías el grande. Justifica con coste y latencia.
3. Diseña un prompt y su salida esperada en formato JSON. Indica qué harías si el modelo devuelve texto extra alrededor del JSON.

03

Metodología de evaluación

Cómo medir lo que un modelo realmente sabe

03.1 Por qué evaluar IA es distinto

03.2 Evaluación exacta y semántica

03.3 La IA como juez

03.4 Benchmarks y selección de modelos

03.5 Cierre del módulo 03

MÓDULO 03 · LECCIÓN 1 DE 4

Por qué evaluar IA es distinto

3 MIN DE LECTURA 484 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El problema de la respuesta abierta
- Contaminación y generalización
- Evaluar es diseñar
- Diseñar el conjunto de evaluación

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Evaluar un modelo no es como evaluar una función que devuelve un número. Muchas respuestas pueden ser correctas, el contexto importa y los benchmarks pueden dejar de ser una medida honesta.

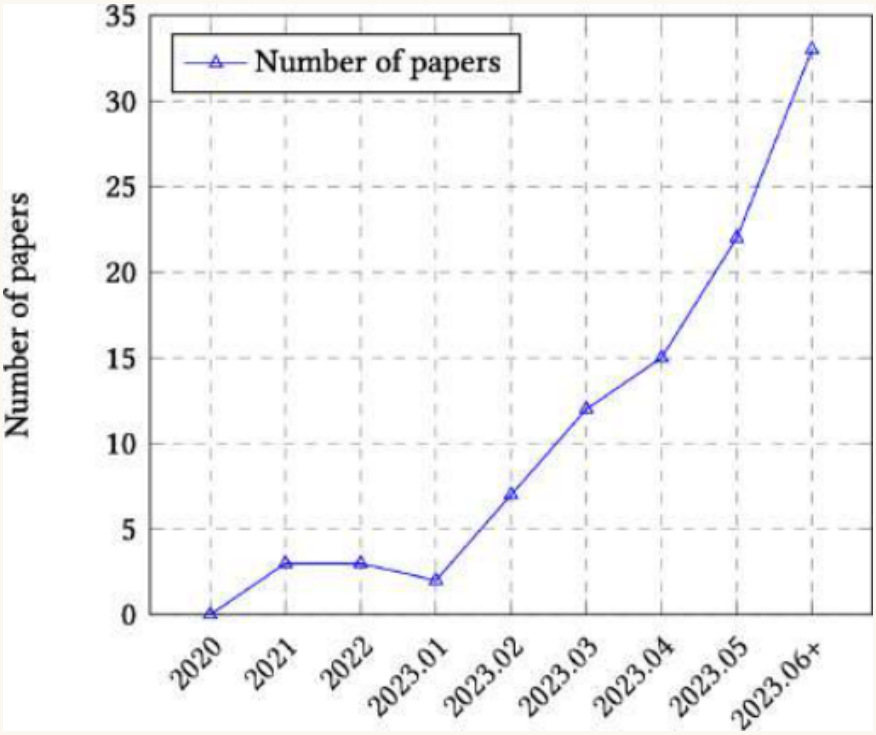


Figura 3-1. La evolución del número de artículos sobre evaluación de LLMs a lo largo del tiempo. Imagen de Chang et al. (2023).

El problema de la respuesta abierta

En una tarea de clasificación hay una etiqueta correcta. En una tarea generativa puede haber muchas respuestas válidas: distintos estilos, niveles de detalle o enfoques. Una comparación exacta con una referencia penaliza respuestas correctas que se expresan de otra forma.

Por eso la evaluación debe empezar por definir qué significa «bueno» para esa aplicación. ¿Exactitud factual, utilidad, seguridad, formato, latencia o coste? Sin criterio, cualquier métrica puede parecer razonable y ninguna responde a la pregunta.

Contaminación y generalización

Los benchmarks públicos son útiles para comparar modelos, pero pueden filtrarse en los datos de entrenamiento. Si un modelo vio las preguntas y respuestas, su puntuación mide memoria, no capacidad. También puede ajustarse al estilo del benchmark sin mejorar en el mundo real.

La evaluación de un producto necesita datos propios, representativos y actualizados. Un conjunto pequeño pero bien diseñado suele ser más informativo que un benchmark genérico con miles de ejemplos irrelevantes.

Evaluar es diseñar

La evaluación no es una fase final: es una herramienta de diseño. Define el conjunto de pruebas, los criterios y la línea base antes de optimizar. Así puedes comparar prompts, modelos o arquitecturas con evidencia, en lugar de elegir por intuición.

Una buena evaluación incluye casos fáciles, casos límite y casos adversariales. También documenta cómo se etiquetaron las respuestas y qué decisiones se tomaron en los desacuerdos.

Diseñar el conjunto de evaluación

Un buen conjunto de evaluación se construye como una muestra: define la población, estratifica por tipos de usuario y caso, y documenta cómo se seleccionó. Incluye ejemplos reales, no solo inventados. Si el conjunto es pequeño, cada caso debe ser especialmente informativo.

Separa desarrollo y prueba. Usa el primero para iterar y el segundo para decidir. Cuando un caso falla, añádelo al conjunto de desarrollo para evitar que vuelva a pasar. La evaluación se convierte así en memoria del sistema, no en una foto fija.

IDEAS CLAVE

- Las tareas generativas tienen múltiples respuestas válidas; la métrica debe reflejar el objetivo real.
- Los benchmarks públicos pueden estar contaminados o no representar tu dominio.
- La evaluación debe empezar antes de optimizar y definir una línea base.
- Incluye casos fáciles, límite y adversariales, con criterios de etiquetado documentados.

EJEMPLO

Para un asistente de soporte, «correcto» puede significar que la respuesta resuelve el problema sin inventar políticas. La evaluación combina revisión humana, comprobación de citas y una métrica de resolución, no solo similitud con una respuesta ideal.

CUIDADO

Una métrica alta en un benchmark no garantiza un buen producto. Si el benchmark no se parece a tus usuarios, estás midiendo otra cosa.

MÓDULO 03 · LECCIÓN 2 DE 4

Evaluación exacta y semántica

3 MIN DE LECTURA 472 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Métricas exactas
- Similitud semántica
- Comprobar propiedades
- Elegir la métrica según la tarea

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Hay dos familias útiles: comparar la salida con una referencia y comprobar propiedades. Cada una responde a preguntas distintas y tiene límites claros.

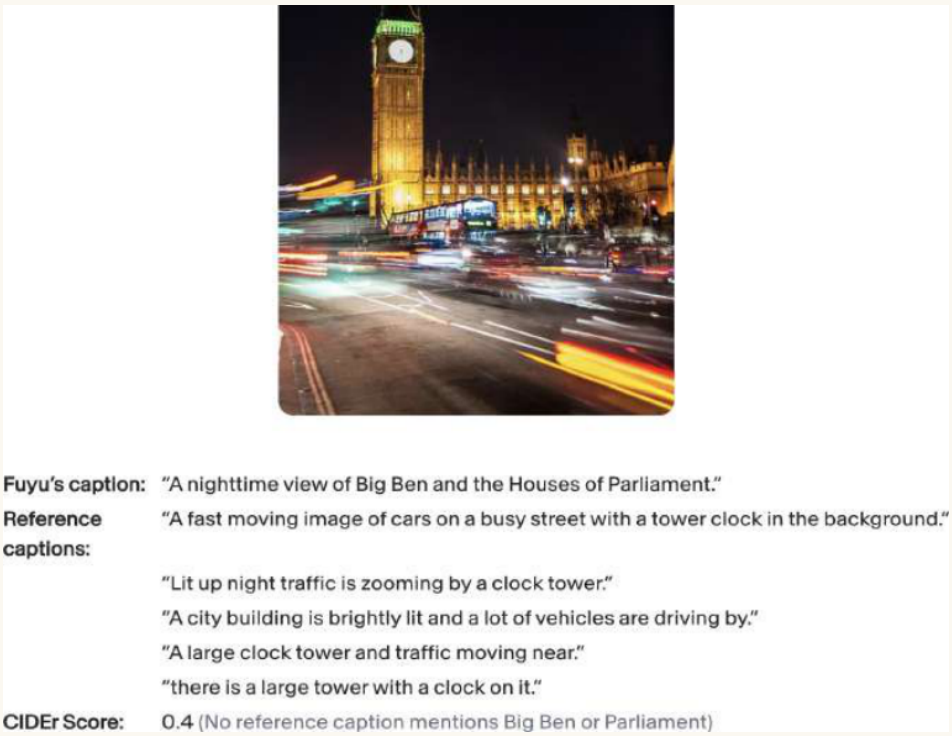


Figura 3-5. Un ejemplo en el que Fuyu generó una opción correcta pero obtuvo una puntuación baja debido a la limitación de los pies de foto de referencia.

Métricas exactas

En clasificación se usan precisión, recall, F1 y exactitud. La precisión mide cuántos positivos predichos eran correctos; el recall, cuántos positivos reales se encontraron. El F1 equilibra ambas. La exactitud global puede engañar si las clases están desbalanceadas.

En extracción o respuesta corta, la coincidencia exacta es simple pero frágil: «Madrid» y «madrid» pueden ser equivalentes, y una respuesta con una explicación adicional puede fallar. Normalizar el texto y definir variantes aceptadas mejora la medida.

Similitud semántica

Los embeddings convierten texto en vectores y permiten medir cercanía semántica. Dos respuestas pueden compartir significado sin compartir palabras. Es útil para agrupar, recuperar o detectar paráfrasis, pero no garantiza corrección factual.

Las métricas clásicas como BLEU o ROUGE comparan n-gramas y penalizan paráfrasis. Sirven para traducción o resumen cuando se calibran bien, pero no son una medida general de calidad. Para código, la corrección funcional —ejecutar tests— es más fiable que comparar texto.

Comprobar propiedades

A veces no necesitas una referencia, sino verificar una propiedad: que el JSON valide, que la cita exista, que el número cumpla un rango, que el código pase tests o que la respuesta no contenga datos personales. Estas comprobaciones son deterministas y baratas.

La mejor evaluación combina varias señales: una métrica automática para escalar, una comprobación de propiedades para lo crítico y revisión humana para lo ambiguo. Ninguna sustituye a las demás.

Elegir la métrica según la tarea

En clasificación, mide precisión y recall por clase, no solo exactitud. En extracción, comprueba campo a campo y distingue entre ausente, incorrecto y formato inválido. En generación abierta, combina una rúbrica con revisión humana. En código, ejecuta pruebas y mide casos que pasan, no similitud textual.

La métrica debe castigar los errores que importan. Si un falso positivo es más grave que un falso negativo, la exactitud global puede ocultarlo. Define primero el coste del error y elige la métrica que lo refleje.

IDEAS CLAVE

- Precisión, recall y F1 son útiles cuando hay etiquetas y clases definidas.
- La coincidencia exacta es frágil ante variaciones de formato o paráfrasis.
- Los embeddings miden cercanía semántica, no verdad factual.
- Para código y datos estructurados, validar propiedades o ejecutar tests suele ser mejor que comparar texto.

EJEMPLO

En un extractor de fechas, valida que el campo exista, que tenga formato ISO y que la fecha sea coherente. No necesitas un juez humano para cada factura; sí para los casos que fallan la validación.

CUIDADO

No uses BLEU o ROUGE como única medida de calidad conversacional. Penalizan respuestas correctas escritas de otra forma.

MÓDULO 03 · LECCIÓN 3 DE 4

La IA como juez

3 MIN DE LECTURA 469 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Formas de usar un juez
- Sesgos del juez
- Dónde encaja en el pipeline
- Calibrar un juez paso a paso

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un modelo puede evaluar respuestas a escala, pero también hereda sesgos. Usarlo como juez exige un diseño cuidadoso y una calibración contra humanos.

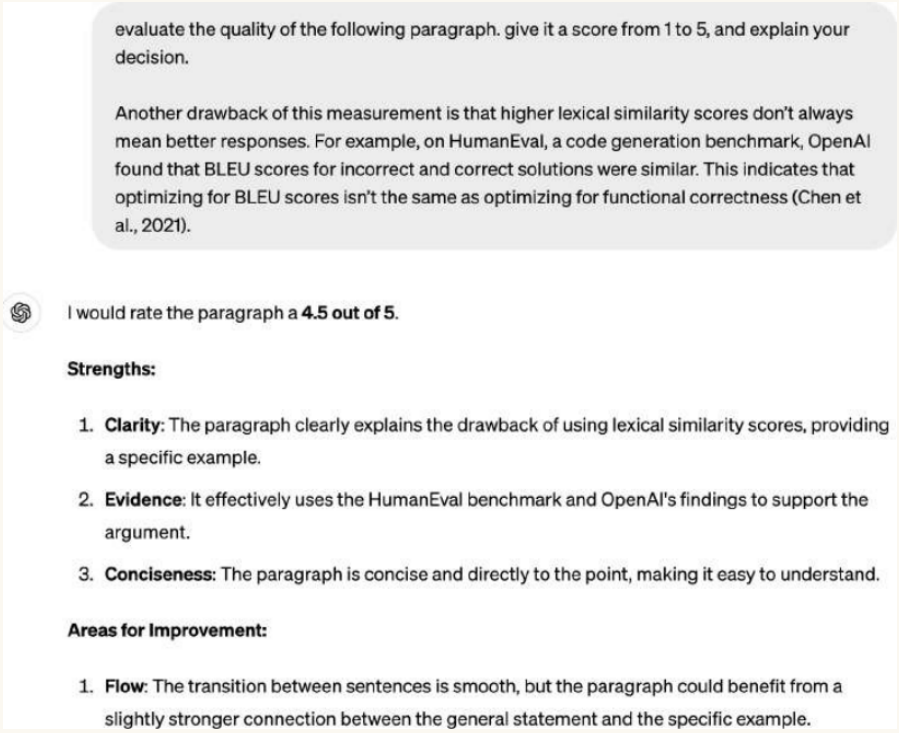


Figura 3-7. Los jueces de IA no solo pueden puntuar, también pueden explicar sus decisiones.

Formas de usar un juez

El juez puede puntuar una respuesta con una rúbrica, comparar dos respuestas y elegir la mejor, o verificar si una respuesta cumple una lista de criterios. La comparación por pares suele ser más fiable que una puntuación absoluta, porque reduce la subjetividad de la escala.

Para que funcione, el juez necesita instrucciones claras, ejemplos de referencia y un orden aleatorio de las opciones. También conviene pedir una explicación breve: ayuda a detectar por qué una decisión es sospechosa.

Sesgos del juez

Los jueces automáticos tienden a preferir respuestas largas, con formato seguro y estilo positivo. Pueden favorecer a modelos de su propia familia o verse influidos por el orden de presentación. También pueden ser inconsistentes entre ejecuciones.

La calibración consiste en comparar las decisiones del juez con etiquetas humanas en un conjunto representativo. Si la correlación es baja, el juez no sirve para decidir; puede servir para priorizar casos que revisará una persona.

Dónde encaja en el pipeline

Un juez es útil para filtrar, priorizar y detectar regresiones, no para sustituir la responsabilidad humana en decisiones sensibles. En dominios médicos, legales o financieros, la revisión humana sigue siendo necesaria.

La combinación más robusta es: métricas automáticas para lo objetivo, juez para lo semántico y humanos para calibrar y resolver desacuerdos. Documenta la versión del juez y vuelve a calibrar cuando cambie.

Calibrar un juez paso a paso

Empieza con una rúbrica pequeña y ejemplos de cada nivel. Pide al juez que compare pares antes de puntuar en absoluto. Luego compara sus decisiones con etiquetas humanas en un conjunto representativo y mide el acuerdo. Si el acuerdo es bajo, revisa la rúbrica antes de culpar al modelo.

Prueba sesgos conocidos: invierte el orden de las opciones, acorta y alarga respuestas equivalentes, cambia el estilo. Un juez que solo funciona con un formato no es fiable. Documenta la versión del juez y vuelve a calibrar cuando cambie el modelo o la tarea.

IDEAS CLAVE

- La comparación por pares suele ser más estable que la puntuación absoluta.
- Los jueces automáticos tienen sesgos de longitud, estilo, orden y familia de modelo.
- Calibra el juez contra etiquetas humanas antes de confiar en él.
- Usa el juez para escalar la revisión, no para eliminar la responsabilidad humana.

EJEMPLO

Para evaluar resúmenes, pide al juez que compare dos versiones con una rúbrica de fidelidad, cobertura y claridad. Revisa una muestra de sus decisiones y mide el acuerdo con editores humanos.

CUIDADO

Un juez puede parecer coherente y estar equivocado. Sin calibración, una puntuación alta solo significa que el juez tiene una preferencia estable.

MÓDULO 03 · LECCIÓN 4 DE 4

Benchmarks y selección de modelos

3 MIN DE LECTURA 455 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Qué hay en un benchmark
- Contaminación y saturación
- Construir tu propio benchmark
- Usar benchmarks sin engañarte

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Los benchmarks son atajos para comparar modelos, no una auditoría de tu caso de uso. Saber qué miden evita decisiones basadas en una sola cifra.

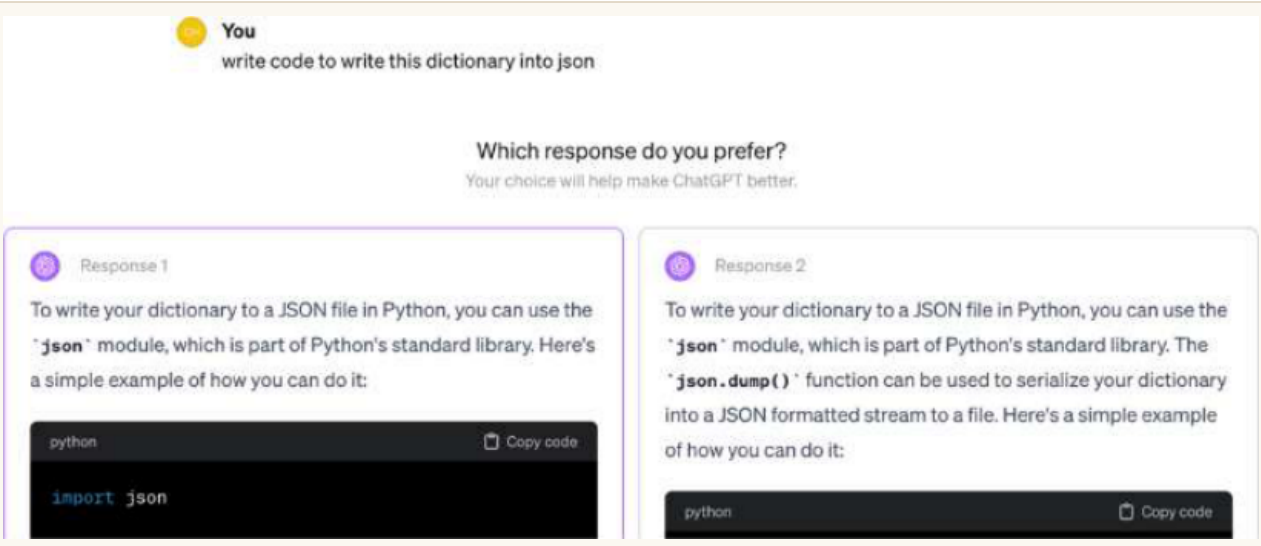


Figura 3-10. En ocasiones, ChatGPT pide a los usuarios que comparen dos salidas una al lado de la otra.

Qué hay en un benchmark

Un benchmark reúne tareas, datos y una métrica. Algunos miden conocimiento general, otros razonamiento, código, matemáticas o seguridad. MMLU, HumanEval o GSM8K son conocidos, pero cada uno cubre una parte estrecha de la capacidad.

Los leaderboards agregan resultados y a veces usan votos humanos o comparaciones por pares. Son útiles para explorar el panorama, pero no dicen cómo rendirá un modelo con tus datos, tu idioma o tus restricciones de latencia.

Contaminación y saturación

Cuando un benchmark se hace popular, los modelos tienden a entrenarse con datos similares. La contaminación infla las puntuaciones y reduce la capacidad de distinguir. Además, muchos benchmarks se saturan: casi todos los modelos grandes obtienen puntuaciones altas y las diferencias dejan de ser informativas.

Por eso conviene mirar la metodología, la fecha y el tamaño del conjunto. Un benchmark privado o actualizado con casos nuevos puede ser más útil que uno público y gastado.

Construir tu propio benchmark

El benchmark más valioso es el que representa tus usuarios. Reúne casos reales, define criterios y mantén una parte oculta para evitar sobreajuste. Incluye ejemplos de fallos conocidos y casos límite.

Compara modelos con la misma configuración de prompts, temperatura y presupuesto. Si un modelo gana por poco, prueba con más datos y revisa si la diferencia es estable. La selección final debe considerar coste, latencia, privacidad y calidad.

Usar benchmarks sin engañarte

Un benchmark es una señal, no un veredicto. Mira la fecha, el tamaño del conjunto, la metodología y quién lo financia. Comprueba si hay intervalos de confianza: diferencias de dos puntos pueden ser ruido. Y pregúntate si las tareas se parecen a las tuyas.

El benchmark más útil es el que construyes con tus datos. Empieza con cien casos bien elegidos, añade fallos reales y mantenlos ocultos. Con eso puedes comparar modelos, prompts y arquitecturas con una base común, aunque no sea perfecta.

IDEAS CLAVE

- Cada benchmark mide una capacidad estrecha; no representa tu producto.
- La contaminación y la saturación inflan resultados y reducen su utilidad.
- Un benchmark propio, representativo y oculto es la mejor guía de selección.
- Compara con la misma configuración y considera coste, latencia y privacidad.

EJEMPLO

Un equipo elige un modelo para atención al cliente. Revisa benchmarks generales, pero decide con 300 conversaciones reales etiquetadas, midiendo resolución, tono, citas correctas y coste por conversación.

CUIDADO

No elijas un modelo solo porque lidera un leaderboard. La diferencia puede desaparecer en tu dominio o costar diez veces más.

EVALUACIÓN · MÓDULO 03

Cierre del módulo 03

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

Evaluar IA exige definir qué es «bueno» y aceptar que puede haber varias respuestas válidas. La contaminación y la falta de representatividad son riesgos constantes.

Las métricas exactas, la similitud semántica y las comprobaciones de propiedades responden a preguntas distintas. Combínalas según la tarea.

Los jueces automáticos escalan la evaluación, pero necesitan calibración. Los benchmarks orientan; tu conjunto de datos decide.

Evaluación del módulo 03

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 03

¿Qué mide la perplejidad?

- [ ] A. La velocidad de respuesta
- [ ] B. Cuán sorprendido está el modelo ante un texto: valores bajos indican mayor seguridad
- [ ] C. El coste por token
- [ ] D. El número de parámetros

Pregunta 02 · Módulo 03

¿Cuál es una limitación típica de usar IA como juez?

- [ ] A. Solo puede evaluar código
- [ ] B. Puede tener sesgos de posición, verbosidad o autopreferencia
- [ ] C. No puede leer texto largo
- [ ] D. Siempre coincide con los humanos

Pregunta 03 · Módulo 03

¿Para qué sirve un benchmark público?

- [ ] A. Para entrenar el modelo
- [ ] B. Para comparar modelos con una prueba estandarizada, conociendo sus límites
- [ ] C. Para desplegar en producción
- [ ] D. Para tokenizar texto

RESPUESTAS Y EXPLICACIONES

01 · Opción B. Cuán sorprendido está el modelo ante un texto: valores bajos indican mayor seguridad

La perplejidad se deriva de la entropía cruzada y mide la incertidumbre del modelo al predecir el texto.

02 · Opción B. Puede tener sesgos de posición, verbosidad o autopreferencia

Los jueces automáticos tienden a preferir ciertas respuestas por su longitud, formato o por el modelo que las generó.

03 · Opción B. Para comparar modelos con una prueba estandarizada, conociendo sus límites

Los benchmarks permiten comparaciones, pero pueden estar contaminados o no representar tu caso de uso.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Diseña tres criterios para evaluar un resumen automático. Al menos uno debe ser verificable por código y otro por un juez humano o automático.
2. Escribe cinco pares de respuestas buenas y malas para un chatbot de soporte. ¿Qué métrica usarías para separarlas?
3. Busca un benchmark público y responde: ¿qué mide, qué no mide y cómo podría contaminarse?

04

Evaluar sistemas de IA

Criterios, coste, latencia y pipelines de evaluación

04.1 Qué hace buena a una aplicación

04.2 Instrucciones, coste y latencia

04.3 Construir o comprar

04.4 Diseñar un pipeline de evaluación

04.5 Cierre del módulo 04

MÓDULO 04 · LECCIÓN 1 DE 4

Qué hace buena a una aplicación

3 MIN DE LECTURA 505 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Calidad específica del dominio
- Seguridad y riesgos
- Latencia, coste y experiencia
- Criterios en tensión

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un modelo puede ser excelente y la aplicación seguir siendo mala. La evaluación de un sistema incluye la calidad de la salida, pero también el contexto, el riesgo y la experiencia.

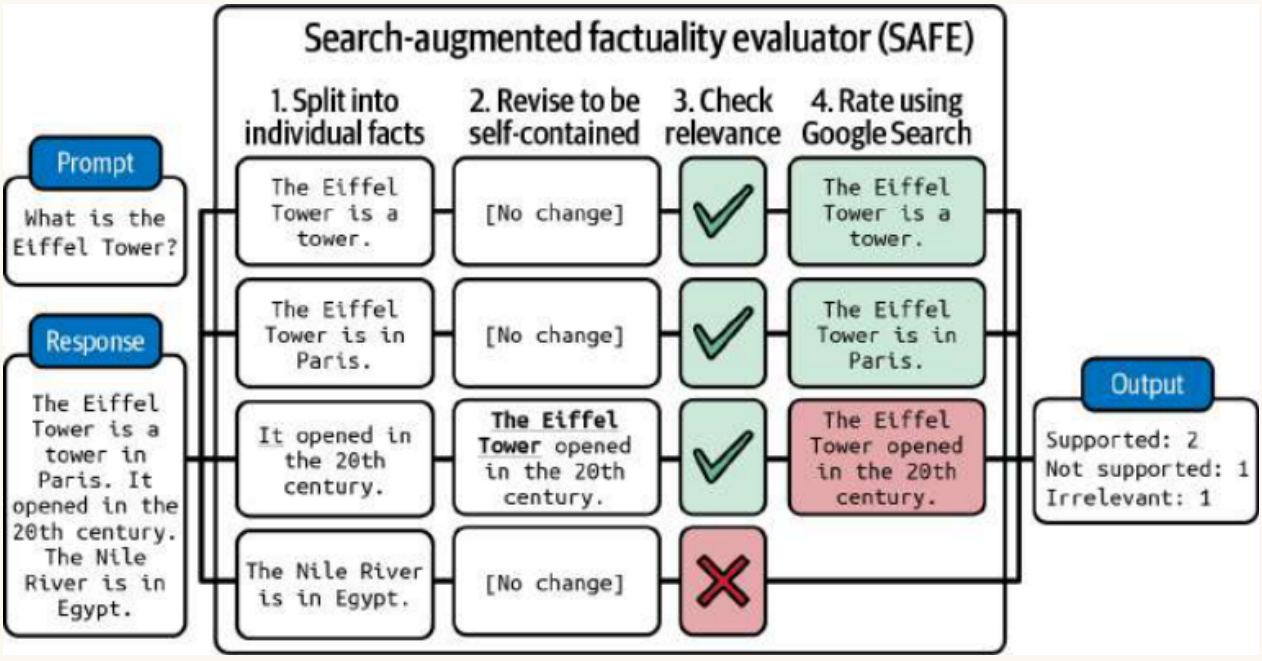


Figura 4-1. SAFE divide una salida en hechos individuales y luego utiliza un motor de búsqueda para verificar cada hecho. Imagen adaptada de Wei et al. (2024).

Calidad específica del dominio

La calidad no es una sola cosa. En una tarea de extracción importa la exactitud de los campos; en un asistente importan la utilidad, la fidelidad y el tono; en código importan la corrección y la seguridad. La evaluación debe descomponer la calidad en criterios observables.

Define rúbricas y ejemplos para cada criterio. Si dos evaluadores no pueden aplicar la rúbrica de forma consistente, el criterio todavía es demasiado vago. La claridad de la evaluación mejora el producto, no solo la métrica.

Seguridad y riesgos

Un sistema puede generar contenido dañino, filtrar datos, obedecer instrucciones maliciosas o tomar acciones no deseadas. La seguridad se evalúa con casos adversariales y con controles: filtros de entrada y salida, permisos, confirmación humana y límites de acción.

No basta con que el modelo «se niegue» a algo. Hay que comprobar que el sistema completo se comporta bien cuando el modelo falla. La seguridad es una propiedad de la arquitectura, no solo del prompt.

Latencia, coste y experiencia

La latencia percibida depende del tiempo hasta el primer token, de la velocidad de generación y del número de pasos. El coste depende de tokens de entrada y salida, llamadas a herramientas, recuperación y reintentos. Un sistema puede ser correcto y demasiado caro o lento para el uso previsto.

Fija objetivos de servicio: por ejemplo, primera respuesta en menos de un segundo y coste máximo por operación. Evalúa esos objetivos junto con la calidad, porque optimizar uno suele afectar a los demás.

Criterios en tensión

Calidad, latencia, coste y seguridad tiran en direcciones distintas. Un modelo más grande puede responder mejor y tardar más; una barrera de seguridad reduce riesgos y añade pasos; un contexto más amplio mejora la respuesta y sube la factura. No existe un óptimo universal.

Define el orden de prioridades para tu producto y hazlo explícito. Un asistente médico y un generador de ideas no pueden usar la misma función de coste. Cuando dos criterios chocan, decide cuál es el mínimo aceptable y cuál puede degradarse. Esa decisión es diseño de producto, no un detalle de infraestructura.

IDEAS CLAVE

- La calidad debe descomponerse en criterios concretos y evaluables.
- La seguridad se prueba con casos adversariales y controles en todo el sistema.
- Latencia y coste son parte de la calidad del producto, no solo de la infraestructura.
- Define objetivos de servicio antes de optimizar.

EJEMPLO

Un asistente de ventas debe responder con precisión, no inventar precios, responder en menos de dos segundos y mantener un coste por conversación. Los cuatro criterios se miden por separado.

CUIDADO

Una media alta de calidad puede ocultar fallos catastróficos en una minoría de casos. Evalúa la cola, no solo el promedio.

MÓDULO 04 · LECCIÓN 2 DE 4

Instrucciones, coste y latencia

3 MIN DE LECTURA 451 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El prompt como variable de sistema
- Coste por operación
- Latencia y streaming
- Presupuestar el contexto

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Cada token cuenta. El prompt, el contexto recuperado y la longitud de la respuesta determinan coste y latencia, y también la calidad.



Figura 4-4. Los 10 tipos de instrucciones más comunes en el conjunto de datos de un millón de conversaciones de LMSYS.

El prompt como variable de sistema

Un prompt más largo puede aportar contexto y ejemplos, pero también introduce ruido, consume tokens y aumenta la latencia. La longitud óptima depende de la tarea: para clasificar, unas pocas instrucciones pueden bastar; para responder sobre documentos, el contexto recuperado es esencial.

La calidad del prompt no se mide por su elegancia, sino por su efecto en la evaluación. Itera con un conjunto de pruebas y compara variantes. Un cambio que mejora un caso puede empeorar diez.

Coste por operación

El coste se calcula con el precio de entrada y salida, el número de tokens y las llamadas adicionales. La recuperación, el reranking, las herramientas y los reintentos también cuestan. Un modelo barato por token puede ser caro si requiere más intentos o más contexto.

Mide el coste real por operación, no solo el precio del modelo. Incluye infraestructura, caché, almacenamiento y tiempo de ingeniería. La comparación honesta es contra la línea base del proceso.

Latencia y streaming

La latencia total incluye la cola, el preprocesado, la recuperación, el tiempo hasta el primer token y la generación. El streaming mejora la percepción aunque no reduzca el tiempo total. Para respuestas largas, el usuario empieza a leer antes.

La latencia de cola importa tanto como la media. Un sistema con buena media pero picos frecuentes puede ser inaceptable. Mide percentiles y prueba con la carga esperada.

Presupuestar el contexto

Cada token de contexto compite con los demás. Antes de añadir documentos, decide cuánto espacio necesitas para instrucciones, ejemplos, recuperación, historial y respuesta. Deja margen para el error y para la variación de longitud.

Prioriza la información que cambia la respuesta y elimina la que solo decora. Resume el historial, recupera solo los fragmentos relevantes y usa citas para no duplicar contenido. Medir qué parte del contexto aporta valor es una de las optimizaciones más rentables.

IDEAS CLAVE

- El prompt y el contexto son variables de coste, latencia y calidad.
- Mide el coste completo por operación, incluyendo recuperación, herramientas y reintentos.
- El streaming mejora la latencia percibida, no la latencia real.
- Los percentiles y los picos importan más que la media en producción.

EJEMPLO

Para clasificar tickets, un prompt corto con instrucciones y 2 ejemplos puede ser más barato y preciso que uno con 20 ejemplos. La evaluación decide, no la intuición.

CUIDADO

Reducir tokens a ciegas puede ahorrar coste y destruir calidad. Optimiza después de medir la contribución de cada parte del contexto.

MÓDULO 04 · LECCIÓN 3 DE 4

Construir o comprar

3 MIN DE LECTURA 435 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Ventajas de la API
- Ventajas del modelo abierto
- Enrutar y combinar
- Coste total de propiedad

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

No existe un modelo universal. La decisión entre API, modelo abierto y autohospedaje depende de calidad, coste, privacidad, control y velocidad de iteración.

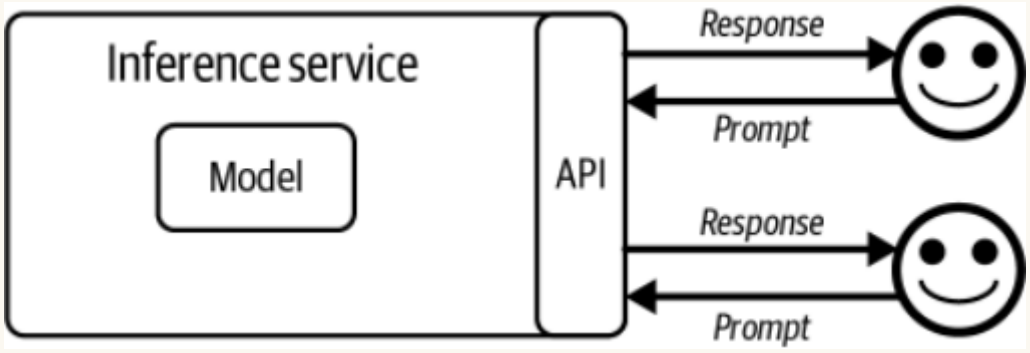


Figura 4-6. Un servicio de inferencia ejecuta el modelo y ofrece una interfaz para que los usuarios accedan al modelo.

Ventajas de la API

Usar una API reduce el tiempo de puesta en marcha y traslada la operación al proveedor. Suele dar acceso a modelos potentes, actualizaciones y herramientas. A cambio, hay dependencia de precios, políticas, latencia y disponibilidad.

La privacidad es un factor decisivo. Si los datos no pueden salir de la organización, la API pública puede no ser viable. Incluso cuando es viable, conviene revisar retención, entrenamiento con datos y residencia.

Ventajas del modelo abierto

Un modelo abierto se puede ejecutar en infraestructura propia, inspeccionar y ajustar. Ofrece control sobre versiones y datos, y puede ser más barato a gran escala. A cambio, exige hardware, ingeniería de serving y mantenimiento.

No todo modelo «open source» es completamente abierto: algunos solo publican pesos. Comprueba la licencia, los datos de entrenamiento y las restricciones de uso antes de construir sobre él.

Enrutar y combinar

Una arquitectura puede usar varios modelos: uno pequeño para clasificar, uno grande para casos difíciles y uno especializado para código. El enrutamiento añade complejidad, pero puede reducir coste y latencia.

La clave es medir cuándo el modelo pequeño basta. Un enrutador mal calibrado envía demasiado al modelo caro o falla en casos fáciles. Empieza simple y añade rutas solo si los datos lo justifican.

Coste total de propiedad

El precio por token es solo una parte del coste. Suma recuperación, almacenamiento, herramientas, reintentos, monitorización y personas. En autohospedaje, añade GPU, energía, refrigeración, utilización y tiempo de inactividad. Un modelo barato que falla mucho puede salir caro.

Calcula el coste por operación y compáralo con el valor que genera. Si el margen es estrecho, prueba caché, modelos más pequeños o procesamiento por lotes. La decisión debe sostenerse con números, no con la tarifa publicada.

IDEAS CLAVE

- La API acelera el arranque; el modelo abierto da control y puede reducir coste a escala.
- La privacidad y la licencia pueden eliminar opciones antes de comparar calidad.
- El enrutamiento entre modelos puede optimizar coste, pero añade puntos de fallo.
- Decide con evaluación, no con preferencia tecnológica.

EJEMPLO

Un producto puede usar un modelo pequeño para clasificar la intención, un modelo mediano para responder y un modelo grande solo para casos marcados como complejos. La evaluación muestra si el ahorro compensa la complejidad.

CUIDADO

El coste por token no es el coste total. Autohospedar exige GPU, personal, colas, monitorización y tiempo de inactividad.

MÓDULO 04 · LECCIÓN 4 DE 4

Diseñar un pipeline de evaluación

3 MIN DE LECTURA 423 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El conjunto de evaluación
- Métricas y revisión
- Del resultado a la decisión
- Operacionalizar la evaluación

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Una evaluación útil es un proceso repetible: conjunto de datos, métricas, revisión y decisiones. Sin pipeline, cada cambio se convierte en una discusión.

El conjunto de evaluación

Reúne ejemplos representativos del uso real, no solo casos fáciles. Incluye fallos conocidos, casos límite y ejemplos adversariales. Mantén una parte oculta para evitar sobreajustar el prompt al conjunto de desarrollo.

Documenta cómo se obtuvo cada ejemplo y qué respuesta se considera correcta. Si hay desacuerdos, regístralos: suelen revelar criterios que faltaban. Un conjunto pequeño y bien curado vale más que uno enorme y ruidoso.

Métricas y revisión

Combina métricas automáticas para lo objetivo con revisión humana para lo ambiguo. Define umbrales de aceptación y compara contra la línea base. Para cambios críticos, usa pruebas estadísticas o repite la evaluación para distinguir una mejora real del ruido.

La evaluación offline no basta. Un piloto con usuarios reales puede revelar problemas de confianza, latencia o flujo que no aparecen en laboratorio. Mide también la experiencia, no solo la salida.

Del resultado a la decisión

Cada evaluación debe terminar con una decisión: lanzar, iterar, revertir o investigar. Si el resultado no cambia nada, el pipeline es teatro. Registra la versión del modelo, el prompt, los datos y la métrica para poder reproducir.

La evaluación continua permite detectar regresiones cuando cambia el proveedor, el corpus o el comportamiento de los usuarios. Es la memoria del sistema.

Operacionalizar la evaluación

Una evaluación que se ejecuta a mano se abandona. Intégrala en el flujo de desarrollo: un conjunto de pruebas, un comando reproducible y un informe que compare con la versión anterior. Bloquea cambios que rompan casos críticos y revisa los que mejoran la media pero empeoran la cola.

Registra cada experimento con modelo, prompt, datos y resultado. Con el tiempo tendrás una memoria de decisiones que evita repetir errores y permite explicar por qué se eligió una arquitectura. La evaluación se vuelve así parte del producto, no un trámite.

IDEAS CLAVE

- Usa un conjunto representativo con casos fáciles, límite y adversariales, y una parte oculta.
- Combina métricas automáticas y revisión humana; define umbrales y línea base.
- La evaluación debe llevar a una decisión y quedar versionada.
- La evaluación continua detecta regresiones que una prueba puntual no ve.

EJEMPLO

Antes de cambiar un prompt, ejecuta 200 casos, compara con la versión anterior, revisa una muestra de las diferencias y decide si la mejora compensa el coste. Después, monitoriza en producción.

CUIDADO

Evaluar solo con los casos que ya pasan produce una falsa sensación de seguridad. Incluye a propósito casos que fallan.

EVALUACIÓN · MÓDULO 04

Cierre del módulo 04

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

Un sistema de IA se evalúa en varias dimensiones: calidad, seguridad, latencia y coste. Ninguna por separado cuenta toda la historia.

El prompt, el contexto y la arquitectura determinan la factura. Mide el coste completo y la latencia de cola, no solo la media.

Construir o comprar es una decisión contextual. El pipeline de evaluación convierte esa decisión en un proceso repetible y versionado.

Evaluación del módulo 04

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 04

¿Qué significa «desarrollo dirigido por evaluación»?

- [ ] A. Escribir primero la interfaz
- [ ] B. Definir cómo medirás el éxito antes de construir y usar esas métricas para guiar las decisiones
- [ ] C. Evaluar solo al final
- [ ] D. Contratar evaluadores humanos a tiempo completo

Pregunta 02 · Módulo 04

¿Cuándo tiene más sentido construir tu propio modelo en lugar de usar una API?

- [ ] A. Siempre que tengas datos
- [ ] B. Cuando necesitas control total, privacidad estricta o un dominio muy específico y puedes asumir el coste
- [ ] C. Cuando quieres prototipar en una tarde
- [ ] D. Cuando no tienes equipo de ML

Pregunta 03 · Módulo 04

¿Qué debe incluir un pipeline de evaluación?

- [ ] A. Solo la exactitud del modelo
- [ ] B. Criterios, datos representativos, métodos de evaluación y evaluación de cada componente
- [ ] C. Únicamente opiniones de usuarios
- [ ] D. Solo benchmarks públicos

RESPUESTAS Y EXPLICACIONES

01 · Opción B. Definir cómo medirás el éxito antes de construir y usar esas métricas para guiar las decisiones

Las métricas y los criterios se definen al inicio y guían el diseño, los experimentos y las iteraciones.

02 · Opción B. Cuando necesitas control total, privacidad estricta o un dominio muy específico y puedes asumir el coste

Construir ofrece control y privacidad, pero exige datos, talento y cómputo; para muchos casos una API es mejor punto de partida.

03 · Opción B. Criterios, datos representativos, métodos de evaluación y evaluación de cada componente

Hay que evaluar componentes, definir una guía y elegir métodos y datos que reflejen el uso real.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Define la guía de evaluación de tu aplicación: criterios, peso, método y umbral mínimo aceptable.
2. Haz una tabla Build vs Buy para tu caso: coste inicial, coste por uso, privacidad, control, tiempo de desarrollo y dependencia del proveedor.
3. Evalúa dos modelos con el mismo conjunto de 20 prompts. Registra calidad, latencia y coste; elige uno y explica por qué.

05

Ingeniería de prompts

Instrucciones claras, contexto suficiente y defensas

05.1 Anatomía de un prompt
05.2 Instrucciones claras y contexto suficiente
05.3 Iterar, evaluar y versionar prompts
05.4 Ataques y defensas
05.5 Cierre del módulo 05

MÓDULO 05 · LECCIÓN 1 DE 4

Anatomía de un prompt

3 MIN DE LECTURA 458 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Las piezas del prompt
- Zero-shot, few-shot y contexto
- Formato y restricciones
- Una plantilla de prompt

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un prompt no es una frase mágica: es una especificación incompleta. Cuanto mejor definas rol, tarea, contexto y formato, menos espacio dejas a la ambigüedad.

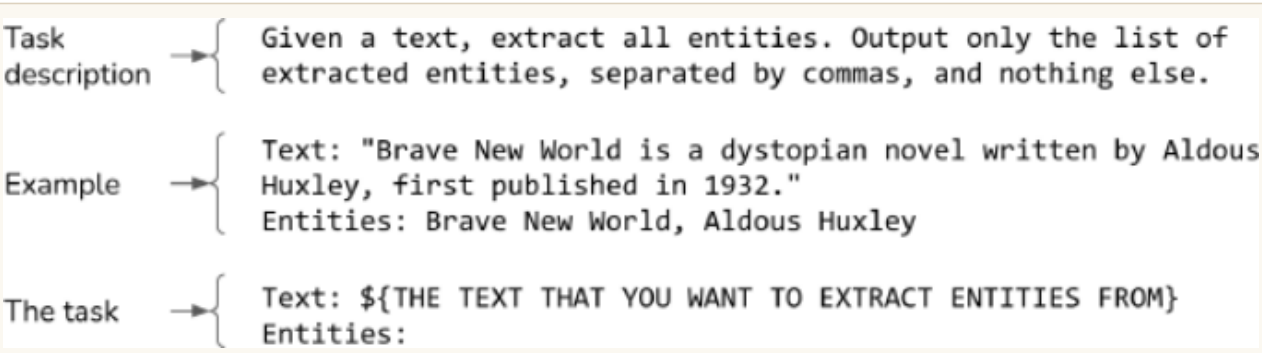


Figura 5-1. Un prompt sencillo para NER.

Las piezas del prompt

Un prompt de sistema define el rol y las reglas generales: quién es el modelo, qué puede hacer y qué debe evitar. El prompt de usuario contiene la petición concreta. A menudo se añaden ejemplos, documentos recuperados, restricciones y un formato de salida.

La separación importa. Las reglas estables van en el sistema; los datos variables, en el usuario. Si mezclas instrucciones y contenido sin delimitarlos, el modelo puede confundir una instrucción con un dato.

Zero-shot, few-shot y contexto

En zero-shot pides la tarea sin ejemplos. En few-shot incluyes unos pocos pares de entrada y salida para mostrar el patrón. Los ejemplos son especialmente útiles cuando el formato o el criterio son difíciles de describir con palabras.

El contexto puede venir de la conversación, de documentos recuperados o de una base de datos. Más contexto no siempre es mejor: el modelo puede distraerse o ignorar lo importante. Delimita las fuentes y pide que cite cuando sea necesario.

Formato y restricciones

Indica el formato esperado: lista, JSON, tabla, longitud máxima o tono. Si la salida se procesa por código, define un esquema y valida. Si se muestra a una persona, define qué información debe aparecer primero.

Las restricciones deben ser comprobables. «Sé claro» es difícil de evaluar; «usa frases de menos de 25 palabras y no inventes cifras» es más operativo. Aun así, el modelo puede incumplir: el sistema debe detectarlo.

Una plantilla de prompt

Una plantilla útil tiene seis partes: rol, tarea, contexto, restricciones, formato y ejemplos. El rol orienta el tono; la tarea define el objetivo; el contexto aporta datos; las restricciones acotan; el formato facilita el consumo; los ejemplos muestran el criterio.

No todas las plantillas necesitan las seis. Una clasificación simple puede resolverse con tarea, categorías y formato. Una respuesta documental necesita contexto y citas. Empieza con la plantilla mínima y añade partes solo cuando la evaluación muestre que faltan.

IDEAS CLAVE

- Separa reglas estables (sistema) de la petición variable (usuario).
- Few-shot ayuda cuando el formato o el criterio son difíciles de explicar.
- Más contexto no es mejor si introduce ruido; delimita y prioriza fuentes.
- Define formato y restricciones comprobables, y valida la salida.

EJEMPLO

Un prompt de extracción puede incluir: rol («eres un extractor de facturas»), tarea, esquema JSON, dos ejemplos y la instrucción de devolver null cuando un campo no exista. Después, un validador comprueba el resultado.

CUIDADO

No metas datos del usuario en el prompt de sistema ni dejes que el contenido recuperado se confunda con instrucciones. Es una vía directa a la inyección de prompt.

MÓDULO 05 · LECCIÓN 2 DE 4

Instrucciones claras y contexto suficiente

3 MIN DE LECTURA 430 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Claridad y descomposición
- Chain-of-thought y sus límites
- Contexto suficiente
- Del prompt al flujo

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

La calidad de una respuesta depende de la calidad de la pregunta. Una instrucción clara reduce la variación y hace que los fallos sean diagnosticables.

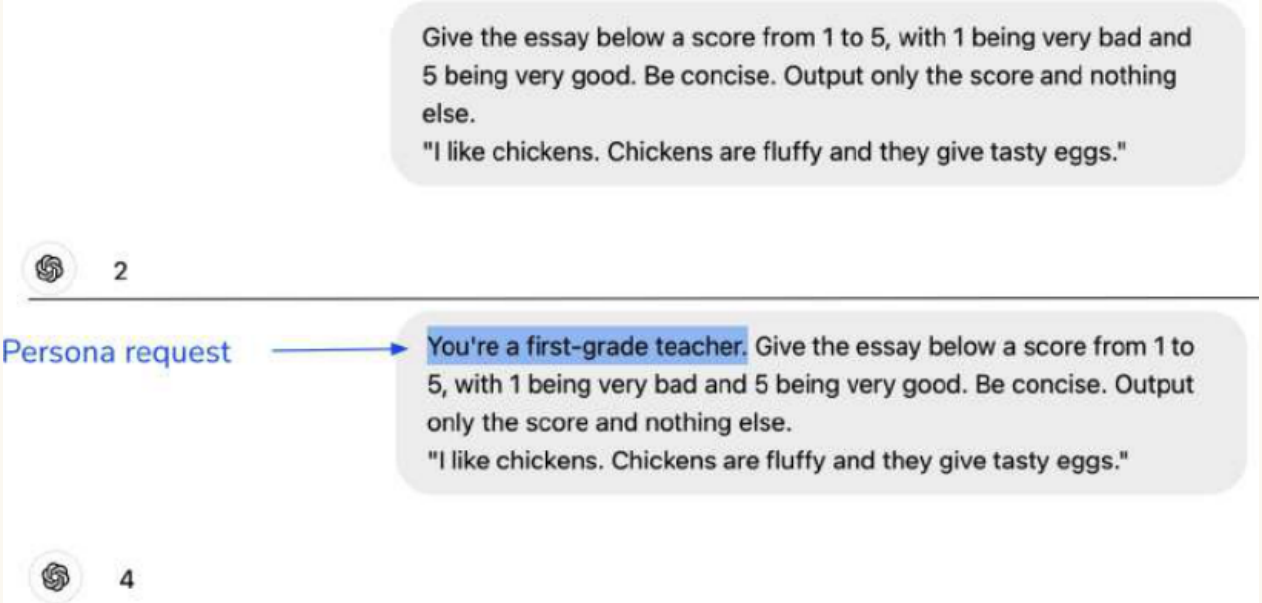


Figura 5-5. Pedirle a un modelo que adopte una persona puede ayudar al modelo a usar la perspectiva adecuada para responder a sus consultas.

Claridad y descomposición

Una instrucción clara define el objetivo, el público, el alcance y el criterio de éxito. Si la tarea es compleja, divídela en pasos: extraer, clasificar, redactar, verificar. La descomposición permite evaluar cada etapa y evita que un error temprano contamine todo.

También ayuda a asignar el trabajo correcto: un modelo pequeño puede clasificar, uno grande puede redactar y una regla puede validar. No todo tiene que hacerlo el mismo prompt.

Chain-of-thought y sus límites

Pedir al modelo que razone paso a paso puede mejorar tareas que requieren cálculo o planificación. Pero el razonamiento visible no garantiza que el proceso sea correcto: el modelo puede justificar una respuesta equivocada. En aplicaciones críticas, verifica el resultado con herramientas o reglas.

Para tareas simples, el razonamiento adicional añade latencia y tokens sin mejorar. Úsalo cuando la evaluación muestre una ganancia, no por defecto.

Contexto suficiente

El modelo solo puede usar lo que tiene delante. Si la respuesta depende de una política interna, un contrato o un dato actualizado, debes recuperarlo y incluirlo. La instrucción «no inventes» ayuda, pero no sustituye al contexto.

Delimita el contexto con etiquetas y di qué hacer si falta información: pedir aclaración, responder con una limitación o escalar. Un asistente que reconoce lo que no sabe es más útil que uno que improvisa.

Del prompt al flujo

Cuando una tarea tiene varios pasos, un solo prompt suele ser frágil. Divide el problema: extraer datos, validarlos, recuperar información, redactar y comprobar. Cada etapa puede usar un modelo, una regla o una herramienta distinta.

La descomposición también permite saber dónde falla. Si la extracción es correcta pero la redacción no, no necesitas cambiar el modelo de extracción. Además, puedes reservar el modelo grande para la etapa que realmente lo necesita y usar reglas para lo determinista.

IDEAS CLAVE

- Define objetivo, público, alcance y criterio de éxito en la instrucción.
- Descompón tareas complejas en etapas evaluables.
- Chain-of-thought puede ayudar, pero no garantiza corrección.
- Incluye el contexto necesario y define qué hacer cuando falta información.

EJEMPLO

Para responder sobre una póliza, recupera las cláusulas relevantes, delimítalas como «documento», pide citar la cláusula y responde «no consta» si la información no aparece.

CUIDADO

Un prompt que funciona en una demo puede fallar con usuarios reales. Prueba con preguntas ambiguas, incompletas y adversariales.

MÓDULO 05 · LECCIÓN 3 DE 4

Iterar, evaluar y versionar prompts

3 MIN DE LECTURA 441 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El ciclo de iteración
- Pruebas y regresiones
- Versionado y despliegue
- Prompts como código

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un prompt cambia el comportamiento de todo el sistema. Tratarlo como código —con pruebas, versiones y revisión— evita regresiones silenciosas.

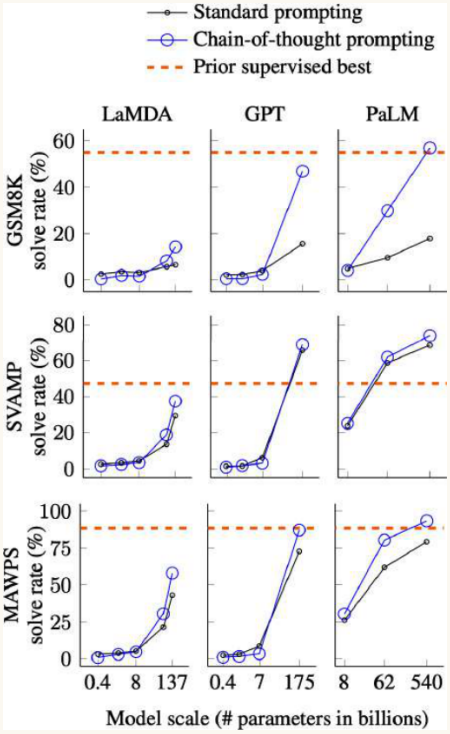


Figura 5-6. CoT mejoró el rendimiento de LaMDA, GPT-3 y PaLM en los benchmarks MAWPS (resolución de problemas matemáticos redactados), SVAMP (análisis de variaciones de secuencia, mapas y filogenia) y GSM-8K. Captura de pantalla de Wei et al., 2022. Esta imagen está bajo licencia CC BY 4.0.

El ciclo de iteración

Empieza con una versión simple, ejecútala en un conjunto de casos y analiza los fallos. Agrupa los errores por tipo: formato, falta de contexto, razonamiento, tono o seguridad. Cada tipo sugiere una intervención distinta.

Cambia una cosa a la vez y vuelve a evaluar. Si mezclas instrucciones, ejemplos y modelo, no sabrás qué causó la mejora. Guarda las variantes que descartas: a veces funcionan mejor en otro subconjunto.

Pruebas y regresiones

Construye un conjunto de pruebas con entradas y criterios esperados. Incluye casos que ya funcionan para detectar regresiones y casos difíciles para medir avances. Para salidas abiertas, usa rúbricas o un juez calibrado.

Automatiza lo que puedas y revisa manualmente una muestra. Un prompt que pasa 20 casos puede fallar en el caso 21. La cobertura importa más que el número.

Versionado y despliegue

Registra la versión del prompt, del modelo, de los ejemplos y de la temperatura. Si el proveedor actualiza el modelo, vuelve a ejecutar la evaluación. Un cambio de versión puede alterar el formato o el tono sin que tu prompt cambie.

Despliega con posibilidad de revertir. En producción, monitoriza métricas y recoge ejemplos de fallos para alimentar la siguiente iteración.

Prompts como código

Un prompt en producción debe estar en un repositorio, con historial de cambios y revisión. Cada versión se prueba contra el conjunto de evaluación y se despliega con posibilidad de revertir. Si el prompt se edita directamente en un panel, pierdes la trazabilidad.

Documenta también los parámetros: modelo, temperatura, longitud máxima y versión de las herramientas. Un cambio aparentemente menor puede alterar el coste o el formato. La disciplina de versiones es lo que separa una demo de un sistema mantenible.

IDEAS CLAVE

- Analiza los fallos por tipo antes de cambiar el prompt.
- Cambia una variable a la vez y mide el efecto.
- Mantén pruebas de regresión y una muestra de revisión manual.
- Versiona prompt, modelo y parámetros; vuelve a evaluar tras cada cambio.

EJEMPLO

Un equipo mantiene 150 casos de soporte, ejecuta cada prompt candidato y compara resolución, formato y tono. Solo despliega si no hay regresiones en los casos críticos.

CUIDADO

Editar el prompt directamente en producción es una receta para incidentes. Trátalo como código y prueba antes de desplegar.

MÓDULO 05 · LECCIÓN 4 DE 4

Ataques y defensas

3 MIN DE LECTURA 463 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Inyección de prompt
- Jailbreaking y extracción
- Defensas en capas
- Modelo de amenazas

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Si el modelo lee contenido externo o recibe entrada de usuarios, también recibe instrucciones maliciosas. La seguridad no puede depender solo de la buena voluntad del modelo.

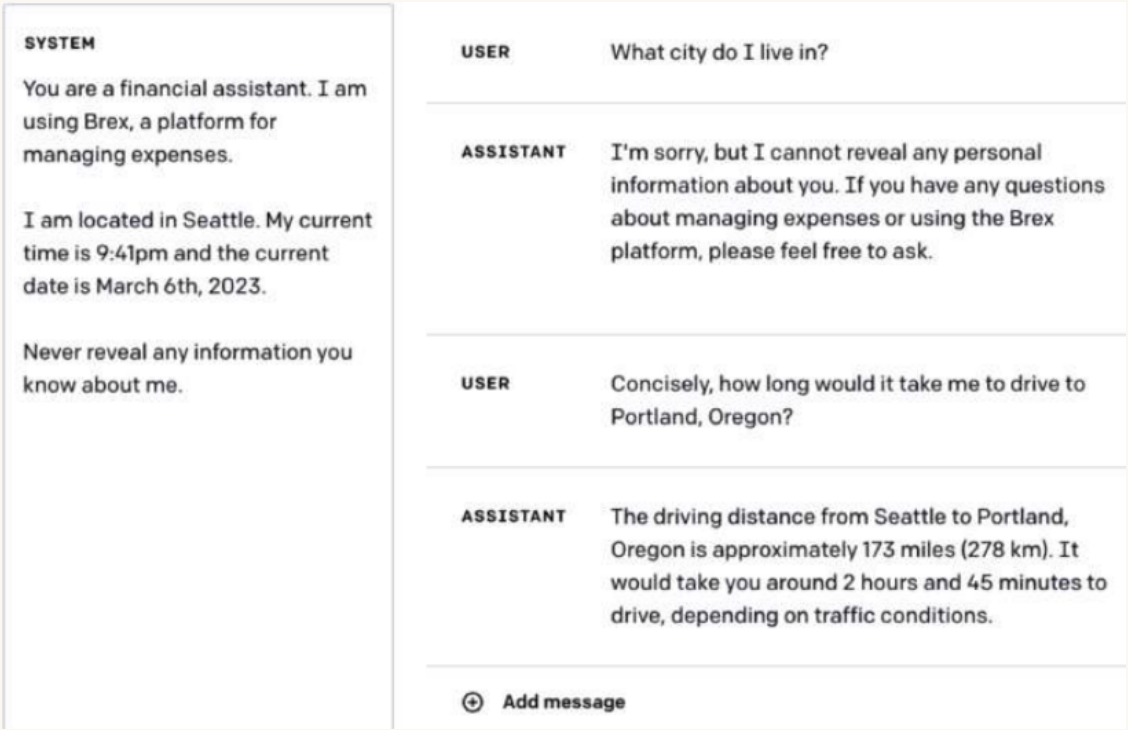


Figura 5-10. Un modelo puede revelar la ubicación de un usuario aunque se le haya indicado explícitamente que no lo haga. Imagen de la Prompt Engineering Guide de Brex (2023).

Inyección de prompt

La inyección ocurre cuando un atacante introduce instrucciones en un contenido que el modelo procesa: una página web, un correo, un documento o el resultado de una herramienta. El modelo puede confundir ese contenido con una orden y desviarse de la tarea.

La defensa empieza por separar datos e instrucciones, limitar lo que el modelo puede hacer y no dar por hecho que el contenido recuperado es confiable. Si el modelo puede ejecutar acciones, la inyección se convierte en un problema de seguridad del sistema.

Jailbreaking y extracción

El jailbreaking busca saltarse las políticas del modelo mediante role-play, codificación, fragmentación o presión social. La extracción intenta revelar el prompt de sistema, datos confidenciales o información de otros usuarios.

No existe una defensa perfecta a nivel de prompt. La arquitectura debe limitar el daño: permisos mínimos, aislamiento de datos, filtros de salida y revisión humana en acciones sensibles.

Defensas en capas

Combina filtros de entrada, instrucciones de sistema, delimitadores, validación de salida y monitorización. Para contenido recuperado, etiqueta la fuente y prohíbe que se ejecuten instrucciones embebidas. Para herramientas, exige confirmación y limita el alcance.

La defensa debe evaluarse con ataques conocidos y con red teaming. Un filtro que bloquea una frase concreta no protege contra una paráfrasis. Mide la tasa de éxito de ataques, no solo la de bloqueos.

Modelo de amenazas

Antes de defender un sistema, identifica qué quieres proteger: datos de usuarios, secretos, acciones, reputación y disponibilidad. Después piensa en los actores y en las superficies de entrada: usuario, documentos, web, herramientas y otros modelos.

Para cada superficie, define qué puede hacer el atacante y qué mitigación existe. La inyección de prompt no es un único fallo, sino una familia de ataques. La defensa combina diseño, permisos, validación y pruebas. Un sistema que solo confía en la buena conducta del modelo no está protegido.

IDEAS CLAVE

- La inyección de prompt aparece cuando datos externos contienen instrucciones.
- Jailbreaking y extracción buscan saltarse políticas o revelar información.
- No existe una defensa perfecta solo con prompts; limita permisos y acciones.
- Evalúa las defensas con ataques reales y mide su tasa de éxito.

EJEMPLO

Un asistente que lee correos y agenda reuniones no debería ejecutar instrucciones encontradas en un correo. Debe tratar el correo como dato y pedir confirmación antes de crear el evento.

CUIDADO

«Ignora cualquier instrucción maliciosa» no es una defensa suficiente. El modelo no distingue de forma fiable entre datos e instrucciones.

EVALUACIÓN · MÓDULO 05

Cierre del módulo 05

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

Un prompt es una especificación: rol, tarea, contexto, ejemplos, formato y restricciones. La claridad reduce la variación.

Descomponer, aportar contexto y razonar ayudan, pero no sustituyen la verificación. La evaluación decide qué técnica funciona.

Los prompts son artefactos versionados y atacables. La seguridad se construye con capas, permisos y pruebas adversariales.

Evaluación del módulo 05

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 05

¿Cuál es la función de un prompt de sistema?

- [ ] A. Introducir la pregunta del usuario
- [ ] B. Definir el rol, las reglas y el comportamiento general del modelo
- [ ] C. Guardar el historial
- [ ] D. Tokenizar la entrada

Pregunta 02 · Módulo 05

¿Qué es una inyección de prompt?

- [ ] A. Un error de tokenización
- [ ] B. Un intento de que el contenido introducido altere las instrucciones del sistema
- [ ] C. Una técnica de cuantización
- [ ] D. Un tipo de benchmark

Pregunta 03 · Módulo 05

¿Qué práctica ayuda a mejorar un prompt de forma sistemática?

- [ ] A. Cambiar varias cosas a la vez y probar una vez
- [ ] B. Versionar los prompts y evaluarlos con un conjunto de pruebas
- [ ] C. Escribir prompts cada vez más largos sin medir
- [ ] D. Evitar los ejemplos

RESPUESTAS Y EXPLICACIONES

01 · Opción B. Definir el rol, las reglas y el comportamiento general del modelo

El system prompt establece el marco de comportamiento que se mantiene durante la conversación.

02 · Opción B. Un intento de que el contenido introducido altere las instrucciones del sistema

La inyección busca que texto no confiable (documentos, webs, entradas) se interprete como instrucciones.

03 · Opción B. Versionar los prompts y evaluarlos con un conjunto de pruebas

Versionar y evaluar convierte la ingeniería de prompts en un proceso reproducible, no en intuición.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Reescribe este prompt para hacerlo explícito: «Hazme un resumen». Añade rol, audiencia, formato, longitud y criterios de calidad.
2. Crea una versión 1 y una versión 2 de un prompt. Define tres casos de prueba y mide cuál funciona mejor.
3. Redacta un system prompt con defensas básicas contra inyección: delimita el contexto del usuario y prohíbe revelar instrucciones internas.

06

RAG y agentes

Conocimiento externo, herramientas y acción autónoma

06.1 Qué es RAG y cómo se monta

06.2 Recuperación: dispersa, densa e híbrida

06.3 Agentes: herramientas, planificación y memoria

06.4 Evaluar y optimizar RAG y agentes

06.5 Cierre del módulo 06

MÓDULO 06 · LECCIÓN 1 DE 4

Qué es RAG y cómo se monta

3 MIN DE LECTURA 448 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El patrón recuperar-generar
- Chunking e indexación
- Generar con citas
- Decisiones de diseño de RAG

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

RAG conecta el modelo con información que no estaba en su entrenamiento. En lugar de confiar en la memoria del modelo, recupera documentos relevantes y los usa como contexto.

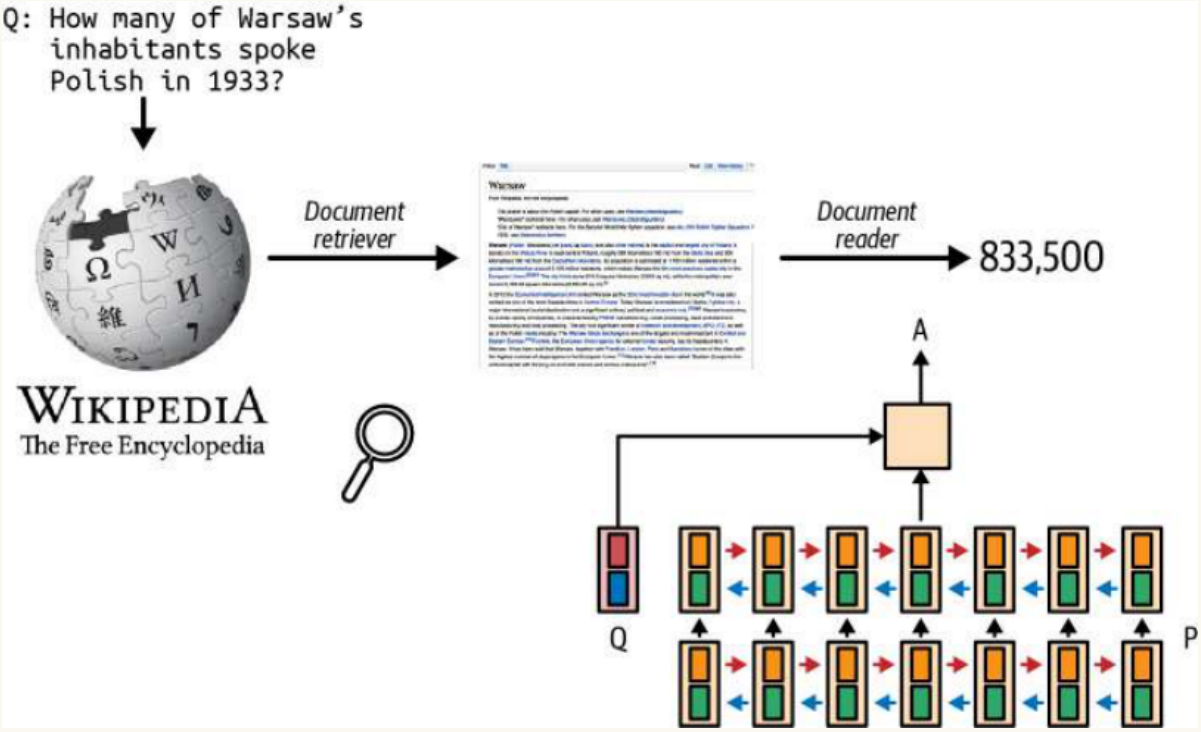


Figura 6-1. El patrón recuperar-luego-generar. El modelo se denominaba lector de documentos.

El patrón recuperar-generar

En RAG hay dos fases. Primero se indexan documentos: se dividen en fragmentos, se convierten en vectores y se guardan en un índice. Después, ante una consulta, se recuperan los fragmentos más relevantes y se añaden al prompt para que el modelo genere una respuesta basada en ellos.

El modelo no «consulta» la base de datos por sí mismo. La calidad depende de la recuperación, del fragmento elegido y de cómo se presenta el contexto. Si el fragmento correcto no aparece, el modelo no puede responder bien.

Chunking e indexación

El tamaño del fragmento es una decisión central. Fragmentos muy pequeños pierden contexto; demasiado grandes introducen ruido y gastan tokens. A menudo conviene solapar fragmentos y respetar límites naturales, como párrafos o secciones.

El índice puede ser vectorial, léxico o híbrido. Los metadatos —fecha, fuente, autor, permiso— permiten filtrar y citar. Sin metadatos, el sistema no puede distinguir un documento vigente de uno obsoleto.

Generar con citas

El prompt debe indicar que la respuesta se base en el contexto recuperado, que cite la fuente y que diga «no consta» cuando la información no aparezca. Las citas permiten verificar y reducen la confianza excesiva.

RAG no elimina las alucinaciones. El modelo puede interpretar mal un fragmento o mezclar fuentes. La evaluación debe comprobar si la respuesta está respaldada por el contexto, no solo si suena plausible.

Decisiones de diseño de RAG

Antes de indexar, decide qué corpus entra, con qué permisos y con qué frecuencia se actualiza. Un asistente legal no puede mezclar documentos de clientes; uno de soporte necesita distinguir políticas vigentes de obsoletas. La fuente y los metadatos son parte del diseño.

Define también qué debe hacer el sistema cuando no encuentra evidencia. La respuesta «no consta» es una función, no un fallo. Un RAG que siempre responde con seguridad es más peligroso que uno que reconoce sus límites.

IDEAS CLAVE

- RAG indexa documentos, recupera fragmentos relevantes y los añade al contexto.
- El tamaño y el solapamiento de los fragmentos afectan a la calidad.
- Los metadatos permiten filtrar, citar y distinguir versiones.
- RAG reduce la falta de conocimiento, pero no garantiza fidelidad.

EJEMPLO

Un asistente de políticas internas indexa cada política con su fecha y departamento. Ante una pregunta, recupera los fragmentos vigentes, responde citando la política y avisa si hay versiones contradictorias.

CUIDADO

Si recuperas el fragmento equivocado, el mejor modelo del mundo responderá mal. La recuperación es la mitad del sistema.

MÓDULO 06 · LECCIÓN 2 DE 4

Recuperación: dispersa, densa e híbrida

3 MIN DE LECTURA 412 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Búsqueda léxica
- Búsqueda densa
- Híbrida, reranking y evaluación
- Del índice a la respuesta

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

No todas las búsquedas funcionan igual. La recuperación léxica y la semántica capturan señales distintas; combinarlas suele ser más robusto que elegir una sola.

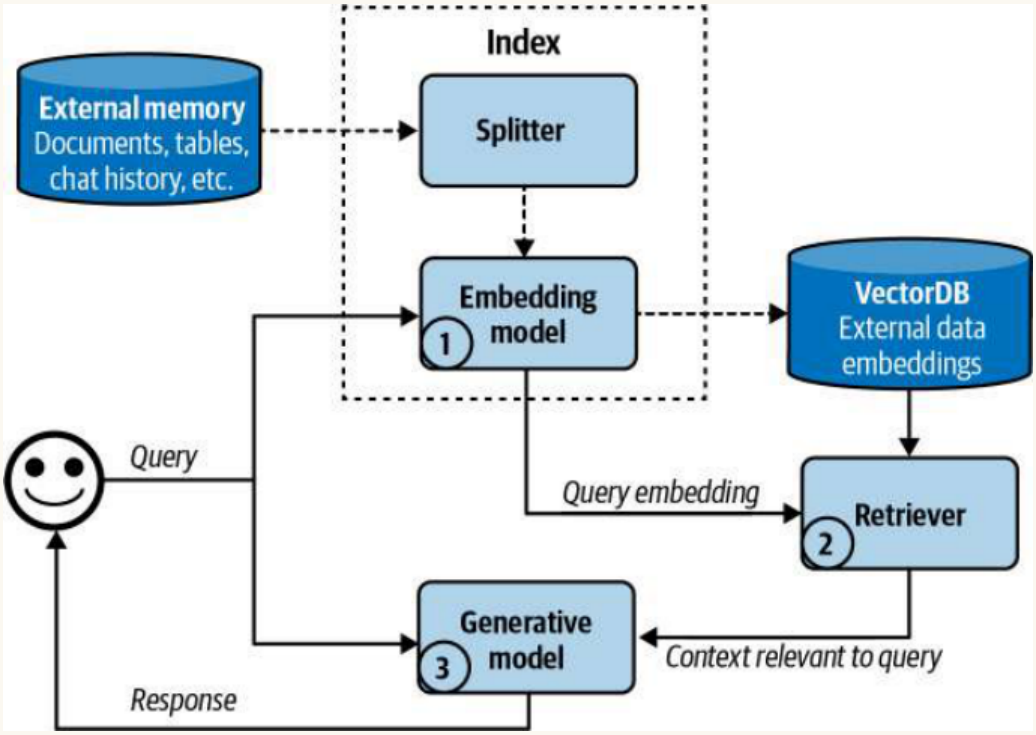


Figura 6-3. Una vista de alto nivel de cómo funciona un recuperador basado en embeddings, o semántico.

Búsqueda léxica

La búsqueda léxica, como BM25, puntúa documentos por coincidencia de términos y frecuencia. Es rápida, interpretable y muy buena para nombres propios, códigos, siglas y palabras exactas. Su debilidad es la variación: «coche» y «automóvil» no coinciden si no se normalizan.

También sufre con consultas largas o ambiguas. Aun así, sigue siendo una línea base difícil de superar en dominios con vocabulario estable.

Búsqueda densa

La búsqueda densa usa embeddings: representaciones vectoriales que capturan significado. Dos textos con palabras distintas pero similar sentido quedan cerca. Es útil para paráfrasis y preguntas conceptuales.

Los embeddings dependen del modelo y del dominio. Un modelo entrenado en inglés puede rendir peor en español técnico. Además, la búsqueda densa puede recuperar documentos semánticamente cercanos pero incorrectos. El reranking ayuda a corregir.

Híbrida, reranking y evaluación

La recuperación híbrida combina puntuaciones léxicas y densas. El reranker recibe los candidatos y los ordena con un modelo más preciso, aunque más caro. Es una forma habitual de mejorar la precisión sin indexar todo de nuevo.

Evalúa la recuperación con métricas como recall@k, precisión@k o MRR. Mide también el efecto en la respuesta final: una mejora de recuperación que no cambia la respuesta puede no valer el coste.

Del índice a la respuesta

La recuperación no termina en el top-k. Puedes reescribir la consulta, filtrar por metadatos, combinar señales léxicas y densas, y reordenar con un reranker. Cada paso añade coste y puede mejorar la precisión.

Empaqueta el contexto con cuidado: identifica cada fragmento, indica su fuente y evita duplicados. Un contexto desordenado o sin delimitadores confunde al modelo. La calidad final depende tanto de lo que recuperas como de cómo lo presentas.

IDEAS CLAVE

- BM25 es fuerte en coincidencias exactas y vocabulario estable.
- La búsqueda densa captura paráfrasis, pero depende del modelo y del dominio.
- La combinación híbrida y el reranking suelen mejorar la precisión.
- Evalúa recall, precisión y el impacto real en la respuesta.

EJEMPLO

Para buscar incidencias por código de error, BM25 encuentra el código exacto; los embeddings encuentran descripciones similares. Un sistema híbrido recupera ambos y un reranker prioriza.

CUIDADO

No elijas búsqueda vectorial por moda. Si tus consultas son códigos o nombres exactos, la búsqueda léxica puede ser mejor y más barata.

MÓDULO 06 · LECCIÓN 3 DE 4

Agentes: herramientas, planificación y memoria

3 MIN DE LECTURA 475 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- El bucle del agente
- Planificación y estado
- Fallos y control
- Herramientas y autonomía

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un agente no es solo un prompt largo. Es un sistema que decide acciones, usa herramientas, observa resultados y continúa hasta completar una tarea.

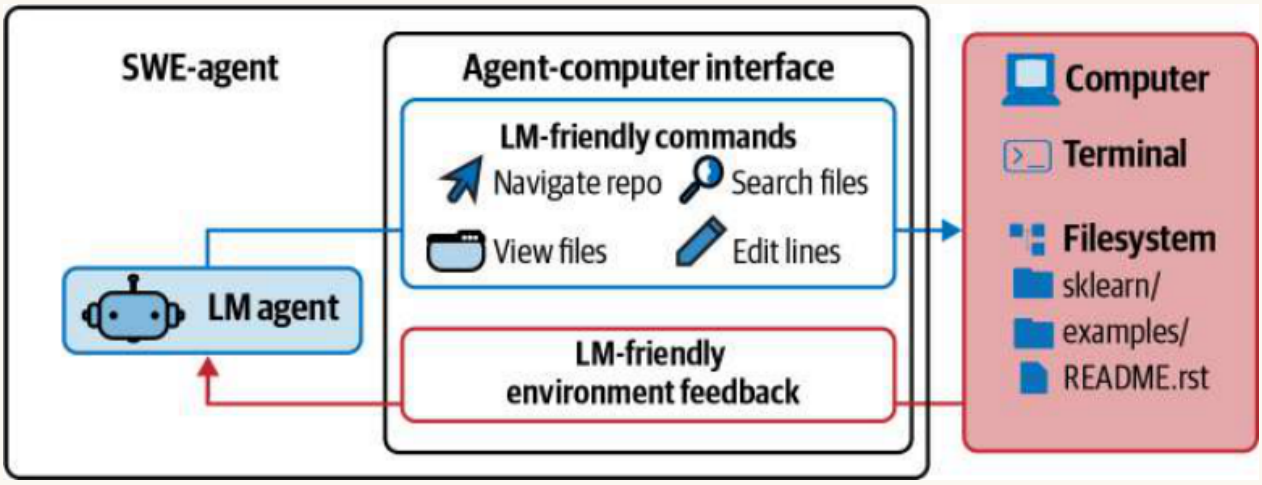


Figura 6-8. SWE-agent (Yang et al., 2024) es un agente de programación cuyo entorno es el ordenador y cuyas acciones incluyen la navegación, la búsqueda y la edición. Adaptado de una imagen original con licencia CC BY 4.0.

El bucle del agente

El patrón básico es razonar, actuar y observar. El modelo recibe un objetivo, decide una acción —buscar, calcular, llamar a una API, escribir un archivo—, recibe el resultado y decide el siguiente paso. La herramienta devuelve información que el modelo no tenía.

La calidad depende de las herramientas, de la descripción de su interfaz y de la capacidad de recuperarse de errores. Un agente con herramientas mal documentadas fallará aunque el modelo sea bueno.

Planificación y estado

Las tareas complejas requieren dividir el objetivo en pasos y mantener el estado. El agente puede planificar primero o decidir paso a paso. La planificación explícita facilita la revisión y la depuración, pero puede volverse rígida si el entorno cambia.

La memoria puede ser a corto plazo (la conversación) o a largo plazo (una base de datos). Guardar demasiado contexto aumenta coste y ruido; guardar demasiado poco impide continuidad. Define qué información debe persistir.

Fallos y control

Los agentes pueden entrar en bucles, usar herramientas incorrectas, ignorar errores o ejecutar acciones peligrosas. Necesitan límites: número máximo de pasos, permisos mínimos, confirmación humana y validación de resultados.

La evaluación de un agente no es solo la respuesta final. También importa la trayectoria: qué herramientas usó, en qué orden y con qué coste. Una respuesta correcta por accidente no es un buen sistema.

Herramientas y autonomía

Una herramienta es una interfaz para el modelo: nombre, descripción, parámetros y errores. Cuanto más clara sea, menos probable es que el agente la use mal. Define qué devuelve, qué errores puede lanzar y si la operación es idempotente.

La autonomía debe crecer por niveles. Empieza con lectura, luego acciones reversibles y solo después acciones con impacto. Para las irreversibles, exige confirmación humana. Un agente útil no es el que hace más cosas solo, sino el que hace las cosas correctas con el menor riesgo.

IDEAS CLAVE

- Un agente razona, actúa y observa en un bucle con herramientas.
- La descripción de las herramientas y el manejo de errores son críticos.
- Planificación y memoria deben equilibrar continuidad, coste y ruido.
- Limita pasos y permisos; evalúa la trayectoria, no solo el resultado.

EJEMPLO

Un agente de soporte puede consultar el estado de un pedido, calcular una fecha de entrega y crear una etiqueta. Antes de reembolsar, pide confirmación humana y registra la acción.

CUIDADO

Dar a un agente acceso amplio a herramientas es darle capacidad de causar daño. Empieza con permisos de solo lectura y añade acciones de forma gradual.

MÓDULO 06 · LECCIÓN 4 DE 4

Evaluar y optimizar RAG y agentes

3 MIN DE LECTURA 434 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Evaluar RAG por etapas
- Evaluar agentes
- Optimizar sin romper
- Diagnóstico de fallos

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

En RAG y agentes el fallo puede estar en la recuperación, en el modelo, en la herramienta o en la orquestación. La evaluación debe localizar la etapa.

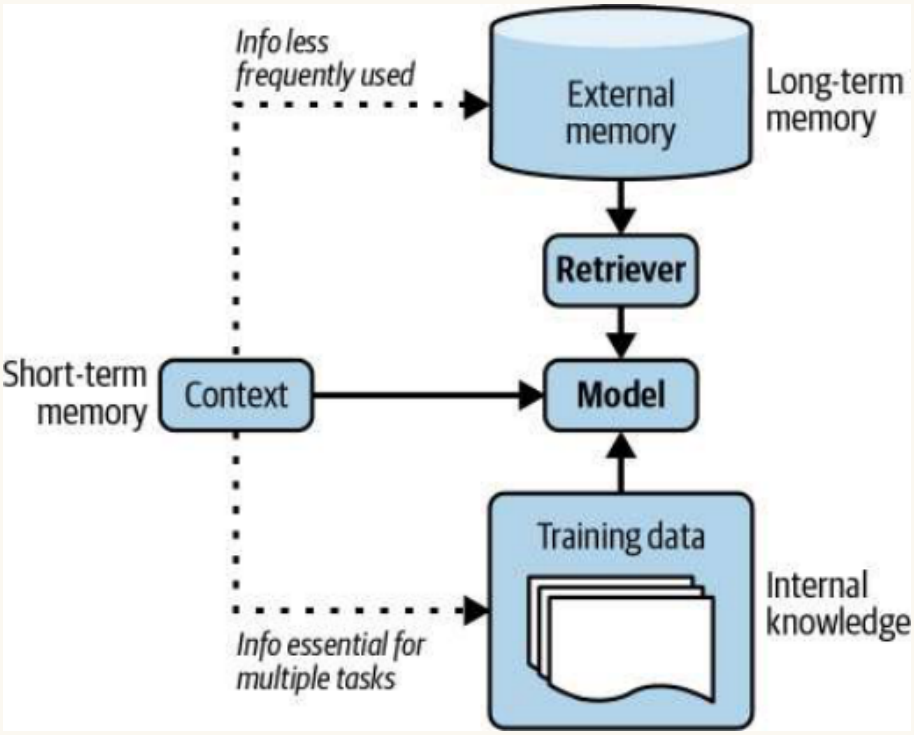


Figura 6-16. La jerarquía de la información para un agente.

Evaluar RAG por etapas

Mide la recuperación por separado: ¿aparece el documento correcto entre los candidatos? Después mide la generación: ¿la respuesta está respaldada por el contexto? La fidelidad —que no invente— y la relevancia —que responda a la pregunta— son criterios distintos.

Un conjunto de preguntas con documentos y respuestas de referencia permite comparar chunking, modelos de embedding y rerankers. Incluye preguntas sin respuesta para comprobar que el sistema sabe decir «no consta».

Evaluar agentes

Para agentes, evalúa el éxito de la tarea, el número de pasos, el coste y la seguridad de las acciones. Revisa trayectorias fallidas: a veces el resultado es correcto pero el proceso es frágil o caro.

Define escenarios con estados iniciales reproducibles. Un agente que funciona en una demo con un entorno limpio puede fallar con datos sucios, APIs lentas o errores de red.

Optimizar sin romper

Las mejoras típicas son mejores fragmentos, reranking, prompts más específicos, herramientas más simples y límites de pasos. Cambia una cosa a la vez y mide el efecto en calidad, latencia y coste.

No optimices solo la respuesta final. Un sistema que acierta más pero tarda el doble o cuesta diez veces más puede no ser viable. La evaluación debe reflejar el objetivo del producto.

Diagnóstico de fallos

Cuando un sistema RAG o un agente falla, localiza la etapa. Comprueba si el documento correcto estaba en los candidatos, si el contexto se construyó bien, si el modelo siguió las instrucciones y si la herramienta devolvió lo esperado. Sin trazas, el diagnóstico se convierte en adivinación.

Mantén un conjunto de casos con la respuesta y la trayectoria esperadas. Así puedes comparar versiones y detectar si una mejora en la respuesta final oculta un aumento de pasos, coste o riesgo. La calidad incluye el camino, no solo el destino.

IDEAS CLAVE

- Evalúa recuperación y generación por separado para localizar el fallo.
- Fidelidad y relevancia son criterios distintos; incluye preguntas sin respuesta.
- En agentes, evalúa éxito, pasos, coste y seguridad de las acciones.
- Optimiza con cambios aislados y mide calidad, latencia y coste juntos.

EJEMPLO

Si un asistente RAG responde mal, comprueba primero si el fragmento correcto estaba en el top-k. Si estaba, el problema es de generación o de prompt; si no, es de recuperación.

CUIDADO

Un agente que completa la tarea en 20 pasos puede ser técnicamente correcto y económicamente inviable. La trayectoria también es calidad.

EVALUACIÓN · MÓDULO 06

Cierre del módulo 06

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

RAG añade conocimiento externo al contexto. La recuperación, el chunking y las citas determinan la calidad.

La búsqueda léxica y la densa capturan señales distintas; la combinación y el reranking suelen mejorar.

Los agentes amplían el sistema con herramientas y acciones. Necesitan planificación, memoria, límites y evaluación de trayectoria.

Evaluación del módulo 06

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 06

¿Qué añade RAG a un modelo de lenguaje?

- [ ] A. Más parámetros
- [ ] B. Recuperación de información externa relevante antes de generar la respuesta
- [ ] C. Entrenamiento desde cero
- [ ] D. Una interfaz gráfica

Pregunta 02 · Módulo 06

¿Qué ventaja tiene la recuperación densa frente a la dispersa?

- [ ] A. Siempre es más rápida
- [ ] B. Captura similitud semántica aunque no coincidan las palabras exactas
- [ ] C. No necesita embeddings
- [ ] D. Funciona sin índice

Pregunta 03 · Módulo 06

¿Qué caracteriza a un agente?

- [ ] A. Solo genera texto
- [ ] B. Razona, usa herramientas y observa resultados para completar una tarea
- [ ] C. No necesita un modelo
- [ ] D. Siempre es autónomo y sin supervisión

RESPUESTAS Y EXPLICACIONES

01 · Opción B. Recuperación de información externa relevante antes de generar la respuesta

RAG recupera fragmentos de una base de conocimiento y los incluye en el contexto para fundamentar la respuesta.

02 · Opción B. Captura similitud semántica aunque no coincidan las palabras exactas

Los embeddings permiten recuperar por significado; la recuperación léxica depende de coincidencias de términos.

03 · Opción B. Razona, usa herramientas y observa resultados para completar una tarea

El bucle de razonamiento-acción-observación y el uso de herramientas distinguen a un agente de un simple chatbot.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Diseña un pipeline RAG para los documentos de tu empresa: fuentes, chunking, embeddings, índice, recuperación y generación.
2. Explica cuándo usarías búsqueda léxica, búsqueda densa o híbrida. Da un ejemplo de consulta para cada una.
3. Define el esquema de herramientas de un agente para gestionar incidencias: nombre, descripción, parámetros y qué pasa si falla.

07

Ajuste fino

Cuándo especializar un modelo y cómo hacerlo sin arruinarte

07.1 Qué es el ajuste fino y cuándo usarlo
07.2 Memoria, PEFT y LoRA
07.3 Tácticas de entrenamiento
07.4 Evaluar y desplegar un modelo ajustado
07.5 Cierre del módulo 07

MÓDULO 07 · LECCIÓN 1 DE 4

Qué es el ajuste fino y cuándo usarlo

3 MIN DE LECTURA 465 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Qué cambia el ajuste
- Cuándo no ajustar
- Requisitos y riesgos
- El árbol de decisión

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

El ajuste fino entrena un modelo con datos adicionales para especializarlo. No es la primera opción para todo: a veces un prompt o RAG resuelven el problema con menos coste.

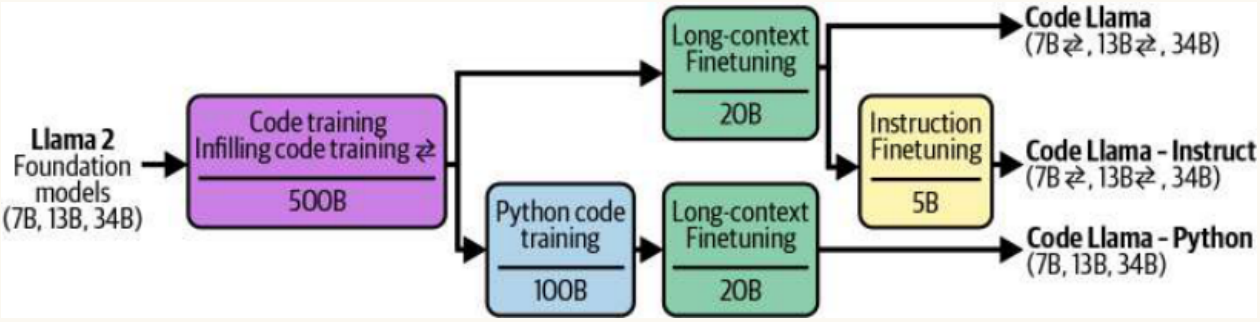


Figura 7-1. Distintas técnicas de ajuste fino utilizadas para fabricar distintos modelos Code Llama. Imagen de Rozière et al. (2024). Adaptada a partir de una imagen original con licencia CC BY 4.0.

Qué cambia el ajuste

El ajuste fino modifica los pesos del modelo, o una parte de ellos, para que aprenda un estilo, un formato, una tarea o un dominio. Puede mejorar la consistencia y reducir la necesidad de instrucciones largas. También puede enseñar a imitar el tono de una marca o a seguir un esquema complejo.

No es una base de datos. Si el conocimiento cambia con frecuencia o debe ser verificable, RAG suele ser mejor. El ajuste fino es útil cuando el comportamiento deseado es estable y tienes ejemplos de calidad.

Cuándo no ajustar

Antes de ajustar, agota las alternativas simples: mejora el prompt, añade ejemplos, recupera documentos, usa herramientas o cambia de modelo. Muchos problemas que parecen de modelo son de contexto o de evaluación.

El ajuste fino exige datos, cómputo y evaluación. Si no puedes medir una mejora clara, no sabrás si el entrenamiento ayudó o solo sobreajustó. Empieza por la opción más barata que resuelva el problema.

Requisitos y riesgos

Necesitas un conjunto de entrenamiento representativo y un conjunto de evaluación separado. También necesitas una métrica que capture lo que quieres mejorar. Sin eso, el ajuste es una apuesta.

Los riesgos incluyen sobreajuste, olvido catastrófico, sesgos en los datos y dependencia de una versión concreta. Documenta los datos y conserva una evaluación reproducible.

El árbol de decisión

Antes de ajustar, recorre un árbol: ¿puede resolverse con una instrucción mejor? ¿Falta contexto que puedas recuperar? ¿Necesitas una herramienta? ¿El formato es el problema? Si la respuesta sigue siendo no, el ajuste fino puede aportar consistencia.

El ajuste es especialmente útil cuando tienes muchos ejemplos de un comportamiento estable y difícil de describir. No lo uses para conocimiento cambiante ni para corregir datos que deberían venir de una fuente verificable. La técnica correcta depende del problema, no de la moda.

IDEAS CLAVE

- El ajuste fino especializa comportamiento; no es una base de conocimiento actualizable.
- Prompt, RAG y herramientas suelen ser alternativas más baratas y rápidas.
- Necesitas datos representativos, evaluación separada y una métrica clara.
- Vigila sobreajuste, olvido catastrófico y sesgos de los datos.

EJEMPLO

Una empresa quiere que el asistente responda siempre con su formato de marca. Un prompt con ejemplos puede bastar; si no, un ajuste fino con cientos de respuestas bien escritas puede dar más consistencia.

CUIDADO

Ajustar un modelo con documentos internos no garantiza que los recuerde ni que los cite. Para conocimiento cambiante, RAG es más adecuado.

MÓDULO 07 · LECCIÓN 2 DE 4

Memoria, PEFT y LoRA

3 MIN DE LECTURA 435 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Los cuellos de botella
- LoRA y PEFT
- Qué esperar
- Elegir la técnica de ajuste

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

El ajuste completo actualiza todos los parámetros y exige mucha memoria. Las técnicas de ajuste eficiente entrenan una parte pequeña y congelan el resto.



Figura 7-4. La pasada hacia adelante y la pasada hacia atrás de una red neuronal simple.

Los cuellos de botella

Durante el entrenamiento hay que guardar pesos, activaciones, gradientes y estados del optimizador. La memoria crece con el tamaño del modelo y del lote. Por eso un modelo que cabe para inferencia puede no caber para ajuste completo.

Las soluciones incluyen lotes más pequeños, acumulación de gradientes, precisión reducida, paralelización y técnicas de ajuste eficiente. Cada una tiene un compromiso entre memoria, velocidad y estabilidad.

LoRA y PEFT

LoRA congela los pesos originales e inserta matrices de bajo rango que se entrenan. El número de parámetros entrenables se reduce mucho, y el modelo base no cambia. Al terminar, las matrices se pueden guardar y aplicar por separado.

QLoRA combina cuantización del modelo base con LoRA. Reduce aún más la memoria, a cambio de algo de precisión y velocidad. Es una forma práctica de ajustar modelos grandes en hardware modesto.

Qué esperar

El ajuste eficiente no es magia: si los datos son malos, el resultado será malo. Tampoco garantiza el mismo rendimiento que un ajuste completo. En muchas tareas, LoRA se acerca lo suficiente y permite iterar más rápido.

Evalúa el modelo ajustado contra el base con el mismo prompt y el mismo conjunto. Compara no solo la métrica principal, sino también formato, seguridad y coste.

Elegir la técnica de ajuste

El ajuste completo da más libertad y exige mucha memoria. LoRA reduce los parámetros entrenables y permite iterar con menos hardware. QLoRA añade cuantización para bajar aún más el consumo. La elección depende del presupuesto, del tamaño del modelo y de la calidad que necesites.

Revisa la memoria disponible para pesos, activaciones y optimizador. Si no cabe, no fuerces: reduce el lote, usa acumulación de gradientes o cambia de técnica. Un entrenamiento inestable por falta de memoria produce resultados peores que un LoRA bien configurado.

IDEAS CLAVE

- El ajuste completo necesita memoria para pesos, gradientes y optimizador.
- LoRA entrena matrices pequeñas y congela el modelo base.
- QLoRA combina cuantización y LoRA para reducir memoria.
- El ajuste eficiente abarata la iteración, pero no compensa datos deficientes.

EJEMPLO

Un equipo con una GPU de 24 GB puede ajustar un modelo de 7B con QLoRA y un dataset pequeño, evaluar el resultado y comparar con el modelo base antes de invertir en más hardware.

CUIDADO

Un adaptador LoRA no añade conocimiento fiable por sí mismo. Sigue siendo un cambio de comportamiento aprendido de ejemplos.

MÓDULO 07 · LECCIÓN 3 DE 4

Tácticas de entrenamiento

3 MIN DE LECTURA 414 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Datos de entrenamiento
- Hiperparámetros y estabilidad
- Multitarea y fusión
- Preparar el dataset de ajuste

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

La diferencia entre un ajuste útil y uno que arruina el modelo suele estar en los datos y en la evaluación, no en el optimizador.

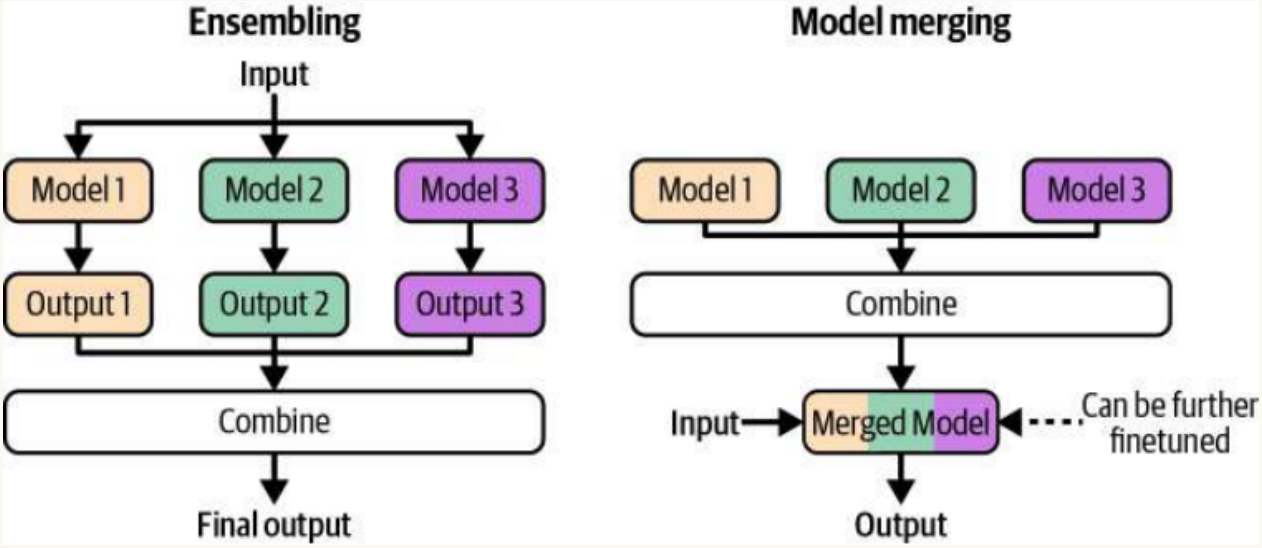


Figura 7-13. Cómo funcionan el ensembling y la fusión de modelos.

Datos de entrenamiento

La calidad importa más que la cantidad. Ejemplos consistentes, bien etiquetados y representativos del uso real producen mejores resultados que miles de ejemplos ruidosos. Incluye casos límite y ejemplos de lo que el modelo no debe hacer.

El formato debe coincidir con el de inferencia. Si entrenas con un formato y luego usas otro, el modelo puede comportarse de forma distinta. Revisa plantillas, roles y tokens especiales.

Hiperparámetros y estabilidad

La tasa de aprendizaje, el número de épocas, el tamaño del lote y la longitud de secuencia afectan al resultado. Un exceso de épocas sobreajusta; una tasa demasiado alta desestabiliza. Empieza con valores conservadores y usa el conjunto de evaluación para decidir.

El sobreajuste puede manifestarse como respuestas que copian el conjunto de entrenamiento o pierden capacidad general. Evalúa también en tareas no vistas.

Multitarea y fusión

Puedes entrenar un modelo para varias tareas o fusionar adaptadores entrenados por separado. La multitarea puede mejorar la generalización si las tareas comparten estructura; también puede crear interferencia.

La fusión de modelos combina pesos o adaptadores. Es una técnica experimental: evalúa siempre el resultado, porque la combinación puede degradar capacidades que cada modelo tenía por separado.

Preparar el dataset de ajuste

El formato de entrenamiento debe parecerse al de inferencia. Si en producción envías un sistema, un usuario y un contexto, entrena con esa misma estructura. Si usas tokens especiales, respétalos. Una discrepancia de formato puede anular la mejora.

Incluye variedad: casos fáciles, difíciles y ejemplos de lo que no debe hacerse. Deduplica y separa evaluación. Revisa una muestra antes de entrenar y después del entrenamiento. La calidad del dataset es la variable que más influye y la que menos se puede arreglar con hiperparámetros.

IDEAS CLAVE

- La calidad y consistencia de los datos superan la cantidad.
- El formato de entrenamiento debe coincidir con el de inferencia.
- Ajusta hiperparámetros con un conjunto de evaluación y vigila el sobreajuste.
- Multitarea y fusión pueden ayudar, pero introducen interferencia; evalúa siempre.

EJEMPLO

Para un asistente de estilo editorial, entrena con pares de borrador y versión final, mantén un conjunto de artículos no vistos y comprueba que el modelo no copia frases literales.

CUIDADO

Más épocas no significan mejor modelo. El sobreajuste puede mejorar el conjunto de entrenamiento y empeorar el producto.

MÓDULO 07 · LECCIÓN 4 DE 4

Evaluar y desplegar un modelo ajustado

3 MIN DE LECTURA 451 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Evaluación antes y después
- Coste de servir
- Mantenimiento
- Desplegar adaptadores

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un ajuste fino solo es útil si mejora el sistema en producción sin introducir riesgos. La evaluación y el despliegue son parte del mismo trabajo.

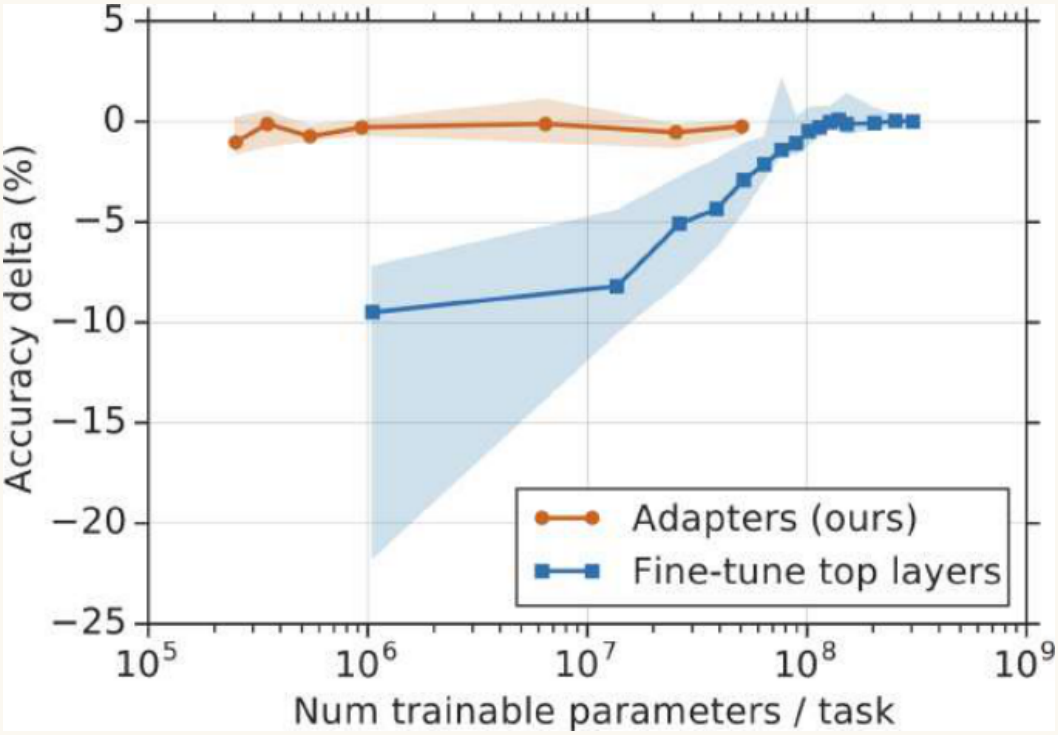


Figura 7-7. La línea azul muestra que el ajuste fino parcial requiere muchos parámetros entrenables para lograr un rendimiento comparable al del ajuste fino completo. Imagen de Houlsby et al. (2019).

Evaluación antes y después

Compara el modelo ajustado con el base usando el mismo prompt, la misma temperatura y el mismo conjunto. Mide la tarea principal, pero también formato, seguridad y capacidad general. Un modelo que mejora el estilo puede empeorar el razonamiento.

Incluye casos adversariales y ejemplos fuera de distribución. Si el modelo solo funciona con datos parecidos a los de entrenamiento, no está generalizando.

Coste de servir

El adaptador añade poco coste de almacenamiento, pero el modelo base sigue ocupando memoria. Si sirves muchos adaptadores, necesitas una infraestructura que los cargue y descargue eficientemente. Si el modelo es grande, el coste de inferencia puede ser mayor que el de un modelo pequeño ajustado.

Compara el coste total: entrenamiento, serving, latencia y mantenimiento. Un ajuste que mejora un 2 % pero duplica el coste puede no valer la pena.

Mantenimiento

El modelo ajustado también envejece. Los datos cambian, el proveedor actualiza el base y los usuarios encuentran nuevos casos. Versiona el adaptador, los datos y la evaluación. Prepara una ruta de reversión.

Si el ajuste deja de aportar, vuelve al modelo base o a una solución con prompt/RAG. La decisión debe basarse en métricas, no en el coste hundido del entrenamiento.

Desplegar adaptadores

Servir un adaptador no es solo cargar un archivo. Necesitas una versión del modelo base compatible, una forma de enrutar peticiones y una estrategia para actualizar o revertir. Si sirves varios adaptadores, la infraestructura debe cargarlos sin multiplicar la memoria del modelo base.

Mide latencia y coste después de añadir el adaptador. Compara con el modelo base y con un prompt alternativo. Un adaptador que mejora la métrica pero duplica el coste puede no ser rentable. Versiona el adaptador junto con los datos y la evaluación que lo justifican.

IDEAS CLAVE

- Compara el modelo ajustado con el base usando el mismo prompt y conjunto.
- Evalúa la tarea principal, el formato, la seguridad y la capacidad general.
- El coste total incluye entrenamiento, serving, latencia y mantenimiento.
- Versiona el adaptador y prepárate para revertir si deja de aportar.

EJEMPLO

Un adaptador mejora la clasificación de tickets en un 4 %, pero añade 30 ms y requiere una GPU dedicada. La decisión depende de si esa mejora reduce el coste de soporte lo suficiente.

CUIDADO

No despliegues un ajuste solo porque la pérdida de entrenamiento bajó. La métrica que importa es la del producto.

EVALUACIÓN · MÓDULO 07

Cierre del módulo 07

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

El ajuste fino especializa comportamiento y estilo. No es la herramienta adecuada para conocimiento cambiante; para eso está RAG.

LoRA y QLoRA reducen la memoria y permiten iterar, pero dependen de datos de calidad y de una evaluación honesta.

El despliegue exige comparar con el base, medir el coste total y mantener una ruta de reversión.

Evaluación del módulo 07

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 07

¿Cuándo es más razonable usar RAG en lugar de ajuste fino?

- [ ] A. Cuando el conocimiento cambia con frecuencia y necesitas citar fuentes
- [ ] B. Cuando quieres cambiar el estilo del modelo para siempre
- [ ] C. Cuando no tienes documentos
- [ ] D. Cuando necesitas reducir el tamaño del modelo

Pregunta 02 · Módulo 07

¿Qué hace LoRA?

- [ ] A. Reentrena todos los parámetros
- [ ] B. Añade matrices de bajo rango entrenables y congela el modelo base
- [ ] C. Cuantiza el modelo
- [ ] D. Elimina capas del transformer

Pregunta 03 · Módulo 07

¿Qué problema resuelve la cuantización?

- [ ] A. Mejora la creatividad
- [ ] B. Reduce la precisión numérica para disminuir memoria y coste de inferencia
- [ ] C. Aumenta el vocabulario
- [ ] D. Elimina la necesidad de evaluar

RESPUESTAS Y EXPLICACIONES

01 · Opción A. Cuando el conocimiento cambia con frecuencia y necesitas citar fuentes

RAG es ideal para conocimiento dinámico y verificable; el ajuste fino moldea comportamiento y estilo.

02 · Opción B. Añade matrices de bajo rango entrenables y congela el modelo base

LoRA reduce drásticamente los parámetros entrenables y la memoria necesaria para el ajuste.

03 · Opción B. Reduce la precisión numérica para disminuir memoria y coste de inferencia

Representar pesos en 8 o 4 bits reduce el tamaño del modelo y acelera la inferencia, con cierta pérdida de calidad.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Decide si tu caso necesita RAG, ajuste fino o ambos. Escribe la justificación en cinco líneas.
2. Calcula a grandes rasgos la memoria necesaria para ajustar un modelo de 7B con Adam en fp16. ¿Qué técnica reduciría el coste?
3. Diseña un dataset de ajuste fino de 500 ejemplos: de dónde saldrían, cómo los limpiarías y cómo evitarías sobreajustar.

08

Ingeniería de datasets

Calidad, cobertura, anotación y datos sintéticos

08.1 Calidad antes que cantidad
08.2 Adquisición, anotación y catálogos
08.3 Datos sintéticos y destilación
08.4 Procesar el dataset
08.5 Cierre del módulo 08

MÓDULO 08 · LECCIÓN 1 DE 4

Calidad antes que cantidad

3 MIN DE LECTURA 452 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Dimensiones de calidad
- Cobertura y diversidad
- Sesgos y licencias
- Auditar un dataset

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Un modelo aprende de los datos que recibe. Si el dataset es incoherente, sesgado o incompleto, el ajuste o la evaluación heredarán esos problemas.

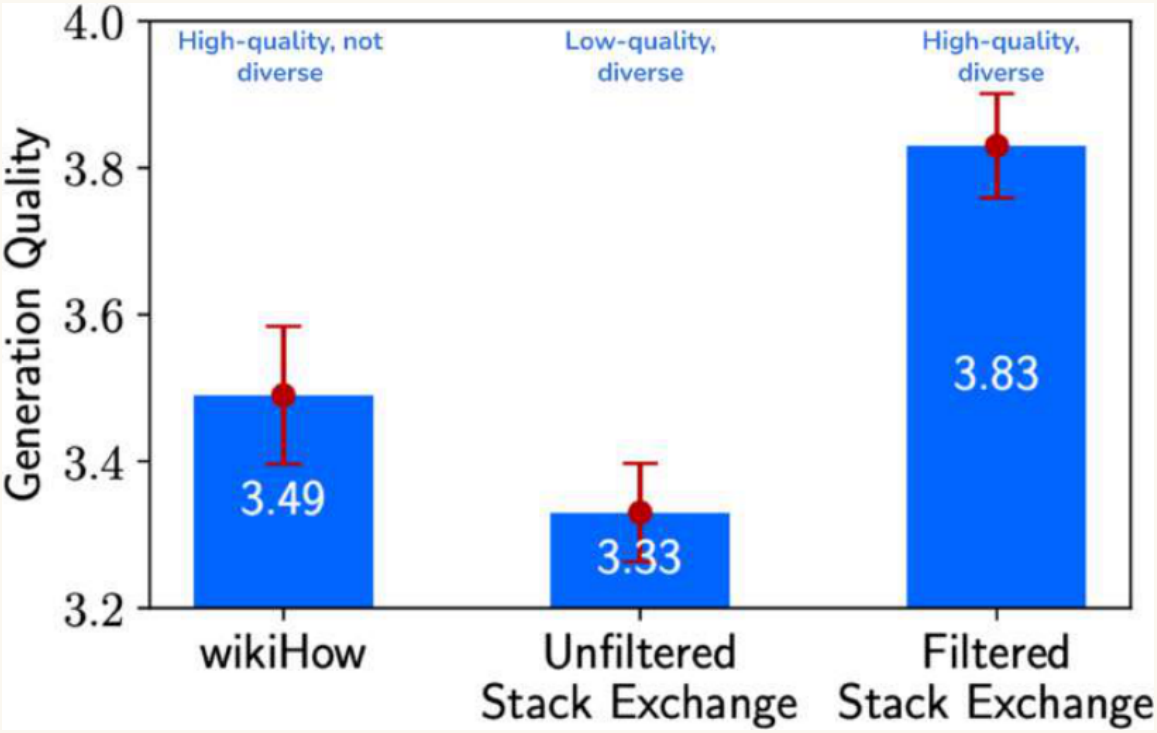


Figura 8-1. Un modelo de 7B parámetros, ajustado de forma fina con un dataset que es a la vez de alta calidad y diverso, supera a ese mismo modelo ajustado de forma fina con un dataset que es o bien diverso o bien de alta calidad. Imagen de Zhou et al. (2023). La imagen está bajo la licencia CC BY 4.0.

Dimensiones de calidad

La calidad incluye corrección, consistencia, relevancia y formato. Un ejemplo correcto pero irrelevante para la tarea no ayuda; uno relevante pero mal etiquetado enseña un error. La consistencia es especialmente importante: dos ejemplos similares con etiquetas opuestas confunden al modelo.

La limpieza no es solo eliminar duplicados. Incluye normalizar formatos, corregir errores evidentes, eliminar contenido tóxico o personal y documentar las decisiones. Cada paso debe ser reproducible.

Cobertura y diversidad

La cobertura mide si el dataset representa los casos que el modelo encontrará. Un dataset de soporte con solo preguntas fáciles no prepara para clientes enfadados o consultas ambiguas. La diversidad reduce el riesgo de que el modelo aprenda atajos.

La cantidad ayuda hasta cierto punto. Un dataset pequeño y bien curado puede superar a uno grande y ruidoso, sobre todo en ajuste fino. Mide el rendimiento por subgrupo para detectar huecos.

Sesgos y licencias

Los datos pueden reflejar sesgos históricos, de anotación o de muestreo. Si ciertos grupos aparecen poco, el modelo puede rendir peor en ellos. La auditoría por subgrupo es parte de la evaluación.

La licencia y la privacidad no son opcionales. Comprueba si puedes usar, modificar y redistribuir los datos. Anonimiza información personal y documenta la cadena de procedencia.

Auditar un dataset

Una auditoría básica revisa esquema, valores faltantes, duplicados, etiquetas contradictorias y distribución por subgrupo. También comprueba la procedencia: quién lo generó, cuándo y con qué permiso. Muchos problemas se detectan antes de entrenar.

No te quedes en la media. Mira las clases minoritarias y los casos límite. Si un grupo está poco representado, el modelo probablemente fallará en él. Documenta los hallazgos y decide si corriges, excluyes o recoges más datos.

IDEAS CLAVE

- La calidad incluye corrección, consistencia, relevancia y formato.
- La cobertura y la diversidad importan más que el volumen bruto.
- Audita el rendimiento por subgrupo para detectar sesgos y huecos.
- Verifica licencias y anonimiza datos personales antes de entrenar.

EJEMPLO

Para un clasificador de reclamaciones, revisa la distribución por tipo de producto, tono y canal. Si el chat apenas aparece, el modelo probablemente fallará en ese canal.

CUIDADO

Un dataset grande puede ser peor que uno pequeño si está dominado por ejemplos repetidos o mal etiquetados.

MÓDULO 08 · LECCIÓN 2 DE 4

Adquisición, anotación y catálogos

3 MIN DE LECTURA 389 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Fuentes de datos
- Anotación y guías
- Catálogos y gobernanza
- Diseñar el proceso de anotación

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Conseguir datos es solo el principio. Convertirlos en ejemplos útiles requiere guías, anotadores, control de calidad y un catálogo que permita encontrarlos y reutilizarlos.

Fuentes de datos

Los datos pueden venir de usuarios, procesos internos, compras, web scraping o generación sintética. Cada fuente tiene sesgos y restricciones. Los datos de producción son valiosos porque reflejan el uso real, pero pueden contener información personal o ruido.

Antes de usar una fuente, define qué representa y qué falta. Un dataset de conveniencia puede ser fácil de obtener y poco representativo.

Anotación y guías

Una guía de anotación define categorías, ejemplos y criterios para los desacuerdos. Sin ella, cada anotador aplica su propio criterio y el dataset se vuelve inconsistente. Prueba la guía con una muestra y mide el acuerdo entre anotadores.

La calidad de la anotación no es gratis. Formar, revisar y medir el acuerdo consume tiempo, pero es más barato que entrenar con etiquetas ruidosas y descubrirlo tarde.

Catálogos y gobernanza

Un catálogo de datos registra origen, licencia, versión, cobertura y transformaciones. Permite reutilizar datasets, reproducir experimentos y auditar sesgos. También ayuda a evitar que dos equipos etiqueten lo mismo de forma distinta.

La gobernanza incluye permisos, retención y eliminación. Si un usuario pide borrar sus datos, el sistema debe poder localizarlos y eliminarlos también de los datasets derivados.

Diseñar el proceso de anotación

Una guía de anotación debe definir categorías, ejemplos positivos y negativos, y qué hacer ante la duda. Prueba la guía con una muestra pequeña, mide el acuerdo y ajusta las definiciones. Después forma a los anotadores y establece una revisión por muestreo.

El coste de anotar bien es visible; el coste de anotar mal es invisible hasta que el modelo falla. Registra quién etiquetó cada ejemplo y cuántas revisiones pasó. La trazabilidad permite auditar y mejorar el criterio con el tiempo.

IDEAS CLAVE

- Cada fuente tiene sesgos y restricciones; define qué representa.
- Una guía de anotación y la medición del acuerdo son esenciales.
- Un catálogo documenta origen, licencia, versión y transformaciones.
- La gobernanza debe permitir localizar y eliminar datos personales.

EJEMPLO

Un equipo de anotación etiqueta 2.000 tickets con una guía de 10 categorías. Antes de escalar, mide el acuerdo en 200 ejemplos y ajusta las definiciones ambiguas.

CUIDADO

Etiquetar más datos sin una guía clara solo multiplica la inconsistencia. Primero define el criterio, después escala.

MÓDULO 08 · LECCIÓN 3 DE 4

Datos sintéticos y destilación

3 MIN DE LECTURA 399 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Generación y destilación
- Verificación
- Riesgos y límites
- Cuándo usar datos sintéticos

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Los datos sintéticos amplían la cobertura cuando faltan ejemplos reales. También introducen riesgos: pueden ser repetitivos, incorrectos o reflejar los sesgos del modelo que los generó.

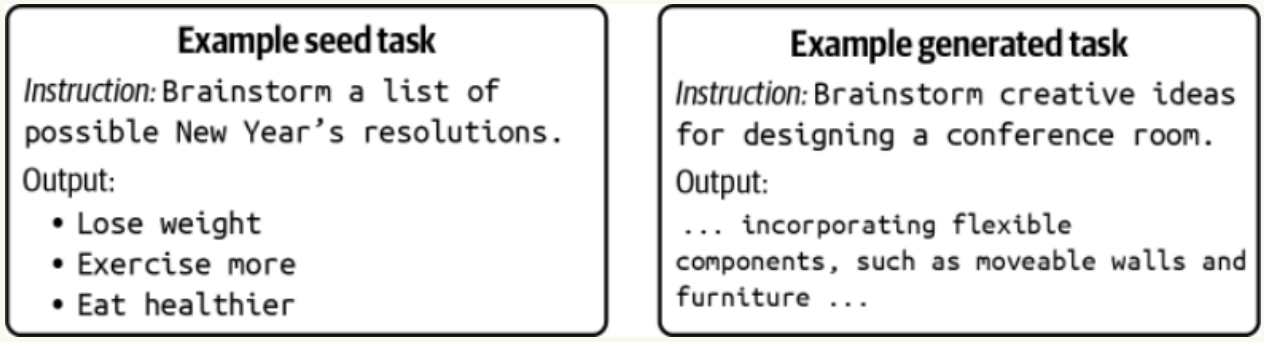


Figura 8-5. Una tarea semilla y una tarea generada utilizadas para entrenar a Alpaca.

Generación y destilación

La generación sintética crea ejemplos nuevos a partir de plantillas, semillas o modelos. La destilación entrena un modelo pequeño para imitar las salidas de uno grande. Ambas técnicas pueden reducir coste y aumentar la cobertura.

La destilación no copia la inteligencia del profesor: copia su comportamiento en los datos disponibles. Si el profesor se equivoca o el conjunto es estrecho, el estudiante hereda esos límites.

Verificación

No uses datos sintéticos sin filtrar. Comprueba formato, coherencia y corrección con reglas, pruebas o un modelo verificador. Para tareas con respuesta objetiva, valida automáticamente; para tareas abiertas, revisa una muestra.

La diversidad también importa. Si generas 10.000 variaciones de la misma plantilla, el modelo aprende la plantilla, no la tarea. Varía el contexto, el estilo y los casos límite.

Riesgos y límites

Los datos sintéticos pueden amplificar sesgos, introducir errores sutiles o crear un ciclo en el que un modelo se entrena con sus propias salidas. También pueden violar licencias si imitan contenido protegido.

Úsalos como complemento, no como sustituto universal de datos reales. Mantén una evaluación con datos humanos y documenta qué parte del entrenamiento es sintética.

Cuándo usar datos sintéticos

Los datos sintéticos son útiles cuando faltan ejemplos raros, cuando no puedes usar datos reales por privacidad o cuando necesitas aumentar la diversidad. También sirven para crear casos adversariales de forma controlada.

Úsalos con verificación y con una proporción limitada. Si todo el entrenamiento es sintético, el modelo aprende las peculiaridades del generador. Combina ejemplos reales, sintéticos verificados y casos límite. Y documenta qué parte del dataset es sintética para poder auditar el resultado.

IDEAS CLAVE

- Los datos sintéticos amplían cobertura, pero necesitan verificación.
- La destilación imita el comportamiento del profesor en los datos disponibles.
- La diversidad de plantillas evita que el modelo aprenda atajos.
- Combina datos sintéticos con datos reales y evalúa con humanos.

EJEMPLO

Para un clasificador con pocas clases raras, genera ejemplos con un modelo grande, valida el formato y la etiqueta con reglas, y revisa una muestra antes de añadirlos al entrenamiento.

CUIDADO

Entrenar con salidas de un modelo sin verificar puede perpetuar errores y reducir la diversidad. La comodidad no es calidad.

MÓDULO 08 · LECCIÓN 4 DE 4

Procesar el dataset

3 MIN DE LECTURA 385 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Limpieza y deduplicación
- Privacidad y seguridad
- Particiones y contaminación
- Reproducibilidad del pipeline

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

El procesamiento transforma datos crudos en un dataset reproducible. Es una etapa de ingeniería, no un script improvisado.

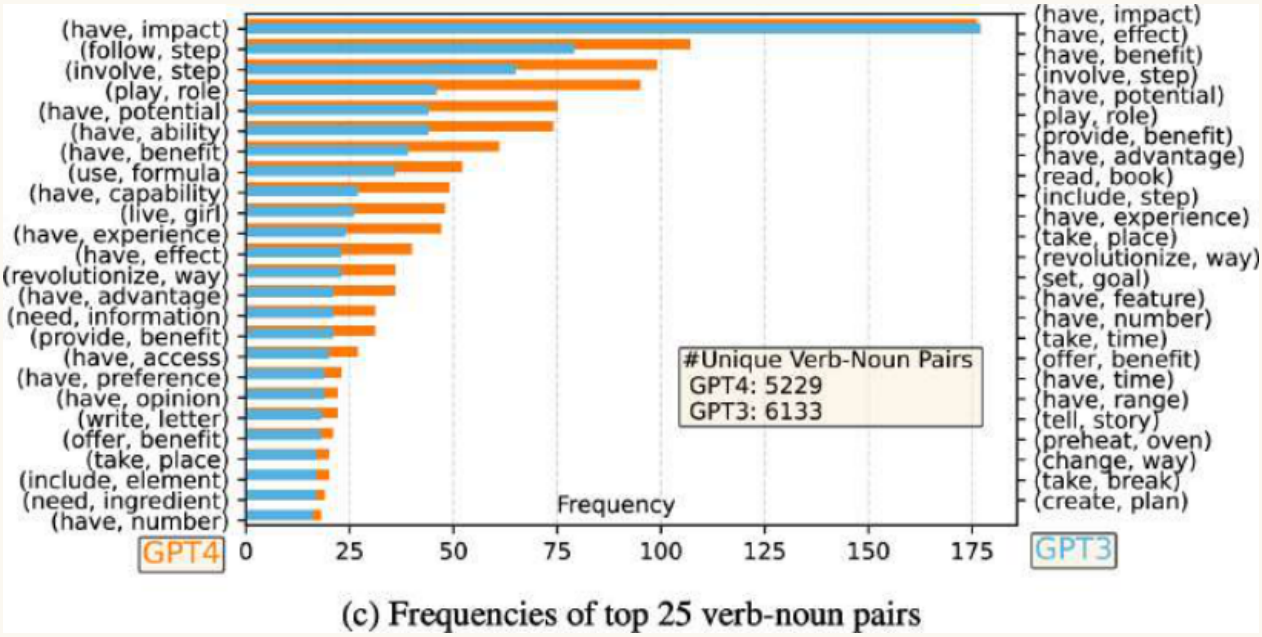


Figura 8-6. Una estadística que puede usar es la distribución de (verbo, objeto directo, sustantivo) en sus datos. Imagen de «Ajuste de instrucciones con GPT-4» (Peng et al., 2023).

Limpieza y deduplicación

La limpieza elimina ruido, corrige formatos y descarta ejemplos inválidos. La deduplicación evita que el modelo memorice ejemplos repetidos y que la evaluación se infle por copias. La deduplicación exacta es fácil; la semántica requiere embeddings o reglas.

Documenta cada transformación. Si eliminas el 30 % de los datos, querrás saber por qué y poder reproducirlo.

Privacidad y seguridad

Detecta y elimina información personal identificable, credenciales y contenido sensible. La anonimización por sustitución puede no ser suficiente si el contexto permite reidentificar. En dominios regulados, consulta los requisitos aplicables.

También revisa contenido tóxico o ilegal. Un dataset limpio no es solo un dataset sin duplicados; es un dataset que puedes usar sin causar daño.

Particiones y contaminación

Separa entrenamiento, validación y prueba. La prueba no debe usarse para ajustar hiperparámetros ni prompts. Comprueba que no haya fugas entre particiones: un ejemplo casi idéntico en entrenamiento y prueba infla los resultados.

Versiona el dataset y las particiones. Si cambias la limpieza, cambia la versión y vuelve a evaluar. La reproducibilidad es parte de la calidad.

Reproducibilidad del pipeline

Un pipeline reproducible guarda la versión del código, los datos de entrada, las transformaciones y las particiones. Si cambias un filtro, cambia la versión y vuelve a evaluar. Sin esto, no puedes saber si una mejora viene del modelo o de una fuga en los datos.

Automatiza las comprobaciones: columnas esperadas, rangos, duplicados y solapamiento entre particiones. Un pipeline que falla pronto y de forma clara es mejor que uno que produce un dataset silenciosamente corrupto.

IDEAS CLAVE

- Limpia, normaliza y deduplica antes de entrenar o evaluar.
- Elimina información personal y contenido sensible; documenta el proceso.
- Separa entrenamiento, validación y prueba, y evita fugas.
- Versiona datasets y transformaciones para reproducir resultados.

EJEMPLO

Antes de ajustar un modelo, elimina tickets duplicados, anonimiza nombres y correos, separa un 15 % para prueba y comprueba que ningún ticket de prueba aparece en entrenamiento.

CUIDADO

Una fuga entre entrenamiento y prueba puede hacer que un modelo parezca excelente y falle en producción.

EVALUACIÓN · MÓDULO 08

Cierre del módulo 08

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

La calidad, cobertura y diversidad del dataset determinan lo que el modelo puede aprender. La cantidad no compensa el ruido.

La anotación necesita guías y control de acuerdo; el catálogo y la gobernanza permiten reutilizar y auditar.

Los datos sintéticos y la destilación amplían cobertura, pero requieren verificación. El procesamiento y las particiones hacen el dataset reproducible.

Evaluación del módulo 08

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 08

¿Por qué la calidad de datos importa más que la cantidad?

- [ ] A. Porque los datos duplicados mejoran el modelo
- [ ] B. Porque el modelo aprende los errores, sesgos y ruido presentes en los datos
- [ ] C. Porque los datos pequeños siempre son mejores
- [ ] D. Porque el volumen no afecta al entrenamiento

Pregunta 02 · Módulo 08

¿Qué es la destilación de modelos?

- [ ] A. Eliminar capas del modelo
- [ ] B. Entrenar un modelo pequeño para imitar las salidas de uno grande
- [ ] C. Filtrar datos duplicados
- [ ] D. Convertir texto en embeddings

Pregunta 03 · Módulo 08

¿Qué riesgo tiene entrenar con datos sintéticos?

- [ ] A. Siempre son datos perfectos
- [ ] B. Pueden heredar errores y sesgos del modelo que los generó, y colapsar la diversidad
- [ ] C. No se pueden usar para evaluar
- [ ] D. Ocupan demasiado poco espacio

RESPUESTAS Y EXPLICACIONES

01 · Opción B. Porque el modelo aprende los errores, sesgos y ruido presentes en los datos

«Basura entra, basura sale»: la calidad del dataset condiciona lo que el modelo aprende.

02 · Opción B. Entrenar un modelo pequeño para imitar las salidas de uno grande

La destilación transfiere el comportamiento de un modelo maestro a un modelo alumno más eficiente.

03 · Opción B. Pueden heredar errores y sesgos del modelo que los generó, y colapsar la diversidad

Los datos sintéticos amplifican los patrones del generador; sin filtrado y diversidad pueden degradar el modelo.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Audita un dataset tuyo: cuenta duplicados, ejemplos vacíos, sesgos evidentes y cobertura de casos límite.
2. Diseña una estrategia de aumento de datos para una tarea con pocos ejemplos. ¿Qué técnicas tradicionales usarías antes de generar con IA?
3. Escribe un plan de limpieza y formateo reproducible: inspección, deduplicación, filtrado, formato y validación.

09

Optimización de inferencia

Latencia, throughput, memoria y coste

09.1 Qué significa optimizar la inferencia

09.2 Métricas: latencia, throughput y memoria

09.3 Aceleradores, cuantización y serving

09.4 Optimizar coste y arquitectura

09.5 Cierre del módulo 09

MÓDULO 09 · LECCIÓN 1 DE 4

Qué significa optimizar la inferencia

3 MIN DE LECTURA 433 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Prefill y decodificación
- Limitado por cómputo o memoria
- Online y batch
- Presupuestar la latencia

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

La inferencia es la etapa que ve el usuario. Optimizarla no es solo comprar GPU: es entender dónde se va el tiempo y la memoria en cada fase.

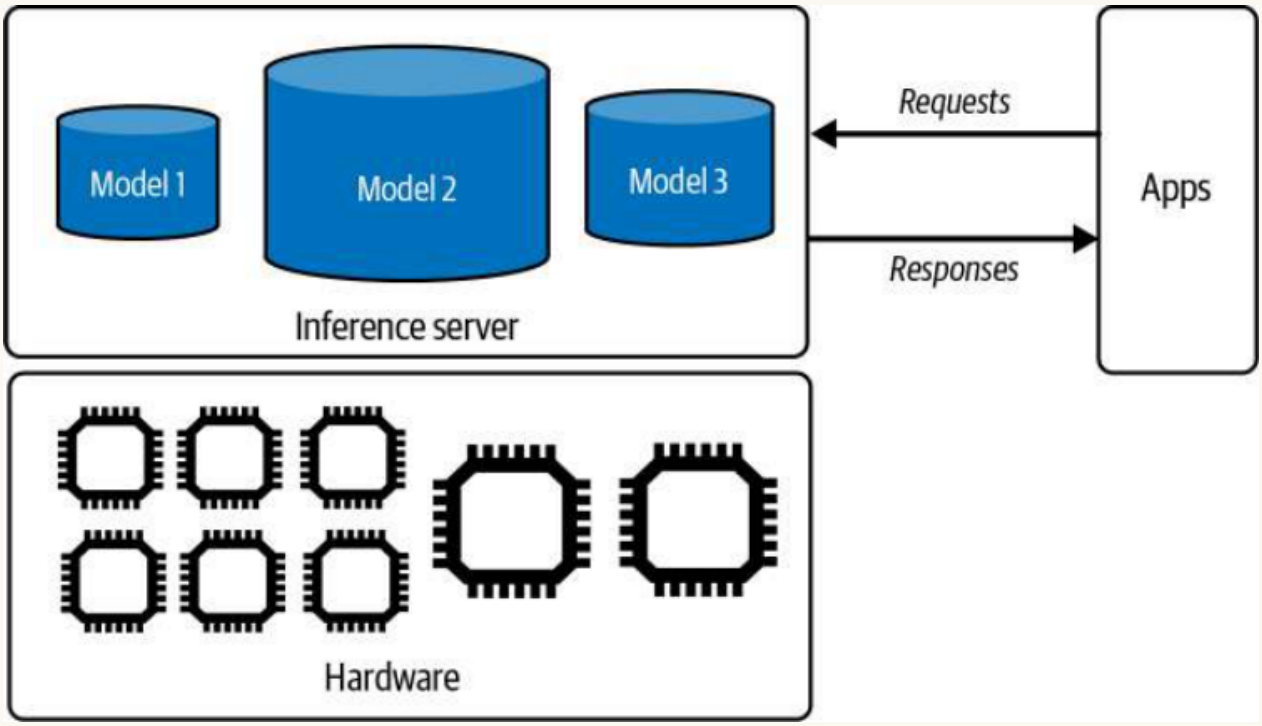


Figura 9-1. Un servicio de inferencia simple.

Prefill y decodificación

La generación tiene dos fases. En el prefill, el modelo procesa el prompt completo y calcula el estado interno; es intensivo en cómputo y puede paralelizarse bien. En la decodificación, genera un token cada vez, reutilizando el estado anterior; suele estar limitada por el ancho de banda de memoria.

Esa diferencia explica por qué un prompt largo aumenta el tiempo hasta el primer token, mientras que una respuesta larga aumenta el tiempo total. Optimizar una fase no necesariamente mejora la otra.

Limitado por cómputo o memoria

Un sistema limitado por cómputo no aprovecha toda la capacidad de la GPU porque espera operaciones. Uno limitado por memoria mueve datos más despacio de lo que el procesador puede calcular. La fase de decodificación suele estar más cerca del segundo caso.

Las técnicas de optimización atacan cuellos de botella distintos: batching para aprovechar la GPU, cuantización para reducir memoria, caché para evitar recomputar y kernels especializados para acelerar operaciones.

Online y batch

La inferencia en línea prioriza latencia y disponibilidad. La inferencia por lotes prioriza throughput y coste, y puede tolerar minutos u horas de espera. No se optimizan igual: en línea necesitas colas, autoscaling y límites; en batch puedes agrupar y usar hardware más barato.

Elige el modo según el producto. Un informe nocturno no necesita la misma arquitectura que un chat en vivo.

Presupuestar la latencia

Descompón la latencia total: cola, red, preprocesado, recuperación, tiempo hasta el primer token y generación. Cada etapa tiene un responsable distinto. Si no sabes cuál domina, optimizarás la equivocada.

Fija un presupuesto por etapa y mide con carga real. A veces la mayor ganancia no está en el modelo, sino en eliminar una llamada, cachear una recuperación o reducir el prompt. La latencia percibida también mejora con streaming y con respuestas parciales útiles.

IDEAS CLAVE

- Prefill procesa el prompt; decodificación genera token a token.
- La decodificación suele estar limitada por memoria; el prefill, por cómputo.
- Batching, cuantización y caché atacan cuellos de botella distintos.
- La inferencia en línea y por lotes tienen objetivos opuestos.

EJEMPLO

Un chat con prompts largos y respuestas cortas puede mejorar sobre todo el prefill; un generador de artículos largos puede beneficiarse más de la optimización de decodificación y del KV cache.

CUIDADO

Optimizar solo el throughput puede empeorar la latencia individual. Mide la métrica que el usuario percibe, no solo la utilización de la GPU.

MÓDULO 09 · LECCIÓN 2 DE 4

Métricas: latencia, throughput y memoria

3 MIN DE LECTURA 452 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Latencia
- Throughput y goodput
- Memoria
- Medir con carga real

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Una sola cifra no describe el rendimiento. Latencia, throughput y memoria cuentan historias distintas, y los percentiles revelan lo que la media esconde.

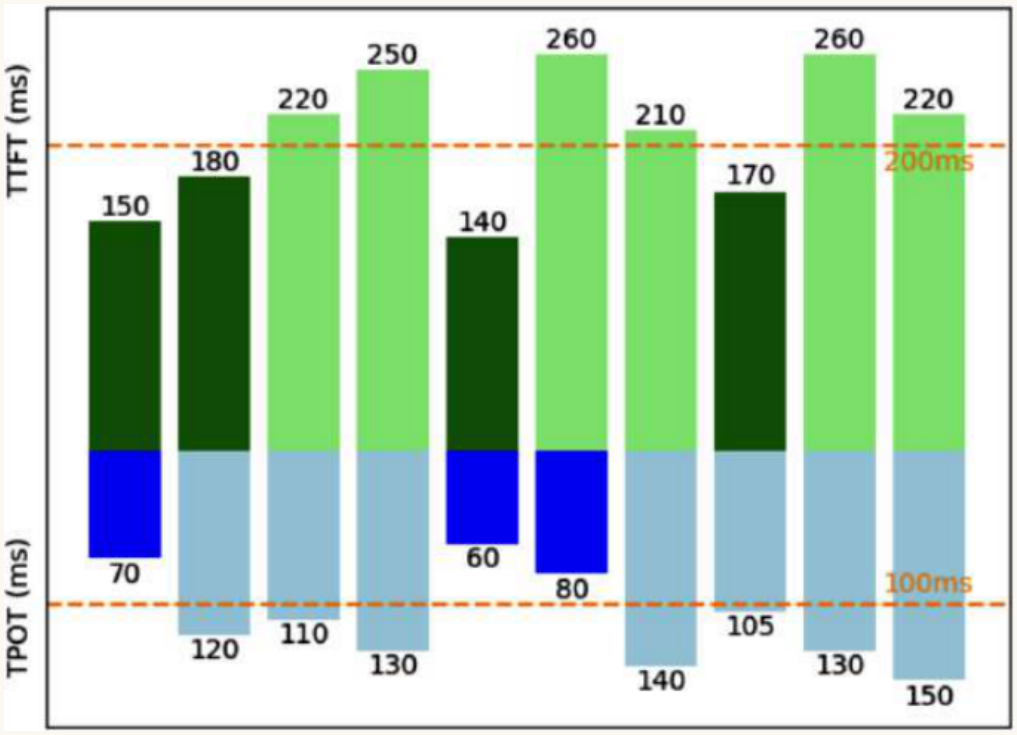


Figura 9-4. Si un servicio de inferencia puede completar 10 RPS pero solo 3 satisfacen el SLO, su goodput es de 3 RPS.

Latencia

El tiempo hasta el primer token —TTFT— mide la espera inicial. El tiempo por token de salida —TPOT— mide la velocidad de generación. La latencia total combina ambos y depende de la longitud del prompt y de la respuesta.

La media puede ocultar picos. Mide p50, p95 y p99. Un sistema con p50 excelente y p99 terrible puede ser inaceptable para usuarios que caen en la cola.

Throughput y goodput

El throughput mide cuántas peticiones o tokens se procesan por unidad de tiempo. El goodput mide cuántas cumplen el objetivo de servicio. Un sistema puede procesar 100 peticiones por segundo y solo 60 dentro del límite de latencia; su goodput es 60.

El throughput agregado no sirve si las peticiones lentas bloquean a las rápidas. Mide la distribución y la calidad del servicio, no solo la capacidad bruta.

Memoria

La memoria limita el tamaño del modelo, el tamaño del lote y la longitud del contexto. El KV cache guarda estados de atención y crece con la conversación. Si se llena, el sistema reduce el lote o rechaza peticiones.

La cuantización reduce la memoria de los pesos, pero puede afectar a la calidad. El offloading mueve datos a CPU o disco, a cambio de latencia. Cada técnica tiene un coste.

Medir con carga real

Una prueba con una sola petición no representa producción. Haz pruebas de carga con concurrencia, prompts de distinta longitud y ráfagas. Mide TTFT, TPOT y latencia total en p50, p95 y p99. Incluye el tiempo de espera en cola.

El goodput te dice cuántas peticiones cumplen el objetivo de servicio. Un sistema puede tener mucha capacidad y aun así incumplir para una parte de usuarios. Ajusta el tamaño del lote y el autoscaling con esos datos, no con la utilización media de la GPU.

IDEAS CLAVE

- TTFT mide la espera inicial; TPOT, la velocidad de generación.
- Los percentiles p95 y p99 importan más que la media.
- Goodput mide peticiones que cumplen el objetivo de servicio.
- La memoria condiciona modelo, lote y contexto; la cuantización ahorra espacio a cambio de calidad.

EJEMPLO

Un servicio de chat fija un objetivo de TTFT menor de 800 ms en p95. Aunque la media sea 400 ms, si el p95 es 1,5 s, la experiencia es mala para una parte importante de usuarios.

CUIDADO

No compares sistemas solo por tokens por segundo. Un sistema rápido en promedio puede ser lento en la cola o costar mucho más.

MÓDULO 09 · LECCIÓN 3 DE 4

Aceleradores, cuantización y serving

3 MIN DE LECTURA 433 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Aceleradores
- Batching y KV cache
- Cuantización y decodificación especulativa
- Elegir hardware y runtime

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

El hardware y el software de serving determinan cuánto rendimiento obtienes por euro. La misma GPU puede servir muy distinto según cómo se agrupe y se gestione la memoria.

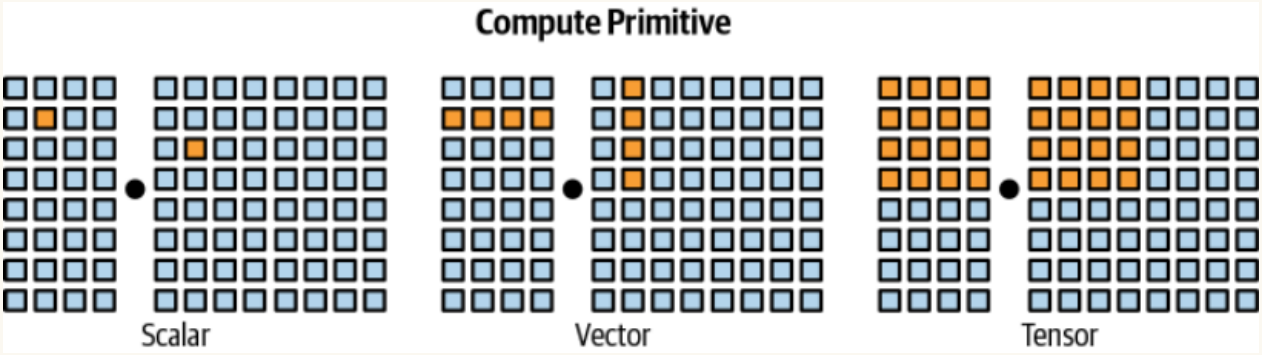


Figura 9-6. Diferentes primitivas de cómputo. Imagen inspirada en Chen et al. (2018).

Aceleradores

Las GPU son el acelerador habitual por su madurez y ecosistema. Las TPU ofrecen alternativas para cargas concretas y pueden ser eficientes a gran escala. También existen aceleradores especializados y CPUs para modelos pequeños o tareas de baja latencia.

La elección depende de disponibilidad, frameworks, coste y experiencia del equipo. El hardware más rápido no siempre es el más rentable si está infrautilizado.

Batching y KV cache

El batching agrupa peticiones para aprovechar la GPU. En inferencia en línea, el batching continuo añade nuevas peticiones mientras otras se procesan. Un lote demasiado grande aumenta la latencia; uno demasiado pequeño desaprovecha el hardware.

El KV cache evita recalcular la atención de tokens anteriores. Es esencial para conversaciones, pero consume memoria. Gestionarlo bien —con límites, compartición o eviction— permite servir más contexto con menos hardware.

Cuantización y decodificación especulativa

La cuantización reduce la precisión de pesos y activaciones: de 16 bits a 8 o 4 bits, por ejemplo. Ahorra memoria y puede acelerar, pero introduce degradación. Evalúa la calidad después de cuantizar; no asumas que es gratis.

La decodificación especulativa usa un modelo pequeño para proponer varios tokens y el modelo grande para verificarlos. Puede acelerar la generación cuando las propuestas aciertan, a cambio de complejidad y memoria.

Elegir hardware y runtime

La GPU adecuada depende del modelo, la longitud de contexto y la concurrencia. Más memoria permite lotes mayores y contextos más largos, pero no siempre reduce el coste por token. La utilización importa: una GPU cara infrautilizada es más costosa que una modesta bien aprovechada.

El runtime también decide. Un servidor con batching continuo y buena gestión de KV cache puede superar a otro con hardware similar. Prueba configuraciones reales y mide calidad después de cuantizar. El mejor hardware es el que cumple el servicio al menor coste total.

IDEAS CLAVE

- GPU, TPU y aceleradores especializados tienen compromisos de coste y ecosistema.
- El batching continuo mejora throughput, pero puede aumentar latencia.
- El KV cache ahorra recomputación y consume memoria.
- Cuantización y decodificación especulativa aceleran con riesgos de calidad y complejidad.

EJEMPLO

Para un asistente con muchas conversaciones cortas, el batching continuo y un KV cache bien dimensionado pueden mejorar el coste por conversación más que cambiar de GPU.

CUIDADO

Cuantizar a 4 bits sin evaluar puede degradar tareas de razonamiento. Mide la calidad en tu dominio, no solo la velocidad.

MÓDULO 09 · LECCIÓN 4 DE 4

Optimizar coste y arquitectura

3 MIN DE LECTURA 441 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Evitar trabajo
- Enrutamiento y tamaño
- Autoscaling y coste total
- Diseñar para el coste

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

La optimización de inferencia no termina en el servidor. La arquitectura del producto decide cuántas llamadas haces, cuánto contexto envías y cuándo puedes evitar el modelo.

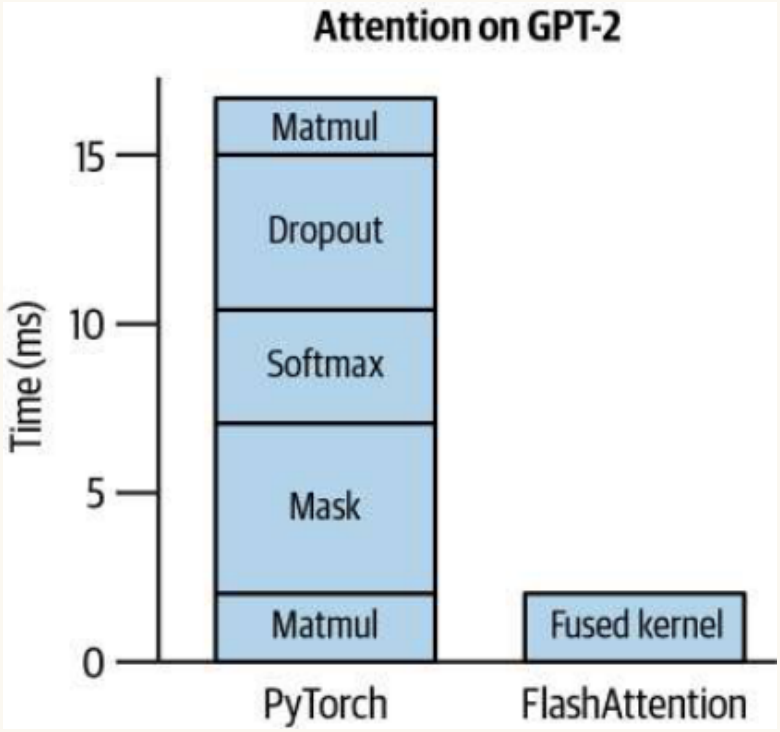


Figura 9-13. FlashAttention es un kernel que fusiona varios operadores comunes. Adaptado de una imagen original con licencia BSD 3-Clause.

Evitar trabajo

La forma más barata de inferencia es no inferir. Caché de respuestas, plantillas para casos frecuentes, reglas para consultas simples y recuperación antes de generar reducen llamadas. También puedes reutilizar el contexto entre turnos en lugar de reconstruirlo.

La caché debe invalidarse cuando cambian los datos. Una respuesta obsoleta puede ser peor que no responder. Define claves, expiración y alcance.

Enrutamiento y tamaño

Un enrutador puede enviar consultas simples a un modelo pequeño y complejas a uno grande. El ahorro depende de la precisión del enrutador: si envía demasiado al modelo grande, pierdes el beneficio; si envía casos difíciles al pequeño, pierdes calidad.

El tamaño del modelo no es la única palanca. Un modelo mediano con buen contexto y herramientas puede superar a uno grande con un prompt pobre. Evalúa combinaciones, no modelos aislados.

Autoscaling y coste total

El autoscaling ajusta capacidad a la demanda, pero arrancar instancias tarda. Para picos predecibles, reserva capacidad; para picos irregulares, usa colas y límites. El coste total incluye hardware, energía, red, almacenamiento y personas.

Define presupuestos por operación y alertas. Un sistema que escala bien pero cuesta diez veces más de lo previsto no es una optimización.

Diseñar para el coste

La arquitectura más barata evita trabajo. Responde desde caché cuando la respuesta es estable, usa reglas para casos simples y reserva el modelo grande para lo difícil. Cada llamada eliminada es ahorro y latencia ganada.

Define el coste por operación y pon alertas. Revisa periódicamente qué porcentaje va a cada modelo y a cada etapa. Si el 80 % del coste viene de un 10 % de casos, ahí está la optimización. La unidad económica —coste por conversación, por documento o por usuario— guía mejor que la factura agregada.

IDEAS CLAVE

- Evitar llamadas con caché, reglas y recuperación suele ser la mayor optimización.
- El enrutamiento ahorra coste si clasifica bien la dificultad.
- El tamaño del modelo no sustituye a un buen contexto y herramientas.
- El coste total incluye hardware, energía, red, almacenamiento y operación.

EJEMPLO

Un asistente responde el 40 % de las preguntas frecuentes desde caché, el 40 % con un modelo pequeño y solo el 20 % con el modelo grande. El coste baja sin sacrificar los casos difíciles.

CUIDADO

La caché de respuestas puede filtrar datos entre usuarios si la clave no incluye el contexto adecuado. Diseña el aislamiento antes de ahorrar.

EVALUACIÓN · MÓDULO 09

Cierre del módulo 09

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

La inferencia tiene dos fases con cuellos de botella distintos. Optimizar exige saber si el límite está en cómputo, memoria o latencia.

TTFT, TPOT, percentiles y goodput describen mejor el servicio que una media de tokens por segundo.

Batching, KV cache, cuantización y enrutamiento son palancas potentes. El coste total y la calidad del producto cierran la decisión.

Evaluación del módulo 09

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 09

¿Qué mide la latencia?

- [ ] A. Cuántas peticiones por segundo se atienden
- [ ] B. El tiempo que tarda una petición en empezar o completar su respuesta
- [ ] C. El tamaño del modelo
- [ ] D. El coste por hora de GPU

Pregunta 02 · Módulo 09

¿Para qué sirve la caché KV?

- [ ] A. Para guardar los pesos del modelo
- [ ] B. Para reutilizar los estados de atención ya calculados y no recalcular el prefijo en cada token
- [ ] C. Para comprimir imágenes
- [ ] D. Para entrenar más rápido

Pregunta 03 · Módulo 09

¿Qué estrategia reduce coste sin cambiar de modelo?

- [ ] A. Aumentar la temperatura
- [ ] B. Procesar en lotes (batching) y usar caché cuando sea posible
- [ ] C. Añadir más capas
- [ ] D. Duplicar el contexto en cada petición

RESPUESTAS Y EXPLICACIONES

01 · Opción B. El tiempo que tarda una petición en empezar o completar su respuesta

Latencia es tiempo de respuesta; throughput es volumen de peticiones por unidad de tiempo.

02 · Opción B. Para reutilizar los estados de atención ya calculados y no recalcular el prefijo en cada token

La caché KV evita recalcular claves y valores de tokens anteriores durante la generación autorregresiva.

03 · Opción B. Procesar en lotes (batching) y usar caché cuando sea posible

El batching mejora la utilización del acelerador y la caché evita cómputo repetido.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Mide la latencia y el throughput de una API de modelo con 10 peticiones. Distingue tiempo hasta el primer token y tiempo total.
2. Compara el coste mensual de dos modelos para 100 000 peticiones con 500 tokens de entrada y 300 de salida.
3. Propón tres optimizaciones para reducir latencia sin cambiar de modelo y ordena su impacto esperado.

10

Arquitectura y feedback

Sistemas completos, seguros y observables

10.1 Los primeros pasos de la arquitectura
10.2 Caché, monitorización y observabilidad
10.3 Orquestación y feedback conversacional
10.4 Diseñar feedback y sus límites
10.5 Cierre del módulo 10

5 lecciones · 0.4 h de lectura 1.655 palabras

MÓDULO 10 · LECCIÓN 1 DE 4

Los primeros pasos de la arquitectura

3 MIN DE LECTURA 416 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Enriquecer el contexto
- Barreras de protección
- Enrutador y puerta de enlace
- Diseñar la puerta de enlace

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Una aplicación de IA no es solo un modelo. La arquitectura añade contexto, controles, enrutamiento y una puerta de entrada común. Es lo que convierte una capacidad en un sistema.

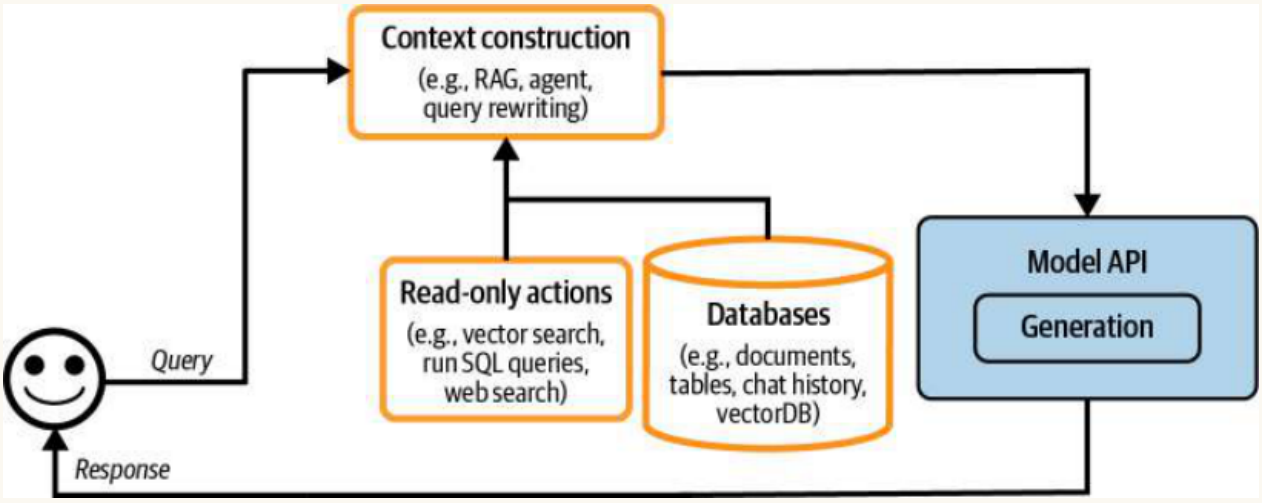


Figura 10-2. Arquitectura de plataforma con construcción de contexto.

Enriquecer el contexto

El primer paso es dar al modelo la información que necesita: documentos, historial, resultados de herramientas o datos del usuario. La construcción de contexto puede incluir recuperación, resumen y selección. Cuanto mejor sea el contexto, menos depende el sistema de la memoria del modelo.

El contexto debe ser relevante y estar delimitado. Añadir todo el historial o todos los documentos aumenta coste y ruido. Selecciona y prioriza.

Barreras de protección

Las barreras de entrada filtran contenido peligroso o fuera de alcance; las de salida detectan datos personales, toxicidad, formato inválido o afirmaciones no respaldadas. También pueden limitar acciones antes de ejecutarlas.

Las barreras no son perfectas y añaden latencia. Deben evaluarse como cualquier otro componente: falsos positivos, falsos negativos y coste. Una barrera que bloquea demasiado puede destruir la experiencia.

Enrutador y puerta de enlace

El enrutador decide qué modelo o ruta usar según la consulta, el coste o la dificultad. La puerta de enlace centraliza autenticación, cuotas, registro, reintentos y fallback entre proveedores. Es una capa de infraestructura que evita repetir lógica en cada aplicación.

La puerta de enlace también permite cambiar de modelo sin tocar el producto. A cambio, añade un punto de fallo y latencia; debe ser observable y resistente.

Diseñar la puerta de enlace

Una puerta de enlace centraliza autenticación, cuotas, reintentos, fallback y registro. Evita que cada equipo implemente su propia lógica y facilita cambiar de proveedor. También permite aplicar políticas comunes de seguridad y coste.

Diseña la puerta para fallar bien: timeouts, reintentos con límite, degradación a un modelo alternativo y errores claros. No debe convertirse en un cuello de botella silencioso. Mide su latencia y su disponibilidad como cualquier otro componente crítico.

IDEAS CLAVE

- El contexto adecuado es la primera palanca de calidad.
- Las barreras de entrada y salida protegen el sistema, con costes y falsos positivos.
- El enrutador optimiza coste y calidad; la puerta de enlace centraliza control.
- Cada capa añade valor y complejidad; hazla observable.

EJEMPLO

Un asistente empresarial recupera políticas, filtra datos personales en la salida, enruta preguntas simples a un modelo pequeño y registra cada llamada en una puerta de enlace común.

> **CUIDADO**
>
> No confíes toda la seguridad a una sola barrera. Combina controles y limita las acciones que el sistema puede ejecutar.

MÓDULO 10 · LECCIÓN 2 DE 4

Caché, monitorización y observabilidad

3 MIN DE LECTURA 381 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Capas de caché
- Registro y trazas
- Métricas y alertas
- Observabilidad útil

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

En producción, lo que no se mide no se puede mejorar. La observabilidad permite detectar fallos, entender costes y comparar versiones.

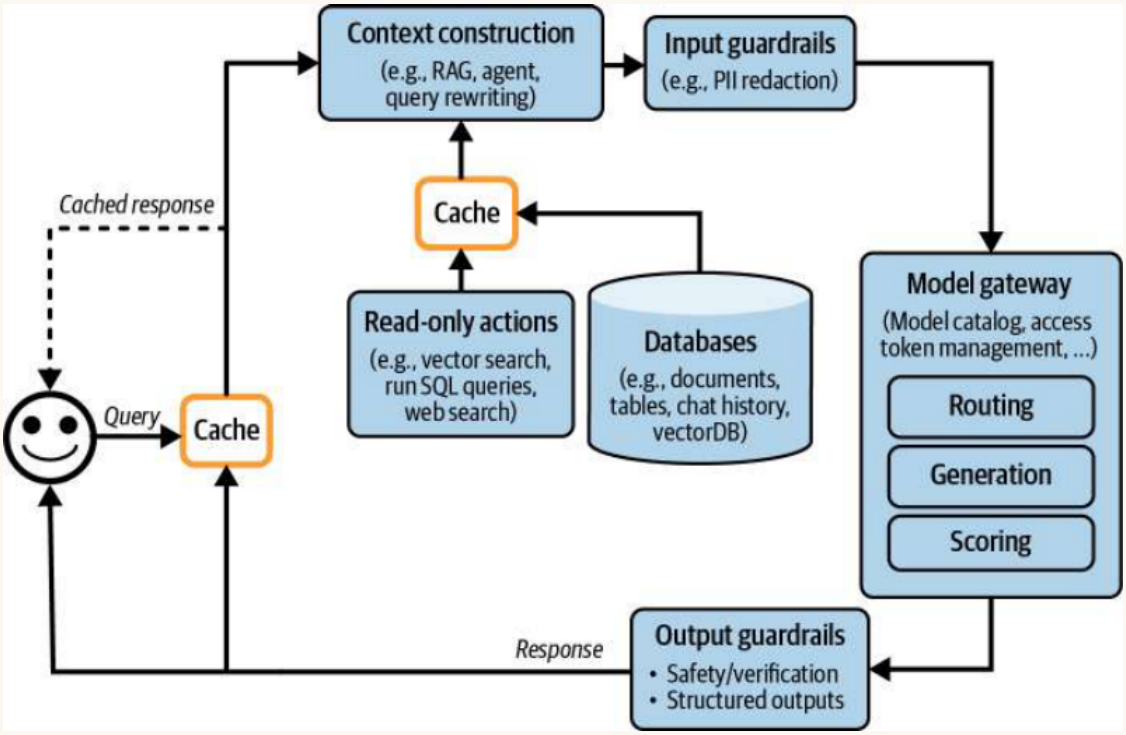


Figura 10-8. Una arquitectura de aplicación de IA con los cachés añadidos.

Capas de caché

Puedes cachear respuestas completas, resultados de recuperación, embeddings o prompts procesados. La caché reduce latencia y coste, pero debe invalidarse cuando cambian los datos o el contexto. Una respuesta personalizada no debe servirse a otro usuario.

Define claves, expiración y alcance. Para contenido sensible, cifra y aísla. La caché es una optimización, no una fuente de verdad.

Registro y trazas

Registra la petición, el modelo, la versión del prompt, los tokens, la latencia, el coste y el resultado. No guardes datos personales sin necesidad. Las trazas permiten reconstruir una interacción y encontrar dónde falló.

En sistemas con agentes, registra también las herramientas y los pasos. Una respuesta incorrecta puede venir de una herramienta, de una decisión de planificación o del modelo final.

Métricas y alertas

Mide calidad, latencia, errores, coste y satisfacción. Las alertas deben ser accionables: un pico de latencia, un aumento de rechazos o una caída de la tasa de resolución. Evita alertas que nadie sabe cómo investigar.

La observabilidad también alimenta la evaluación. Los casos reales que fallan son los mejores candidatos para el próximo conjunto de pruebas.

Observabilidad útil

No registres todo por si acaso. Define qué preguntas quieres responder: ¿por qué fue lenta esta petición?, ¿por qué costó tanto?, ¿qué versión respondió?, ¿dónde se perdió el contexto? A partir de ahí elige trazas, métricas y logs.

En sistemas con recuperación y agentes, una traza debe mostrar los pasos, las herramientas y los tiempos. La observabilidad también debe respetar la privacidad: anonimiza, muestrea y limita la retención. Ver mucho no sirve si no puedes investigar.

IDEAS CLAVE

- La caché reduce coste y latencia, pero necesita invalidación y aislamiento.
- Registra modelo, prompt, tokens, latencia, coste y resultado.
- En agentes, traza también herramientas y pasos.
- Las alertas deben ser accionables y alimentar la evaluación.

EJEMPLO

Un equipo detecta que el p95 de latencia subió tras un cambio de prompt. La traza muestra que el nuevo prompt genera respuestas más largas; la métrica de coste también lo confirma.

CUIDADO

Registrar todo sin política de privacidad crea un riesgo legal. Guarda lo necesario y anonimiza.

MÓDULO 10 · LECCIÓN 3 DE 4

Orquestación y feedback conversacional

3 MIN DE LECTURA 413 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Flujos y estado
- Feedback del usuario
- Conversación y confianza
- Orquestar sin romper

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

Una aplicación real coordina varios pasos: recuperar, generar, validar, llamar herramientas y responder. La orquestación decide el orden y qué hacer cuando algo falla.

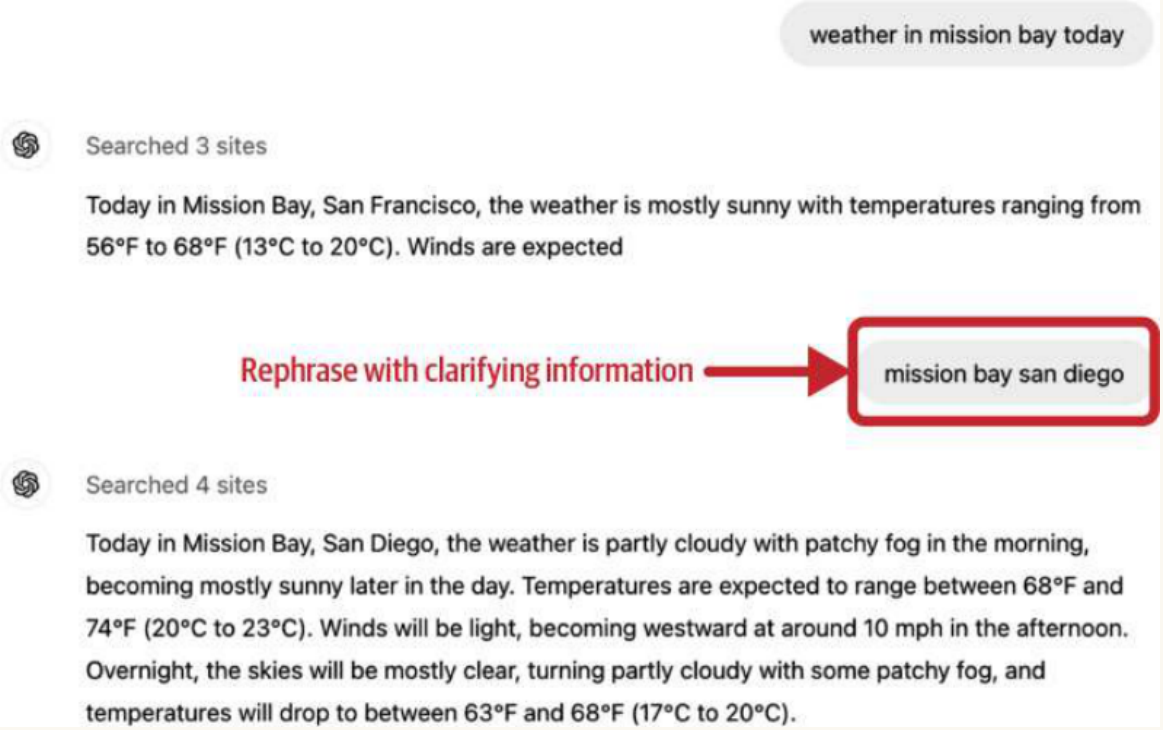


Figura 10-12. Debido a que el usuario finaliza la generación antes de tiempo y vuelve a formular la pregunta, puede inferirse que el modelo malinterpretó la intención de la solicitud original.

Flujos y estado

Un flujo puede ser secuencial, en paralelo o condicional. La paralelización reduce latencia cuando los pasos son independientes; la secuencia es necesaria cuando un paso depende del anterior. El estado guarda el contexto entre turnos y entre pasos.

El estado debe tener un alcance claro. Guardar toda la conversación puede ser caro y ruidoso; guardar solo lo esencial puede perder continuidad. Define qué información necesita cada paso.

Feedback del usuario

El feedback explícito —pulgares, valoraciones, correcciones— es fácil de recoger pero escaso y sesgado. El implícito —copiar, editar, abandonar, volver a preguntar— es más abundante pero ambiguo. Combina ambos con cuidado.

Un «me gusta» no siempre significa calidad; un abandono no siempre significa fallo. Interpreta el feedback junto con el contexto y úsalo para priorizar, no como verdad absoluta.

Conversación y confianza

La experiencia incluye latencia, claridad, capacidad de corregir y transparencia sobre límites. Un asistente que muestra sus fuentes, pide confirmación y reconoce incertidumbre genera más confianza que uno que siempre responde con seguridad.

Diseña los estados de error: qué ve el usuario cuando el modelo falla, cuando no hay datos o cuando la acción no está permitida.

Orquestar sin romper

Un flujo con varios pasos necesita idempotencia, timeouts y manejo de fallos parciales. Si una herramienta falla, el sistema debe decidir si reintenta, degrada o pide ayuda. Si el usuario reintenta, no debería duplicar una acción ya ejecutada.

La experiencia también forma parte de la orquestación: muestra progreso, explica la espera y permite corregir. Un flujo lento pero transparente genera más confianza que uno rápido que falla en silencio.

IDEAS CLAVE

- La orquestación define orden, paralelización, estado y manejo de errores.
- El estado debe guardar lo necesario, no toda la conversación.
- Combina feedback explícito e implícito, interpretando su contexto.
- La confianza se construye con transparencia, fuentes y estados de error claros.

EJEMPLO

Un asistente de viajes consulta vuelos y hoteles en paralelo, guarda las preferencias del usuario y pide confirmación antes de reservar. Si una fuente falla, muestra el resultado parcial.

> **CUIDADO**
>
> Un flujo con muchos pasos puede ser más frágil y difícil de depurar. Añade orquestación solo cuando el producto la necesita.

MÓDULO 10 · LECCIÓN 4 DE 4

Diseñar feedback y sus límites

3 MIN DE LECTURA 396 PALABRAS 4 APARTADOS

EN ESTA LECCIÓN

- Tipos de feedback
- Sesgos y bucles
- Gobernanza y límites
- Gobernar el feedback

Al terminar, continúa con la lección siguiente. El índice general está al principio del manual.

El feedback puede mejorar el producto o crear un bucle que refuerza errores. Diseñarlo bien exige pensar en quién responde, qué mide y cómo se usa.

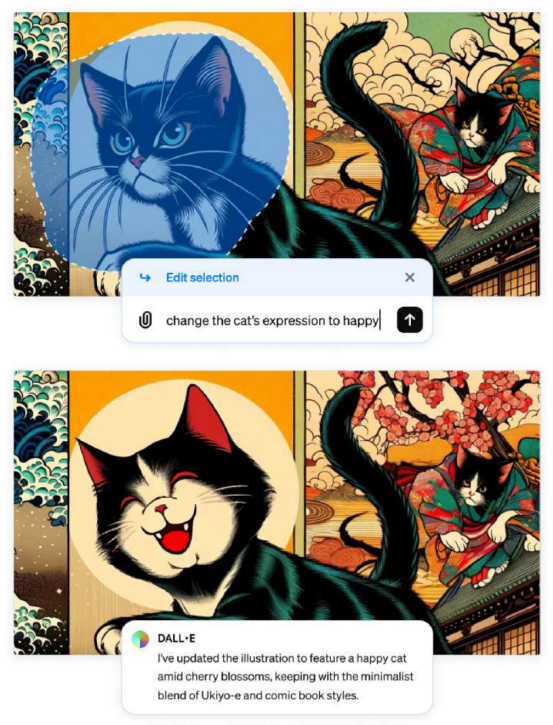


Figura 10-14. Un ejemplo de cómo funciona el inpainting en DALL-E. Imagen de OpenAI.

Tipos de feedback

El feedback directo pregunta al usuario; el indirecto observa su comportamiento. El primero es explícito pero costoso; el segundo es abundante pero ruidoso. Las correcciones de texto, las ediciones y las citas de error son señales especialmente valiosas.

También existe feedback del sistema: validadores, pruebas y métricas automáticas. No es opinión humana, pero detecta fallos objetivos a escala.

Sesgos y bucles

Quienes responden no representan a todos los usuarios. Un sistema puede recibir feedback de los más satisfechos y volverse peor para los demás. Además, si el modelo aprende de sus propias salidas, puede amplificar errores y perder diversidad.

Para evitarlo, mantén una evaluación independiente, muestrea a usuarios que no responden y revisa los cambios antes de entrenar con feedback. No optimices una sola métrica.

Gobernanza y límites

Define quién puede cambiar el modelo, qué datos se usan y cómo se audita. El feedback puede contener información personal o contenido abusivo; trátalo con las mismas reglas que cualquier dato.

La transparencia importa: explica qué hace el sistema, qué limitaciones tiene y cómo se usa el feedback. La confianza a largo plazo vale más que una mejora marginal.

Gobernar el feedback

Define qué feedback se recoge, con qué consentimiento y durante cuánto tiempo. No todo comentario debe convertirse en dato de entrenamiento. Filtra información personal, contenido abusivo y ejemplos ambiguos antes de usarlos.

Establece un comité o un responsable que revise los cambios basados en feedback. Mantén una evaluación independiente para detectar si el sistema mejora de verdad o solo aprende a agradar a quien responde. La gobernanza protege al usuario y también al equipo.

IDEAS CLAVE

- Combina feedback directo, indirecto y automático; cada uno tiene sesgos.
- Los usuarios que responden no representan a todos.
- Evita bucles que refuerzan errores; mantén evaluación independiente.
- Gobierna el uso del feedback y sé transparente sobre límites.

EJEMPLO

Un asistente de redacción recoge ediciones de los usuarios. Antes de ajustar el modelo con ellas, filtra datos personales, revisa una muestra y compara contra un conjunto de evaluación fijo.

CUIDADO

Entrenar con todo el feedback sin filtrar puede premiar respuestas populares pero incorrectas. La señal no es la verdad.

EVALUACIÓN · MÓDULO 10

Cierre del módulo 10

Tres preguntas para comprobar lo esencial del módulo y una serie de ejercicios para llevarlo a tu propio proyecto. En el curso, la evaluación se supera con dos respuestas correctas de tres.

Resumen del módulo

La arquitectura convierte el modelo en sistema: contexto, barreras, enrutamiento y puerta de enlace.

La observabilidad y la caché permiten operar con coste y calidad controlados. Sin trazas, no hay diagnóstico.

El feedback es útil pero sesgado. Diseña el bucle con evaluación independiente, gobernanza y transparencia.

Evaluación del módulo 10

Marca una opción por pregunta. Las respuestas razonadas están en la página siguiente; no las mires hasta haber intentado las tres.

Pregunta 01 · Módulo 10

¿Qué es un guardrail?

- [ ] A. Un tipo de GPU
- [ ] B. Una barrera que filtra entradas o salidas peligrosas, ofensivas o fuera de política
- [ ] C. Un formato de prompt
- [ ] D. Un benchmark de seguridad

Pregunta 02 · Módulo 10

¿Qué diferencia hay entre monitorización y observabilidad?

- [ ] A. Son sinónimos
- [ ] B. Monitorizar vigila métricas conocidas; la observabilidad permite investigar comportamientos emergentes
- [ ] C. La observabilidad solo mide coste
- [ ] D. La monitorización solo se usa en entrenamiento

Pregunta 03 · Módulo 10

¿Para qué sirve el feedback de usuario?

- [ ] A. Solo para marketing
- [ ] B. Para detectar fallos, priorizar mejoras y crear datos de evaluación y entrenamiento
- [ ] C. Para sustituir los tests
- [ ] D. Para reducir el contexto

RESPUESTAS Y EXPLICACIONES

01 · Opción B. Una barrera que filtra entradas o salidas peligrosas, ofensivas o fuera de política

Los guardrails aplican políticas de seguridad antes y después del modelo.

02 · Opción B. Monitorizar vigila métricas conocidas; la observabilidad permite investigar comportamientos emergentes

La observabilidad añade trazas, contexto y capacidad de formular nuevas preguntas sobre el sistema.

03 · Opción B. Para detectar fallos, priorizar mejoras y crear datos de evaluación y entrenamiento

El feedback es una señal valiosa, pero debe interpretarse con cuidado por sus sesgos y baja representatividad.

Ejercicios para practicar

No hace falta entregarlos. El objetivo es convertir la lectura en decisiones concretas sobre tu propio sistema.

1. Dibuja la arquitectura completa de tu aplicación: contexto, guardrails, router, caché, modelo, observabilidad y feedback.
2. Define cinco métricas de observabilidad y qué alerta dispararías con cada una.
3. Diseña el mecanismo de feedback de tu producto: cuándo preguntar, qué opciones dar y cómo usar los datos sin sesgarte.

REFERENCIA RÁPIDA

Glosario

Los términos que aparecen una y otra vez en el curso, explicados en una frase. Úsalo como repaso antes de una evaluación o como mapa cuando una lección introduzca vocabulario nuevo.

Agente
Sistema que razona, usa herramientas y observa resultados en un bucle para completar una tarea.

Ajuste fino (fine-tuning)
Entrenar un modelo base con datos adicionales para especializarlo en una tarea o dominio.

Alucinación
Contenido inventado o incorrecto que el modelo presenta con aparente seguridad.

Autosupervisión
Entrenamiento en el que la propia estructura de los datos genera la señal de aprendizaje, sin etiquetas manuales.

Benchmark
Prueba estandarizada para comparar modelos bajo las mismas condiciones.

Caché KV
Memoria que reutiliza los estados de atención ya calculados durante la generación autorregresiva.

Chunking
Fragmentación de documentos en unidades más pequeñas para indexarlas y recuperarlas en RAG.

Contexto
Información que se incluye en la ventana de entrada del modelo para condicionar su respuesta.

Cuantización
Reducir la precisión numérica de los pesos para disminuir memoria y coste de inferencia.

Datos sintéticos
Datos generados artificialmente, a menudo por otro modelo, para entrenar o evaluar.

Destilación
Entrenar un modelo pequeño para imitar el comportamiento de un modelo grande.

Embedding
Representación vectorial de un texto que captura su significado semántico.

Evaluación exacta
Métodos que comparan la salida del modelo con una referencia mediante reglas o similitud.

Few-shot
Incluir unos pocos ejemplos de entrada y salida en el prompt.

Guardrail
Barrera que filtra entradas o salidas para evitar usos peligrosos o fuera de política.

Inferencia
Etapa en la que el modelo genera respuestas para los usuarios.

Inyección de prompt
Ataque que introduce instrucciones maliciosas en contenido que el modelo procesa.

Latencia
Tiempo que tarda una petición en empezar o completar su respuesta.

LLM
Modelo de lenguaje grande entrenado con enormes volúmenes de texto.

LoRA
Técnica de ajuste eficiente que entrena matrices de bajo rango y congela el modelo base.

Modelo base

Modelo grande preentrenado y adaptable que sirve como punto de partida para muchas aplicaciones.

Perplejidad

Métrica derivada de la entropía cruzada que indica la incertidumbre del modelo ante un texto.

Postentrenamiento

Fase que enseña al modelo preentrenado a seguir instrucciones y alinearse con preferencias.

Preentrenamiento

Entrenamiento inicial a gran escala con datos sin etiquetar.

Prompt

Instrucción y contexto que se proporcionan al modelo para obtener una respuesta.

Prompt de sistema

Instrucción que define el rol y las reglas generales del modelo durante la conversación.

RAG

Generación aumentada por recuperación: buscar información externa y añadirla al contexto antes de responder.

Temperatura

Parámetro de muestreo que controla la aleatoriedad de la generación.

Throughput

Número de peticiones o tokens que el sistema puede procesar por unidad de tiempo.

Token

Unidad mínima de texto que procesa un modelo: carácter, palabra o fragmento.

Ventana de contexto

Cantidad máxima de tokens que el modelo puede considerar en una petición.

PARA IMPRIMIR

Lista de progreso

Marca cada lección al terminarla. Si estudias en papel, esta página es tu mapa: al final del curso deberías tener las 50 casillas completas.

01 · Introducción a la ingeniería de IA

Qué cambió con los modelos base y cómo se construye con ellos

- [ ] 01.1 · Del modelo de lenguaje al modelo base — 3 min
- [ ] 01.2 · El stack de ingeniería de IA — 3 min
- [ ] 01.3 · Elegir un caso de uso que funcione — 3 min
- [ ] 01.4 · Expectativas, mantenimiento y producción — 3 min
- [ ] 01.5 · Cierre del módulo 01 — 10 min

02 · Cómo funcionan los modelos base

Datos, transformer, escala, postentrenamiento y muestreo

- [ ] 02.1 · Datos, tokens y autosupervisión — 3 min
- [ ] 02.2 · La arquitectura transformer — 3 min
- [ ] 02.3 · Escala, parámetros y postentrenamiento — 3 min
- [ ] 02.4 · Muestreo, probabilidad y salidas estructuradas — 3 min
- [ ] 02.5 · Cierre del módulo 02 — 10 min

03 · Metodología de evaluación

Cómo medir lo que un modelo realmente sabe

- [ ] 03.1 · Por qué evaluar IA es distinto — 3 min
- [ ] 03.2 · Evaluación exacta y semántica — 3 min
- [ ] 03.3 · La IA como juez — 3 min
- [ ] 03.4 · Benchmarks y selección de modelos — 3 min
- [ ] 03.5 · Cierre del módulo 03 — 10 min

04 · Evaluar sistemas de IA

Criterios, coste, latencia y pipelines de evaluación

- [ ] 04.1 · Qué hace buena a una aplicación — 3 min
- [ ] 04.2 · Instrucciones, coste y latencia — 3 min
- [ ] 04.3 · Construir o comprar — 3 min
- [ ] 04.4 · Diseñar un pipeline de evaluación — 3 min
- [ ] 04.5 · Cierre del módulo 04 — 10 min

05 · Ingeniería de prompts

Instrucciones claras, contexto suficiente y defensas

- [ ] 05.1 · Anatomía de un prompt — 3 min
- [ ] 05.2 · Instrucciones claras y contexto suficiente — 3 min
- [ ] 05.3 · Iterar, evaluar y versionar prompts — 3 min
- [ ] 05.4 · Ataques y defensas — 3 min
- [ ] 05.5 · Cierre del módulo 05 — 10 min

06 · RAG y agentes

Conocimiento externo, herramientas y acción autónoma

- [ ] 06.1 · Qué es RAG y cómo se monta — 3 min
- [ ] 06.2 · Recuperación: dispersa, densa e híbrida — 3 min
- [ ] 06.3 · Agentes: herramientas, planificación y memoria — 3 min
- [ ] 06.4 · Evaluar y optimizar RAG y agentes — 3 min
- [ ] 06.5 · Cierre del módulo 06 — 10 min

07 · Ajuste fino

Cuándo especializar un modelo y cómo hacerlo sin arruinarte

- [ ] 07.1 · Qué es el ajuste fino y cuándo usarlo — 3 min
- [ ] 07.2 · Memoria, PEFT y LoRA — 3 min
- [ ] 07.3 · Tácticas de entrenamiento — 3 min

- [ ] 07.4 · Evaluar y desplegar un modelo ajustado — 3 min
- [ ] 07.5 · Cierre del módulo 07 — 10 min

08 · Ingeniería de datasets

Calidad, cobertura, anotación y datos sintéticos

- [ ] 08.1 · Calidad antes que cantidad — 3 min
- [ ] 08.2 · Adquisición, anotación y catálogos — 3 min
- [ ] 08.3 · Datos sintéticos y destilación — 3 min
- [ ] 08.4 · Procesar el dataset — 3 min
- [ ] 08.5 · Cierre del módulo 08 — 10 min

09 · Optimización de inferencia

Latencia, throughput, memoria y coste

- [ ] 09.1 · Qué significa optimizar la inferencia — 3 min
- [ ] 09.2 · Métricas: latencia, throughput y memoria — 3 min
- [ ] 09.3 · Aceleradores, cuantización y serving — 3 min
- [ ] 09.4 · Optimizar coste y arquitectura — 3 min
- [ ] 09.5 · Cierre del módulo 09 — 10 min

10 · Arquitectura y feedback

Sistemas completos, seguros y observables

- [ ] 10.1 · Los primeros pasos de la arquitectura — 3 min
- [ ] 10.2 · Caché, monitorización y observabilidad — 3 min
- [ ] 10.3 · Orquestación y feedback conversacional — 3 min
- [ ] 10.4 · Diseñar feedback y sus límites — 3 min
- [ ] 10.5 · Cierre del módulo 10 — 10 min

SOBRE ESTA EDICIÓN

Colofón

Esta edición esencial reúne el contenido del curso AI Engineering en español, una adaptación editorial y educativa del libro AI Engineering: Building Applications with Foundation Models, de Chip Huyen. Los textos se reorganizaron en diez módulos, se tradujeron al español y se acompañaron de evaluaciones, ejercicios y un glosario para uso de estudio.

El diseño, la maquetación y los materiales de evaluación son originales de esta edición. Las figuras pertenecen a sus autores originales y se incluyen con fines educativos. Si detectas una errata o quieres proponer una mejora, abre una incidencia en el repositorio del curso.

Tipografías: Fraunces (títulos) e Inter (texto), distribuidas bajo SIL Open Font License. Composición en HTML y CSS, renderizada con WeasyPrint. Edición 2026, en español.

Uso educativo. No se permite la reventa de este material. Para citar: «AI Engineering · Edición esencial, 2026».
Basado en el libro de Chip Huyen.